The Cost of Custody

Praneeth Gunasekaran

Dissertation submitted to the Faculty of the

Virginia Polytechnic Institute and State University

in partial fulfillment of the requirements for the degree of

Master of Science

in

Computer Science

Shinichiro Matsuo, Chair

Wenjing Lou, Co-chair

Na Meng

April 29, 2026

Arlington, Virginia

Keywords: Bitcoin, Script extensions, vault custody, covenants, formal verification, fee dynamics

Copyright 2026, Praneeth Gunasekaran

The Cost of Custody

Praneeth Gunasekaran

(ABSTRACT)

Four Bitcoin Script extension proposals—`OP_CTV` (BIP-119), `OP_VAULT` (BIP-345), `OP_CCV` (BIP-443), and the `OP_CAT` + `OP_CHECKSIGFROMSTACK` composition (BIP-347 + BIP-348)—enable self-custody vaults that enforce spending conditions beyond simple signature checks, but no prior work compares them empirically. We build all four vault implementations, run 15 experiments on Bitcoin regtest, and additionally implement Simplicity on Elements regtest as a cross-substrate reference point. We formalise a defender-loss metric $\mathcal{L}$ and, threat by threat, characterise how $\mathcal{L}$ varies with the fee rate across designs. Two results constrain the design space: (1) a Griefing–Safety Incompatibility (no vault simultaneously achieves permissionless recovery and griefing resistance), and (2) a No Pareto-Optimal Design theorem (for every design, there exists a threat model and fee rate at which another design has strictly lower defender loss). We measure OP_VAULT and CCV watchtower splitting capacity (3,427 and 6,172 splits/block respectively), show that defender-side batched recovery—spec-level in BIP-345 §"Batching" and BIP-443 aggregation—approximately doubles the dust-threshold fee rate and flips the CCV/OP_VAULT ordering under watchtower exhaustion, and provide the first parameterised attack cost functions across fee environments. Alloy bounded model checking verifies fund conservation, recovery liveness, and no-extraction properties. Simplicity's federated-sidechain threat model is treated separately. The framework, measurements, and formal models are open-source.

The Cost of Custody

Praneeth Gunasekaran

(GENERAL AUDIENCE ABSTRACT)

Bitcoin users who store their own cryptocurrency face a fundamental problem: if someone steals their private key, the funds are gone. *Vault* protocols are a proposed solution—specially-scripted Bitcoin transactions that enforce a time delay before funds can be moved, giving the owner a window to recover stolen coins. Four different vault designs have been proposed for Bitcoin, each with different tradeoffs. We built all four, tested them side-by-side, and discovered that no single design is safest against every threat: the right choice depends on which attack you most fear and on the transaction fee environment. We also found that a batching optimisation built into two of the designs roughly doubles the fee rate at which a common denial-of-service attack becomes practical, and unexpectedly reverses the relative safety of those two designs. We additionally studied Simplicity, a fundamentally different scripting language deployed on a Bitcoin-adjacent sidechain, as a point of comparison. Our measurements and testing framework are publicly available.

Dedication

To my parents and family, for their unconditional love and support, and to the Bitcoin open-source community, whose relentless pursuit of self-sovereign custody inspired this work.

Acknowledgments

I would like to express my sincere gratitude to my advisor, Dr. Shinichiro Matsuo, for his guidance, encouragement, and support throughout this research. I am also grateful to my co-advisor, Dr. Wenjing Lou, and committee member, Dr. Na Meng, for their valuable feedback and time reviewing this thesis.

I owe a debt to the Bitcoin covenant developers whose work made this research possible. Jeremy Rubin provided feedback on the overall research direction and helped shape the comparative methodology. James O'Beirne gave detailed feedback on the OP_VAULT experiments, including insights on reducing fee overhead in the vault lifecycle. Salvatore Ingala's OP_CHECKCONTRACTVERIFY design and pymatt framework were essential to the CCV experiments. Andrew Poelstra's CAT and Schnorr Tricks work laid the foundation for the CAT+CSFS vault. The Blockstream Research team—particularly Russell O'Connor (Simplicity) and the Simplex SDK developers—helped narrow down the research ideas and provided the language and tooling for the Simplicity vault implementation.

I would also like to thank the Commonwealth Cyber Initiative for supporting this work.

Finally, I thank my friends and family for their patience, support, and encouragement throughout my graduate studies.

Contents

List of Figures xiii

List of Tables xvii

List of Abbreviations xx

1 Introduction 1

1.1 Main Results 2

1.2 Contributions 3

1.3 Scope and Limitations 5

1.4 Thesis Organization 6

2 Background 7

2.1 Bitcoin Script and Its Limitations 7

2.2 Transaction Introspection 9

2.3 Covenants: Generic and Purpose-Built 10

2.3.1 Generic Covenants 10

2.3.2 Purpose-Built Covenants 11

2.3.3 The Recursive Covenant Question 11

2.4 Vault Lifecycle and Threat Model 12

2.4.1 Threat Model Assumptions (Kerckhoffs) 14

2.4.2 Threat Taxonomy 14

2.5 OP_CHECKTEMPLATEVERIFY (CTV) 15

2.6 OP_CHECKCONTRACTVERIFY (CCV) 16

2.7 OP_VAULT (BIP-345) 17

2.8 CAT+CSFS (BIP-347 + BIP-348) 19

2.9 Simplicity on Elements: An Alternative Script Primitive 22

2.10 The Design-Space Axes 23

2.10.1 Axis Definitions 24

2.10.2 Four-Covenant Position Table 25

2.10.3 Structural Dependencies Between Axes 26

3 Measurement Methodology **28**

3.1 Framework Architecture 28

3.1.1 Adapter Interface 29

3.1.2 Capability-Based Dispatch 30

3.1.3 Implementation Provenance 30

3.2 Primary Metric: Virtual Size 31

3.2.1 Fee Projection Model 32

3.3 Rationality and Scope Conventions 33

3.4 Regtest Validity 34

3.5 Experiment Catalog 35

3.6 Node Infrastructure 37

4 Empirical Results **38**

4.1 Lifecycle Cost Measurements 38

4.2 OP_VAULT Vsize Correction 39

4.3 Batching Efficiency 40

4.4 Revault Amplification 42

4.5 Address Reuse 43

4.6 Fee Sensitivity Projections 43

4.7 Synthesis: What the Measurements Imply 45

4.8 Case Study: Simplicity on Elements 45

5 Security Analysis **47**

5.1 Griefing–Safety Impossibility 47

5.2 Adversary Advantage and Structural Analysis 49

5.2.1 Framework 50

5.2.2 Design-Space Decomposition 51

5.2.3 Feature–Vulnerability Implications 52

5.2.4 Structural Consequence 54
5.2.5 Relationship to Proposition 5.2 55
5.2.6 Fee-Rate Sensitivity of TM4 55
5.3 Class Z1: Fee-Channel Pinning (TM1) 59
5.4 Class Z3: Per-UTXO Recovery-Cost Scaling (TM4) 62
5.4.1 Batched Defender Strategy 63
5.5 Class Z2: Permissionless Griefing (TM2) 65
5.6 Class Z4: Hot-Key Theft via Destination Substitution (TM3) 66
5.7 Class Z5: Cold-Key Theft via Unconstrained Recovery (TM11) 68
5.8 Projection onto the (g, b) Sublattice 70
5.9 Threat Model Matrix 72
5.10 Deployment Guidance 72
5.11 Class Z6: Mode/Parameter Bypass (TM8) 74
5.11.1 Susceptibility 75
5.11.2 Immunity Derivations 75
5.11.3 Framing: Specified Behaviour, Not a Novel Vulnerability 75
5.11.4 Rationality and Scope 76
5.12 Rationality Blocks for Classes Z1–Z5 77
5.12.1 Class Z1 (Fee-Channel Pinning) 77
5.12.2 Class Z2 (Permissionless Griefing) 78

5.12.3 Class Z3 (Per-UTXO Recovery-Cost Scaling) 78

5.12.4 Class Z4 (Hot-Key Theft via Destination Substitution) 79

5.12.5 Class Z5 (Cold-Key Theft via Unconstrained Recovery) 80

6 Formal Verification **82**

6.1 Alloy and Bounded Model Checking 82

6.2 Properties and Results 83

6.3 Three-Layer Evidence Methodology 85

7 Discussion **87**

7.1 Limitations 87

7.1.1 Regtest Environment 87

7.1.2 Reference Implementations 88

7.1.3 Rational Adversary Assumption 88

7.1.4 Threats Not Modeled 89

7.2 Mitigations 90

7.2.1 TRUC/v3 Transactions 90

7.2.2 Batched Recovery 91

7.2.3 Poelstra Reset Model 91

7.3 Open Problems 91

8 Related Work **94**

8.1 Vault Custody Protocols 94

8.2 Covenant Proposals and Analysis 95

8.3 Formal Verification of Bitcoin Contracts 96

8.4 Layer-Two Security and Watchtower Economics 98

8.5 Qualitative Comparisons 99

8.6 Summary of Positioning 99

9 Conclusion **101**

9.1 Future Work 103

Bibliography **105**

Appendices **111**

Appendix A Alloy Model Details **112**

A.1 Model Architecture 112

A.2 Per-Covenant Detailed Results 114

A.3 Limitations of the Formal Models 115

Appendix B Simplicity on Elements: The Unoccupied Optimal Corner **116**

B.1 Scope and Motivation 116

B.2 Design-Space Position 117

B.3 Lifecycle Measurements 118

B.4 Foundations and Substrate 118

B.5 Future Work: Porting the Optimal Corner to Bitcoin 119

Appendix C Rationality Block Template and Full Catalogue **120**

C.1 Purpose 120

C.2 Class Z1: Fee-Channel Pinning 120

C.3 Class Z2: Permissionless Griefing 123

C.4 Class Z3: Per-UTXO Recovery-Cost Scaling 124

C.5 Class Z4: Hot-Key Theft via Destination Substitution 126

C.6 Class Z5: Cold-Key Theft via Unconstrained Recovery 128

C.7 Class Z6: Mode/Parameter Bypass 129

List of Figures

2.1 Vault lifecycle state machine. All covenant designs implement this four-state model. The trigger-to-withdrawal transition enforces a relative timelock (CSV) that creates the watchtower monitoring window. Recovery is available from both the Funded and Triggered states. 13

2.2 CTV vault mechanism. The vault UTXO is encumbered by a bare CTV hash H that commits to the unvault transaction's entire output structure; any spend whose template hash differs from H fails. The unvault UTXO is a P2WSH script with two Taproot leaves: a hot-key withdrawal gated by `OP_CSV`, and a CTV-committed cold sweep. A 3,300-sat anchor output enables CPFP fee bumping. 16

2.3 CCV vault mechanism. The vault UTXO is a Taproot with two leaves: a *trigger* leaf containing `OP_CCV`, and a keyless *recover* leaf. Spending the trigger leaf invokes the per-output `OP_CHECKCONTRACTVERIFY` check, whose mode byte selects whether to verify the scriptPubKey, the amount, both, or the current input. Taptree preservation enables recursion, supporting partial withdrawal via `trigger_and_revault`. Spending the recover leaf is permissionless and sends funds to the pre-committed address without a signature. 18

2.4 OP_VAULT mechanism. The opcode pair `OP_VAULT` and `OP_VAULT_RECOVER` encodes vault-specific semantics (amount preservation, auto-revault, authorized recovery) directly. Three keys are separated: the *trigger key* initiates withdrawals, the *recoveryauth key* authorizes recovery, and the *recovery address* (embedded in the Taproot internal-key tweak) is the pre-committed destination. Value preservation requires a fee-wallet input on every trigger and recovery transaction (∼80–90 vB overhead). 20

2.5 CAT+CSFS vault mechanism. The Taproot script has two leaves. **Hot-key leaf (top):** the spender provides sighash preimage fields; `OP_CAT` concatenates them; `OP_CSFS` verifies the Schnorr signature against this stack-assembled preimage; `OP_CHECKSIG` verifies the *same* signature against the real transaction sighash. Both succeed only if the two preimages agree, binding outputs through `hashOutputs`. **Recovery leaf (bottom):** bare `cold_pk` `OP_CHECKSIG`—no CAT, no CSFS, no output binding. 21

2.6 Simplicity vault mechanism on Elements. At compile time, the program (a DAG of combinators) embeds an expected outputs-hash H as a parameter. At spend time, `jet::outputs_hash()` computes the Merkle hash of the spending transaction's outputs. If the computed hash matches H, the program accepts; otherwise the program fails at the consensus level. Both trigger and recovery leaves enforce this jet—unlike CAT+CSFS, where recovery is unconstrained. 23

3.1 Framework architecture. The experiment runner invokes vault operations through the `VaultAdapter` interface. Each adapter wraps a covenant-specific implementation and communicates with its required Bitcoin or Elements node variant. CTV and CAT+CSFS share the Bitcoin Inquisition node. 29

5.1 Fee-dependent security inversion. CTV fee pinning (blue) executes in one block regardless of fee rate. CCV per-UTXO recovery-cost scaling (red) requires progressively fewer blocks as fees rise, because each split creates a larger dust threshold. Below ∼50 sat/vB, exhaustion requires hundreds of blocks (infeasible). Above $r^* \approx 66$ sat/vB, exhaustion completes within a few blocks, making it practical. The crossover defines where the safety ranking inverts. . 59

5.2 Fee pinning attack against CTV (**TM1**). ① Attacker broadcasts the unvault, starting the CSV window. ② Attacker chains 25 dust-rate descendants off the anchor output, filling the mempool descendant limit. ③ The watchtower's CPFP recovery is rejected by mempool policy (adding it would exceed the 25-descendant cap). ④ When CSV expires, the attacker's pre-committed withdrawal confirms. 60

5.3 Watchtower exhaustion via `trigger_and_revault` (**TM4**). The attacker (holding the trigger key) repeatedly splits the vault into smaller UTXOs. As UTXOs shrink, the per-recovery fee ($\text{vsize}_{\text{rec}} \times r$) consumes an increasing fraction of each UTXO's value. Once a UTXO's value falls below $\text{vsize}_{\text{rec}} \times r$, the watchtower cannot economically justify recovery—those funds are effectively lost. 62

5.4 Recovery griefing loop (**TM2**) for CCV. The attacker triggers permissionless recovery (122 vB); the owner must re-deposit and re-trigger (454 vB) to resume withdrawal; the attacker invokes recovery again. The per-round cost asymmetry $\gamma = 0.27$ strongly favors the attacker. 66

5.5 Trigger key theft race (**TM3**). The attacker broadcasts an unvault (upper lane). The watchtower detects it at time δ and broadcasts recovery (lower lane). The outcome hinges on whether recovery confirms before CSV expiry: safe if recovery wins, DoS if tied, theft if the attacker successfully pins the watchtower's transaction (combining TM3 with TM1). 68

5.6 CAT+CSFS asymmetric key safety. **Left:** hot key compromise leaks only the trigger authority—the dual CSFS + CHECKSIG binding forces outputs to match the embedded `sha_single_output` constant (**TM3** reduces to griefing). **Right:** cold key compromise yields total theft (**TM11**): the recovery leaf is bare `OP_CHECKSIG`—no timelock, no introspection, no destination constraint. The attacker sweeps funds to any address in one 125 vB transaction. 69

5.7 Projection of the covenant design space onto the (g, b) sublattice. Three cells are occupied by the four proposed Bitcoin Script extensions; the fourth is structurally unreachable. CAT+CSFS appears in two cells by Remark 5.12. 71

6.1 Three-layer evidence methodology. Alloy determines structural possibility. Regtest measures concrete costs. The fee model projects costs into real-world fee environments to determine economic rationality. 85

A.1 Alloy model hierarchy. `btc_base` defines UTXOs, keys, and transactions. `vault_base` defines the abstract vault state machine. Five concrete models extend the base with covenant-specific transition guards. `threat_model` and `cross_covenant` are cross-cutting modules. 113

List of Tables

1.1 Five covenant vault designs compared in this work. "BIP status" refers to the proposal's standing in the Bitcoin Improvement Proposal process, not deployment on mainnet—none of the Bitcoin proposals are activated on mainnet as of April 2026. 6

2.1 Threat taxonomy. Each row defines an adversary profile used in the security analysis (Chapter 5). 15

2.2 The four Bitcoin covenant proposals, positioned on the seven axes of Section 2.10.1. Every attack-class susceptibility and immunity claim in Chapter 5 is a lookup against this table; numerical thresholds are measured in Chapter 4. 26

3.1 Adapter capability matrix. Experiments dispatch on these capabilities rather than adapter names. 30

3.2 Regtest validity. Structural properties transfer to mainnet; temporal and economic dynamics do not. 35

3.3 Experiment catalog. Each experiment is identified by a letter (A–P, no G) and classified by scope and concern. 36

3.4 Node variants required for each covenant. Switching nodes wipes regtest state; experiments are isolated per-covenant. 37

4.1 Lifecycle vsize measurements for the four Bitcoin covenant designs. Each value is regtest-measured and structurally deterministic (range = 0 across repeated measurements). Lifecycle total = tovault + trigger + withdraw (the happy-path cost). Simplicity (Elements) measurements appear in Appendix C. The *axis measured* column identifies the primary axis (Table 2.2) whose position determines the row's cost structure. 39

4.2 OP_VAULT vsize corrections. Prior estimates are from informal community discussion; measured values are from regtest lifecycle runs on the `opvault-demo` reference implementation. 40

4.3 Batched trigger vsize measurements. CCV and OP_VAULT are measured from actual multi-input transactions on regtest. CTV and CAT+CSFS cannot batch. 41

4.4 Revault amplification. Per-step withdrawal vsize is constant across steps because the script structure does not change with balance. CTV and CAT+CSFS cannot revault natively; their cost is the full unvault–withdraw cycle per withdrawal. 42

4.5 Fee sensitivity projections. Lifecycle cost = $\text{vsize}_{\text{total}} \times r$. Fee pinning cost is fee-invariant. Recovery griefing cost is the defender's per-round cost. Watchtower exhaustion shows the number of splits needed to exhaust a 0.5 BTC vault to dust. 44

5.1 Recovery model and griefing–safety tradeoff for each covenant design. 49

5.2 The four proposed Bitcoin Script extensions located in the (f, a, g, b) design space. The column "dist." gives Hamming distance to the dominant corner $D^\star = (f_{\neq\text{anchor}}, a_{\text{atomic}}, g_{\text{key}}, b_{\text{bound}})$. 54

5.3 Crossover fee rate r^* as a function of vault value V for CCV. Below r^*, per-UTXO recovery-cost scaling requires more than $B_{\max}$ blocks (infeasible). Above r^*, exhaustion completes within $B_{\max}$ blocks. 58

5.4 Measured batched-recovery constants on regtest. The fit $\text{vsize}(N) = \alpha + \beta \cdot N$ replaces the single-input $\text{vsize}_{\text{recover}}$ with an amortized per-UTXO cost $\alpha/N + \beta \to \beta$ for large N. 64

5.5 Recovery griefing cost comparison. The attacker invokes recovery; the defender re-deposits and re-triggers. $\gamma < 1$ favors the attacker. 67

5.6 Threat model matrix (TM1–TM11). Severity labels are bolded uniformly; every "N/A" cell carries a one-phrase reason. Severity scale: **N/A** = structurally impossible; **Low** = requires difficult preconditions; **Moderate** = feasible under specific conditions; **High** / **Severe** = significant risk under realistic assumptions; **Safe** = correct behaviour by design. 73

6.1 Alloy verification results. 34 of 35 safety checks pass; the one failure confirms cold key extraction in CAT+CSFS (TM11). 84

6.2 Three-layer evidence alignment for representative attacks. 86

7.1 Implementation choices in author-built vaults. These are design decisions, not BIP requirements; alternative designs would produce different measurements. 88

8.1 Comparison with closely related prior work. 99

List of Abbreviations

BIP	Bitcoin Improvement Proposal
CAT	OP_CAT (BIP-347), stack concatenation opcode
CCV	OP_CHECKCONTRACTVERIFY (BIP-443)
CLTV	OP_CHECKLOCKTIMEVERIFY, absolute timelock
CPFP	Child Pays For Parent, fee bumping mechanism
CSFS	OP_CHECKSIGFROMSTACK (BIP-348)
CSV	OP_CHECKSEQUENCEVERIFY, relative timelock
CTV	OP_CHECKTEMPLATEVERIFY (BIP-119)
DAG	Directed Acyclic Graph
HTLC	Hash Time-Locked Contract
P2TR	Pay-to-Taproot (Taproot output type)
P2WPKH	Pay-to-Witness-Public-Key-Hash
P2WSH	Pay-to-Witness-Script-Hash
RBF	Replace-By-Fee
TRUC	Topologically Restricted Until Confirmation (v3 transactions)
UTXO	Unspent Transaction Output
vB	Virtual bytes (segwit-discounted transaction size)
WU	Weight units (1 vB = 4 WU)

Chapter 1

Introduction

Bitcoin's custody model places the entire burden of key security on the asset holder. A compromised private key results in irreversible fund loss, with no recourse mechanism built into the protocol. Covenant-based vaults address this by interposing a time-delayed spending path between key compromise and fund movement: when the hot key initiates a withdrawal, a configurable delay window allows a watchtower to detect the unauthorized spend and trigger emergency recovery to a cold storage address [25, 43].

Four concrete covenant proposals have emerged as candidates for Bitcoin activation: `OP_CTV` [38], `OP_CCV` [19], `OP_VAULT` [31], and `CAT+CSFS` [39, 46]. Each enforces the vault lifecycle through a different mechanism. CTV pre-commits to the exact transaction structure at vault creation time. CCV validates contract state transitions dynamically at spend time. OP_VAULT provides dedicated vault opcodes with built-in recovery authorization. CAT+CSFS composes two general-purpose opcodes to achieve transaction introspection via dual signature verification. A fifth design, built with the Simplicity language on the Elements sidechain [32], demonstrates what covenant enforcement looks like with full transaction introspection and no soft-fork dependency.

Despite active community deliberation on which proposal to activate—including a 66-developer open letter requesting Bitcoin Core prioritize CTV+CSFS review [11], calls for comprehensive analysis before activation [35, 45], and the subsequent withdrawal of BIP-345 in favor of BIP-443 [30]—no published work compares these designs empirically. The BIP

specifications describe each opcode in isolation. Qualitative comparison matrices exist [14], but they contain no transaction sizes, no attack cost functions, and no fee-environment analysis. The only quantitative estimate in the literature is Harding's back-of-envelope calculation of watchtower splitting capacity at approximately 3,000 splits per block [18], based on assumed—not measured—transaction sizes.

This thesis fills that gap. We build all four Bitcoin vault implementations, instrument them through a uniform adapter interface, and run 15 experiments on regtest measuring transaction costs, security properties, and capability differences. We additionally build two original vault implementations—a CAT+CSFS vault using the explicit `OP_CHECKSIGFROMSTACK` (BIP-348) dual verification pattern, and a Simplicity vault on Elements using jet-based covenant enforcement—and develop Alloy bounded model checking specifications for all five designs.

1.1 Main Results

Two results constrain the design space of covenant vaults.

Griefing–safety impossibility. Recovery in a covenant vault is either permissionless (anyone can invoke it, as in CCV) or key-gated (requiring a specific key, as in OP_VAULT). Permissionless recovery maximizes fund safety under key loss—the owner never loses the ability to recover—but enables denial-of-service griefing by any third party who can broadcast a recovery transaction. Key-gated recovery eliminates griefing but introduces key-loss risk: if the recovery authorization key is lost, funds become permanently unrecoverable. We formalize this as Proposition 5.2 in Chapter 5: no covenant vault simultaneously achieves permissionless recovery and griefing resistance. This tradeoff is a design constraint, not an

implementation artifact.

Per-threat worst-case characterization. CTV vaults are vulnerable to descendant-chain fee pinning, where an attacker constructs 25 low-value transactions off the unvault output to block the watchtower's child-pays-for-parent recovery. CCV and OP_VAULT vaults are vulnerable to watchtower exhaustion, where an attacker repeatedly splits the vault into smaller UTXOs until individual recovery transactions become uneconomical. Rather than aggregating disparate attack outcomes (theft versus griefing) into a single "security ranking", we formalise a per-strategy defender-loss metric $\mathcal{L}$ and characterise, threat by threat, how $\mathcal{L}$ varies across designs with the fee rate r and the defender's batching strategy. The per-threat analysis surfaces a combinatorial impossibility—no design minimises $\mathcal{L}$ across all threats and fee rates—that is a direct corollary of the orderings, not a standalone claim about "safest" designs.

1.2 Contributions

This work makes seven contributions:

1. **First empirical comparison of the four Bitcoin Script extensions.** Regtest-measured transaction sizes for CTV, CCV, OP_VAULT, and CAT+CSFS under a uniform adapter interface, spanning 15 experiments and 11 threat models. We correct prior hand-estimates of OP_VAULT trigger transaction size from approximately 200 vB to 292 vB (a 46% error), revealing that OP_VAULT's lifecycle is 36% more expensive than CCV's.

2. **Formal adversary-advantage framework.** A defender-loss metric $\mathcal{L}(\mathrm{TM}, D, V, r, b)$

and adversary-advantage metric Adv (Definitions 5.3 and 5.4) that uniformly compare theft, griefing, and dust-lockup outcomes across threat models, fee rates, and defender batching strategies.

3. **Per-threat worst-case characterisation.** For each of TM1–TM11, a closed-form or bounded expression for $\mathcal{L}$ across designs as a function of fee rate, including the dust-threshold rate $r^\dagger$ for per-UTXO recovery-cost scaling (Section 5.2).

4. **Griefing–safety impossibility (Proposition 5.2).** No covenant vault achieves both permissionless recovery and griefing resistance.

5. **Structural design-space decomposition (Proposition 5.9).** A four-dimensional decomposition of covenant-vault design choices (f: fee model; a: amount flexibility; g: recovery gating; b: recovery binding). Each dimension has a deterministic feature-to-vulnerability implication (Lemmas 5.5–5.8). No proposed Bitcoin Script extension occupies the dominant corner $(f_{\neq\text{anchor}}, a_{\text{atomic}}, g_{\text{key}}, b_{\text{bound}})$; each falls short by at least one dimension with a corresponding vulnerability surface.

6. **Batched-defender measurements and ordering-flip finding.** On-chain measurements of BIP-345 and BIP-443 batched recovery yield $(\alpha, \beta) = (54, 68)$ vB for CCV and $(154, 92)$ vB for OP_VAULT. The batched dust-threshold rate $r^\dagger$ rises by 1.8× (CCV) and 2.7× (OP_VAULT), and the CCV/OP_VAULT ordering under per-UTXO recovery-cost scaling reverses (Section 5.4.1).

7. **Two original vault implementations and Alloy formal verification.** A CAT+CSFS vault using the explicit `OP_CHECKSIGFROMSTACK` (BIP-348) opcode for dual signature verification, distinct from prior OP_CAT-only vaults that achieve introspection via the Schnorr G-trick without CSFS [34, 37]. Additionally, a Simplicity vault using `jet::outputs_hash()` on Elements regtest via the Simplex SDK, treated

throughout as a cross-substrate reference point rather than a peer of the four Bitcoin proposals. Five Alloy models verify fund conservation, recovery liveness, and no-extraction properties.

1.3 Scope and Limitations

All experiments run on Bitcoin Core regtest (or Elements regtest for Simplicity). Transaction virtual size (vsize) is the primary metric; it is structurally deterministic and identical on regtest and mainnet. Fee amounts are computed as projections (fee = vsize $\times$ r, where r is an exogenous fee rate) rather than measured directly, because regtest has no fee market. Temporal dynamics of recovery races—where confirmation latency determines whether the watchtower or the attacker wins—are not captured, because regtest mines blocks on demand.

The four Bitcoin Script extension proposals are tested using their reference implementations: `simple-ctv-vault` [38], `pymatt` [19], `opvault-demo` [31], and our `cat-csfs-vault`. Where a reference implementation has a bug or missing feature, we distinguish this from a design-level property of the underlying BIP. Simplicity on Elements is included throughout as a cross-substrate reference point rather than a peer of the four Bitcoin proposals. Elements is a federated sidechain with functionary block production rather than proof-of-work, and the threat taxonomy does not transfer identically across that substrate boundary. Our formal results (Proposition 5.9, Proposition 5.2) therefore range over the four Bitcoin Script extensions only; Simplicity measurements are presented separately.

Protocol-layer constructs such as Ark and BitVM are excluded. Both operate above the opcode level—Ark uses an operator-mediated virtual UTXO model, BitVM uses off-chain fraud proofs—and require modeling interactive communication rounds that fall outside our single-transaction covenant framework.

Table 1.1: Five covenant vault designs compared in this work. "BIP status" refers to the proposal's standing in the Bitcoin Improvement Proposal process, not deployment on mainnet—none of the Bitcoin proposals are activated on mainnet as of April 2026.

Design	BIP	Enforcement mechanism	Chain	BIP status
CTV	119	Template hash commitment	Bitcoin	Proposed
CCV	443	Contract state verification	Bitcoin	Proposed
OP_VAULT	345	Dedicated vault opcodes	Bitcoin	Withdrawn
CAT+CSFS	347 + 348	Dual signature introspection	Bitcoin	Proposed
Simplicity	—	Jet-based introspection	Elements	Deployed

1.4 Thesis Organization

Chapter 2 surveys the four Bitcoin Script extension proposals and introduces Simplicity as an alternative script primitive on Elements, defining the vault lifecycle model and threat taxonomy used throughout. Chapter 3 describes the measurement framework, adapter architecture, and regtest validity analysis. Chapter 4 presents the empirical measurements: lifecycle costs, batching efficiency, revault amplification, and the corrections to prior estimates. Chapter 5 contains the core security analysis, including Propositions 5.2 and 5.9, parameterized attack cost functions, and the full threat model matrix. Chapter 6 describes the Alloy formal verification methodology and results. Chapter 7 discusses limitations, deployment guidance, and open problems. Chapter 8 positions this work relative to prior vault security research. Chapter 9 summarizes findings and future directions.

Chapter 2

Background

This chapter builds the conceptual foundation for covenant-based vault custody. Section 2.1 explains why Bitcoin Script cannot natively enforce spending conditions on future transactions. Section 2.2 introduces transaction introspection as the missing capability that covenants provide. Section 2.3 distinguishes generic from purpose-built covenant opcodes and discusses the recursive covenant question. Section 2.4 formalizes the vault lifecycle model and threat assumptions. Sections 2.5–2.8 describe the four Bitcoin Script extension proposals that are the primary subjects of this thesis. Section 2.9 introduces Simplicity as an *alternative script primitive* on the Elements federated sidechain, included as a cross-substrate reference point rather than a peer of the four Bitcoin proposals.

2.1 Bitcoin Script and Its Limitations

Bitcoin [26] transactions transfer value between *unspent transaction outputs* (UTXOs) [3, 13]. Each UTXO is encumbered by a *locking script* (scriptPubKey) that specifies the conditions under which the UTXO may be spent. The spending transaction provides a *witness* (scriptSig or Segwit witness [23]) that satisfies these conditions.

Bitcoin Script is deliberately limited. It is not Turing-complete: there are no loops, no persistent state between transactions, and a hard 10,000-byte script size limit. The language operates on a stack of byte strings and supports three categories of operations relevant to

custody:

- **Signature verification.** `OP_CHECKSIG` and its variants verify that a provided digital signature matches a public key embedded in the script. This is the basis of all standard Bitcoin spending: proving possession of the private key corresponding to the output's public key.
- **Hash preimage gates.** `OP_HASH160`, `OP_SHA256`, and related opcodes enable hash time-locked contracts (HTLCs), where spending requires revealing a preimage that hashes to a committed value. This is the foundation of the Lightning Network's payment channels.
- **Timelocks.** `OP_CHECKLOCKTIMEVERIFY` (CLTV) and `OP_CHECKSEQUENCEVERIFY` (CSV) [12, 17] enforce absolute and relative time constraints, respectively. A script can require that a UTXO remain unspent until a certain block height or for a certain number of blocks after its creation.

What Bitcoin Script *cannot* do is constrain the *structure* of the spending transaction itself. A standard `OP_CHECKSIG` script verifies that the spender possesses a private key, but it places no restriction on where the funds go, how many outputs the transaction creates, or what scripts encumber those outputs. Once the signature check passes, the spender has full discretion over the transaction's outputs.

This limitation has a direct consequence for custody: a compromised private key grants the attacker unconditional control over the funds. There is no mechanism within the script to impose a time delay, require a second authorization, or force funds to a predetermined recovery address. The custodian's only defense is to keep the key secure — there is no protocol-level fallback.

2.2 Transaction Introspection

Transaction introspection is the ability of a script to examine fields of the transaction that attempts to spend the UTXO it guards. Instead of merely verifying a signature or hash preimage, an introspective script can inspect the spending transaction's outputs, input count, sequence numbers, lock time, or other structural fields and accept or reject the spend based on whether those fields match expected values.

Introspection enables a qualitatively different class of spending conditions. Rather than asking "does the spender possess the right key?" the script asks "does the spending transaction have the right structure?" This distinction is the conceptual foundation of covenants.

Consider a concrete example. Suppose Alice deposits 1 BTC into a UTXO whose script enforces: "this UTXO may only be spent by a transaction that creates exactly two outputs—one sending 0.9 BTC to address X after a 144-block delay, and one sending 0.1 BTC to address Y immediately." No standard Bitcoin Script can express this condition, because the script has no access to the spending transaction's output fields. An introspective script can: it reads the outputs at spend time and verifies that they match the embedded constraints.

The degree of introspection varies across proposals:

- **Output-hash introspection.** The script computes a hash over all outputs of the spending transaction and compares it to an embedded commitment. CTV (BIP-119) and Simplicity's `jet::outputs_hash()` use this approach. The script sees only the aggregate hash, not individual output fields.
- **Per-output introspection.** The script examines individual outputs by index—verifying a specific output's scriptPubKey, amount, or both. CCV (BIP-443) operates at this granularity, checking that a particular output index matches an expected

Taproot structure.

- **Sighash preimage introspection.** The script reconstructs the transaction's signature hash from witness-provided fields and verifies it against both a stack-based signature check (`OP_CHECKSIGFROMSTACK`) and the real transaction sighash (`OP_CHECKSIG`). CAT+CSFS uses this technique to achieve introspection without a dedicated introspection opcode.

2.3 Covenants: Generic and Purpose-Built

A *covenant* is a spending condition that restricts not just who can spend a UTXO but how the resulting transaction is structured [25]. The term originates from property law, where deed covenants restrict how real estate may be used by future owners. In Bitcoin, a covenant restricts how future transactions may spend a UTXO, propagating constraints forward through the transaction graph.

2.3.1 Generic Covenants

A *generic covenant opcode* provides general-purpose introspection that can enforce arbitrary constraints on transaction structure. OP_CAT (BIP-347) is the minimal example: by enabling stack concatenation, it allows scripts to assemble and verify sighash preimages, achieving transaction introspection as a composition of primitives [34]. CTV (BIP-119) is generic in a different sense: it commits to a complete transaction template, constraining the entire output structure in one hash comparison.

Generic covenants are flexible—the same opcode can enforce vaults, payment channels, congestion control, and other applications—but they impose the full complexity of constraint

specification on the script author. There are no guardrails: a subtle error in the script (such as using an undefined mode value in CCV) can silently disable all covenant enforcement.

2.3.2 Purpose-Built Covenants

A *purpose-built covenant opcode* provides vault-specific semantics directly. OP_VAULT (BIP-345) is the primary example: the opcode pair `OP_VAULT` and `OP_VAULT_RECOVER` encodes the vault lifecycle (trigger withdrawal, authorize recovery, enforce timelocks) at the opcode level. Amount preservation, partial withdrawal, and recovery authorization are enforced at the opcode level rather than composed from primitives.

Purpose-built covenants reduce the script author's burden and eliminate classes of composition errors. The tradeoff is reduced generality: OP_VAULT enforces only the vault pattern, and extending it to other applications requires additional opcodes.

2.3.3 The Recursive Covenant Question

A covenant is *recursive* if it can constrain its outputs to contain the same covenant script, creating a chain of constrained transactions that extends indefinitely. Recursive covenants enable patterns such as:

- **Perpetual vaults.** A vault that can be partially withdrawn from indefinitely, with the remainder re-locked under the same rules.
- **State machines.** Multi-step protocols where each transaction advances the contract state and the next transaction is constrained by the current state.
- **Congestion control trees.** A single UTXO that fans out into a tree of pre-committed

transactions, allowing many recipients to claim funds at their own pace.

Whether a covenant opcode supports recursion depends on whether the script can reference its own scriptPubKey in the output constraints. CTV does not support direct recursion: the template hash commits to specific output scriptPubKeys, but the script cannot compute its own hash at spend time. CCV supports recursion via its *taptree preservation* mode, which constrains an output to contain the same Taproot script tree as the current input. OP_VAULT supports a limited form through automatic revault (excess value beyond the withdrawal amount is re-locked in a new vault UTXO). CAT+CSFS can achieve one level of self-reference by embedding the script's own `sha_single_output` as a constant, but cannot chain indefinitely: the output script's hash would need to include itself as a constant, creating a circular dependency that cannot be resolved at script construction time. Simplicity programs cannot reference their own commitment hash at runtime, placing them in the same category as CTV for recursion purposes.

The recursive covenant question has practical consequences for vault design: it determines whether partial withdrawals can be chained (relevant to the revault amplification and per-UTXO recovery-cost scaling experiments in Chapters 4 and 5).

2.4 Vault Lifecycle and Threat Model

The vault lifecycle follows the model formalized by Swambo et al. [43]. A vault is a UTXO whose spending is constrained to a time-delayed withdrawal process monitored by an automated watchtower. The lifecycle consists of four phases:

1. **Deposit.** The user sends funds to a vault address, creating a covenant- encumbered UTXO. The covenant enforces that this UTXO can only be spent via the trigger or

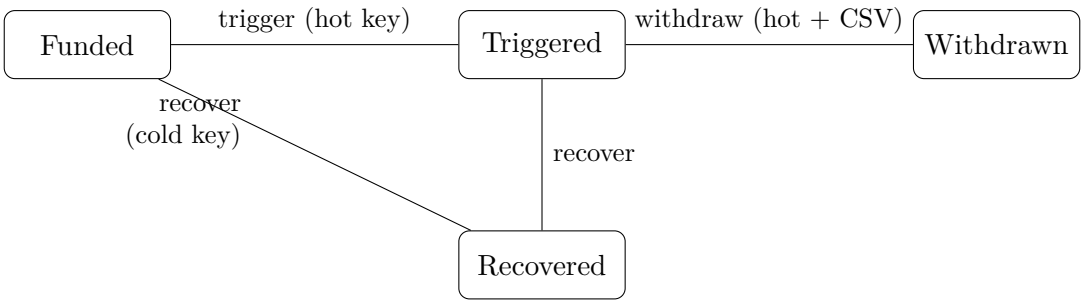

Figure 2.1: Vault lifecycle state machine. All covenant designs implement this four-state model. The trigger-to-withdrawal transition enforces a relative timelock (CSV) that creates the watchtower monitoring window. Recovery is available from both the Funded and Triggered states.

recovery paths.

2. **Trigger (unvault).** The hot key initiates a withdrawal by broadcasting a trigger transaction. This moves funds from the vault UTXO to an unvaulting UTXO that is subject to a relative timelock (`OP_CSV`).

3. **Withdrawal.** After the timelock expires (the "watchtower window"), the hot key completes the withdrawal to the destination address. If the trigger was unauthorized, the watchtower intervenes during the delay.

4. **Recovery.** At any point before withdrawal completes, the cold key (or, in CCV's case, anyone) can invoke emergency recovery, sweeping funds to a pre-committed recovery address. Recovery bypasses the timelock—it is always immediately available.

The watchtower is an automated process that monitors the blockchain for unauthorized trigger transactions. When the watchtower detects an unexpected unvault, it broadcasts a recovery transaction during the CSV delay window. The security of the vault depends on two assumptions: (1) the watchtower is online and connected to the network during the delay period, and (2) the recovery transaction can confirm before the attacker's withdrawal.

2.4.1 Threat Model Assumptions (Kerckhoffs)

We adopt the standard cryptographic threat model. The adversary is assumed to know the vault's design, the covenant type, the script structure, and all public parameters — including the recovery address where applicable. Security rests on keys and on the script's semantics, not on the secrecy of construction. Every attack-feasibility claim in this thesis is evaluated under this assumption. Concretely, we do not rely on taproot leaf hiding, address non-disclosure, or closed-source construction as security mechanisms.

This threat model choice is load-bearing for every attack-class derivation in Chapter 5. A reviewer may object to individual class-susceptibility claims with "but taproot hides that leaf," or "but the attacker would not know the cold address"; those objections target the on-chain information-reveal layer, where the thesis makes no cryptographic claim. The Kerckhoffs assumption moves the debate from "does taproot leak?" (which it does not, in the absence of legitimate recovery use) to "should a vault's griefing or theft immunity depend on the secrecy of its construction?" (which, by Kerckhoffs, it should not). A vault that is griefing-immune only because the attacker does not know its cold address is not cryptographically secure; it is obscured. The moment the recovery path is ever legitimately used on-chain, the cold address is revealed permanently. Security-by-obscurity is therefore not a principled defence, and the thesis assumes an attacker who has passed the obscurity threshold.

2.4.2 Threat Taxonomy

We extend the threat model of Swambo et al. [43] with attack classes specific to the four covenant designs. Table 2.1 summarizes the eleven threat models used throughout this thesis. Each threat model specifies the adversary's capabilities (which keys are compromised), the

Table 2.1: Threat taxonomy. Each row defines an adversary profile used in the security analysis (Chapter 5).

ID	Attack class	Adversary capability	Worst-case outcome
TM1	Fee pinning	Hot key + network access	Fund theft (CTV)
TM2	Recovery griefing	Varies by covenant	Denial of service
TM3	Trigger key theft	Hot/trigger key	Fund theft or DoS
TM4	Watchtower exhaustion	Hot key + capital	Fund loss (dust)
TM5	Trigger xpub theft	OP_VAULT trigger xpub	Amplified TM3
TM6	Dual-key compromise	Hot + cold/auth key	Fund theft or DoS
TM7	Address reuse	None (user error)	Stuck funds (CTV)
TM8	CCV mode bypass	Script knowledge	Total fund loss
TM9	Hot key redirection	Hot key (CAT+CSFS)	Griefing only
TM10	Witness tampering	Hot key (CAT+CSFS)	Impossible
TM11	Cold key theft	Cold key (CAT+CSFS)	Total fund theft

attack mechanism, and the worst-case outcome. The full threat model matrix with per-covenant severity ratings appears in Chapter 5.

2.5 OP_CHECKTEMPLATEVERIFY (CTV)

CTV [38] constrains the spending transaction to match a committed template hash. The hash covers: `nVersion`, `nLockTime`, `scriptSigs` hash, input count, `nSequence` hash, output count, outputs hash, and the current input's index. A script of the form `<hash> OP_CHECKTEMPLATEVERIFY` succeeds only if the spending transaction's template hash matches exactly.

CTV commits to the transaction's output structure but *not* to which UTXOs are consumed as inputs (input prevouts and amounts are excluded from the hash). This design choice means the same CTV script can be satisfied by different input UTXOs, enabling output-centric covenants. However, it also means a second deposit to the same CTV address creates a UTXO that cannot satisfy the template—the committed output amounts are derived from

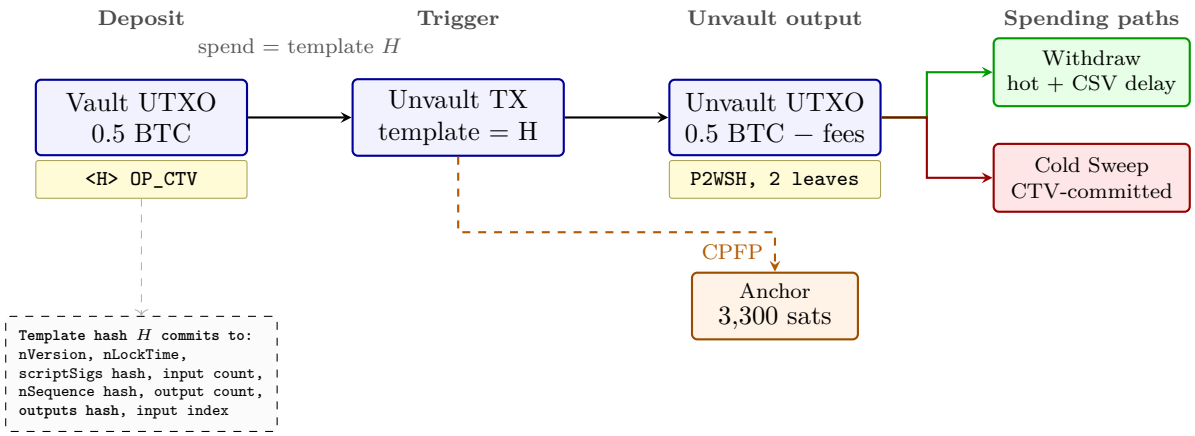

Figure 2.2: CTV vault mechanism. The vault UTXO is encumbered by a bare CTV hash H that commits to the unvault transaction's entire output structure; any spend whose template hash differs from H fails. The unvault UTXO is a P2WSH script with two Taproot leaves: a hot-key withdrawal gated by `OP_CSV`, and a CTV-committed cold sweep. A 3,300-sat anchor output enables CPFP fee bumping.

the original deposit, and a different deposit amount produces a permanently unspendable UTXO.

The CTV vault uses a two-step transaction tree. The vault UTXO is encumbered by a bare CTV script committing to the unvault transaction. The unvault transaction creates a P2WSH output with two spending paths: a hot-key withdrawal after a CSV delay (the normal path), and a CTV-committed cold sweep (the recovery path). Both paths include an anchor output for CPFP fee bumping. The entire tree is deterministic—it can be regenerated from key material and the original deposit amount without storing presigned transactions.

2.6 OP_CHECKCONTRACTVERIFY (CCV)

CCV [19] verifies relationships between inputs, outputs, and Taproot [50, 51] contract structure. The opcode takes five arguments from the stack: optional state data, an output or input index, an expected internal key, an expected taptree root, and a *mode* byte that con-

trols which checks are performed. Mode 0 verifies the output's scriptPubKey matches the expected Taproot structure. Mode 1 checks amounts. Mode 2 checks both. Mode -1 verifies the current input rather than an output.

CCV operates at the per-output granularity: each invocation constrains a single output (or input) by index. This is fundamentally different from CTV's all-outputs-at-once commitment. The per-output model enables two capabilities absent in CTV: partial withdrawal via `trigger_and_revault` (splitting a vault into a withdrawal portion and a re-funded vault in one transaction) and batched triggers (combining multiple vault UTXOs into a single trigger transaction).

The CCV vault combines CCV for state transitions with CTV for the final withdrawal template. The trigger step stores a CTV hash into the contract state. The withdraw path enforces a CSV timelock and executes `OP_CHECKTEMPLATEVERIFY` against the stored hash. Recovery is keyless—anyone can invoke the recover clause, which sends funds to the pre-committed recovery address without requiring a signature.

An important design property: undefined CCV mode values (any value outside $\{-1, 0, 1, 2\}$) trigger `OP_SUCCESS`, making the entire script succeed unconditionally. This is deliberate forward-compatibility behavior specified in BIP-443 to enable future soft-fork extensions, but it creates a developer footgun: a mode byte error silently disables all covenant enforcement.

2.7 OP_VAULT (BIP-345)

OP_VAULT [31] provides dedicated vault opcodes: `OP_VAULT` controls withdrawal initiation, and `OP_VAULT_RECOVER` enables authorized recovery. Unlike CCV's generic contract verification, these opcodes encode vault-specific semantics directly. Amount preservation

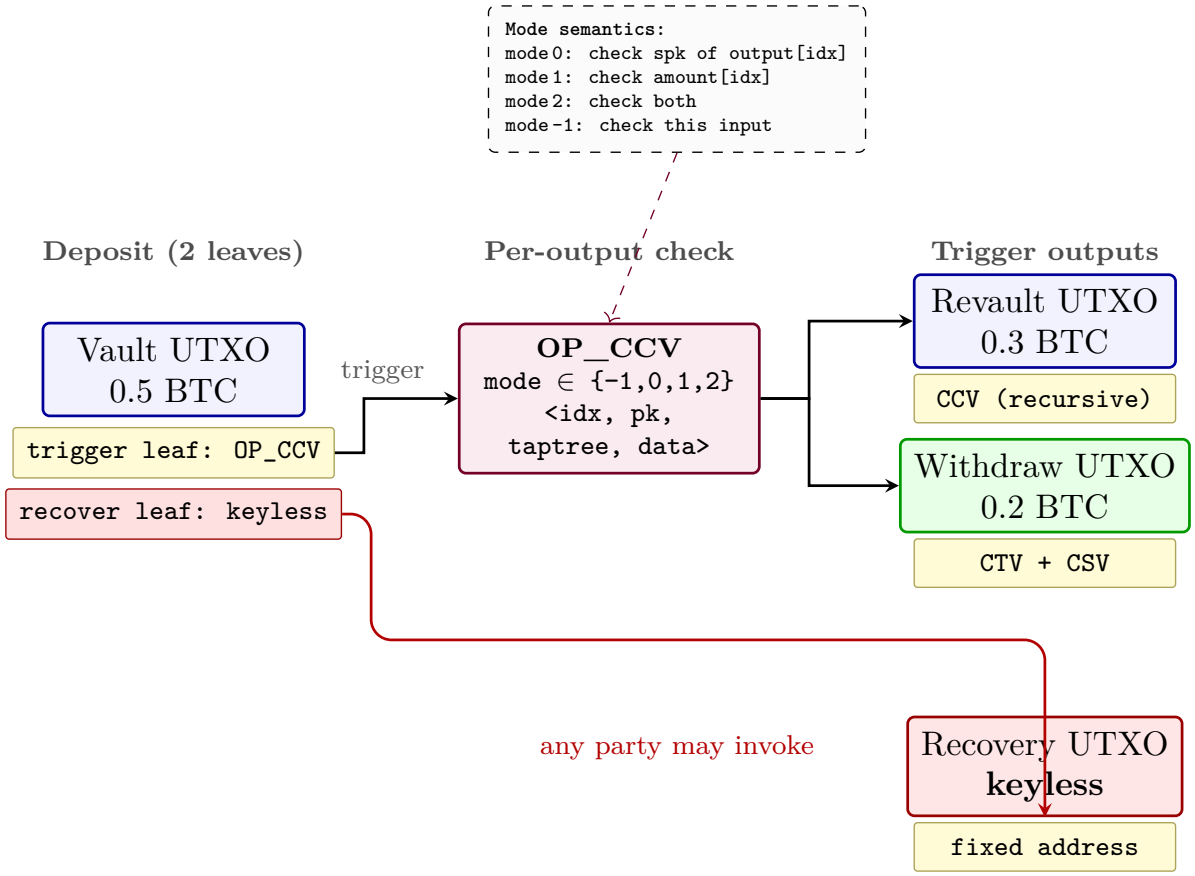

Figure 2.3: CCV vault mechanism. The vault UTXO is a Taproot with two leaves: a *trigger* leaf containing `OP_CCV`, and a keyless *recover* leaf. Spending the trigger leaf invokes the per-output `OP_CHECKCONTRACTVERIFY` check, whose mode byte selects whether to verify the scriptPubKey, the amount, both, or the current input. Taptree preservation enables recursion, supporting partial withdrawal via `trigger_and_revault`. Spending the recover leaf is permissionless and sends funds to the pre-committed address without a signature.

(ensuring outputs account for all input value), automatic revault (excess value beyond the withdrawal amount is re-locked), and recovery authorization (requiring a `recoveryauth` key signature) are enforced at the opcode level.

The OP_VAULT design uses a three-key model: a *trigger key* (derived from an xpub) initiates withdrawals, a *recoveryauth key* authorizes emergency recovery, and a *recovery address* (embedded in the vault's Taproot internal key tweak) is the pre-committed destination for recovered funds. This three-key separation is unique among the designs studied: CTV and

CAT+CSFS use two keys (hot + cold), and CCV uses one (trigger key; recovery is keyless).

A structural consequence of OP_VAULT's design is the *fee-input requirement*. The opcode's value preservation rule requires that all input value be accounted for in outputs. Since transaction fees are paid from input value, the vault UTXO alone cannot cover fees without violating value preservation. The reference implementation solves this by adding a separate fee-wallet UTXO as a second input to every trigger and recovery transaction, adding 80–90 vB of overhead per transaction.

BIP-345 was withdrawn in early 2025 in favor of BIP-443 [30]. We include it in this comparison because (a) it represents a distinct design philosophy (purpose-built vs. generic covenants), (b) the economic comparison with CCV quantifies the cost of that design choice (36% lifecycle overhead), and (c) it serves as a historical reference for the OP_VAULT-to-CCV transition.

2.8 CAT+CSFS (BIP-347 + BIP-348)

CAT+CSFS achieves transaction introspection by composing two general-purpose opcodes [34, 39, 46]. `OP_CAT` (BIP-347) concatenates the top two stack elements. `OP_CHECKSIGFROMSTACK` (BIP-348) verifies a Schnorr signature against an arbitrary stack-provided message rather than the transaction's sighash. The introspection pattern works as follows:

1. The spender provides the sighash preimage fields (transaction version, hash prevouts, hash amounts, hash script pubkeys, hash sequences, hash outputs, spend type, and other BIP-342 fields) as individual witness elements.

2. The script uses `OP_CAT` to assemble these fields into the complete sighash preimage on

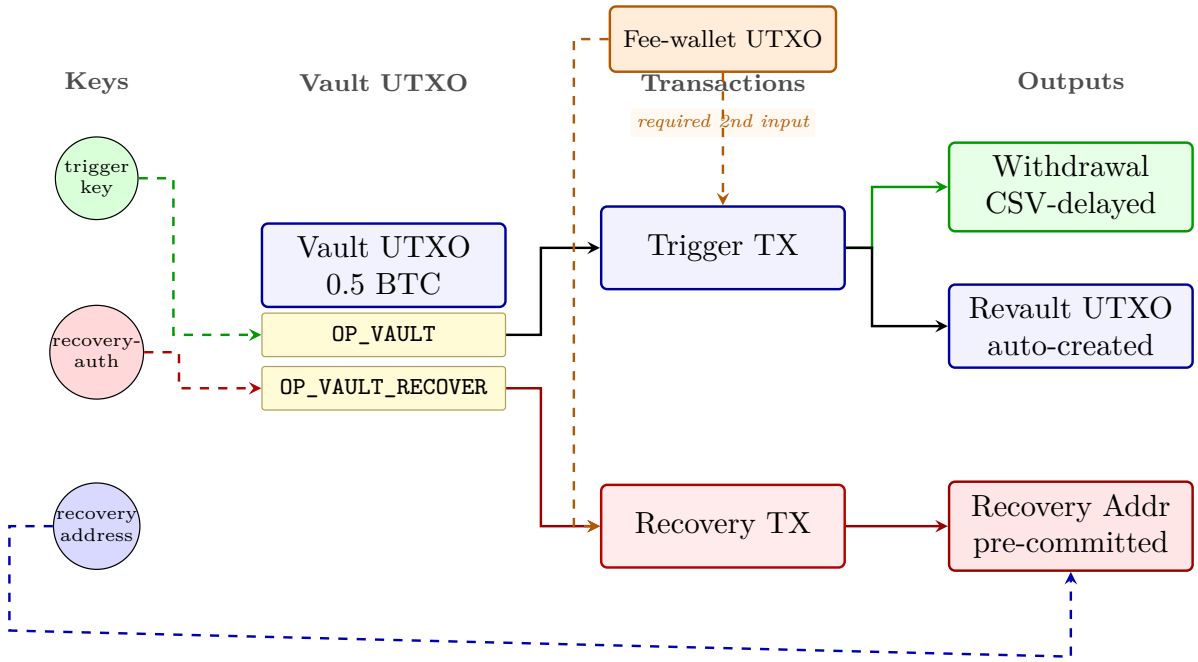

Figure 2.4: OP_VAULT mechanism. The opcode pair `OP_VAULT` and `OP_VAULT_RECOVER` encodes vault-specific semantics (amount preservation, auto-revault, authorized recovery) directly. Three keys are separated: the *trigger key* initiates withdrawals, the *recoveryauth key* authorizes recovery, and the *recovery address* (embedded in the Taproot internal-key tweak) is the pre-committed destination. Value preservation requires a fee-wallet input on every trigger and recovery transaction (∼80–90 vB overhead).

the stack.

3. `OP_CHECKSIGFROMSTACK` verifies a Schnorr signature against this assembled preimage using the hot key.

4. `OP_CHECKSIG` verifies the *same signature* against the real transaction sighash computed by the Bitcoin consensus layer.

If both checks pass with the same key and signature, the witness-provided preimage must be the actual transaction's preimage (otherwise the signature could not satisfy both). The script can embed constants (such as `sha_single_output`, the SHA-256 hash of the expected output) that the spender cannot alter, effectively constraining the transaction's outputs

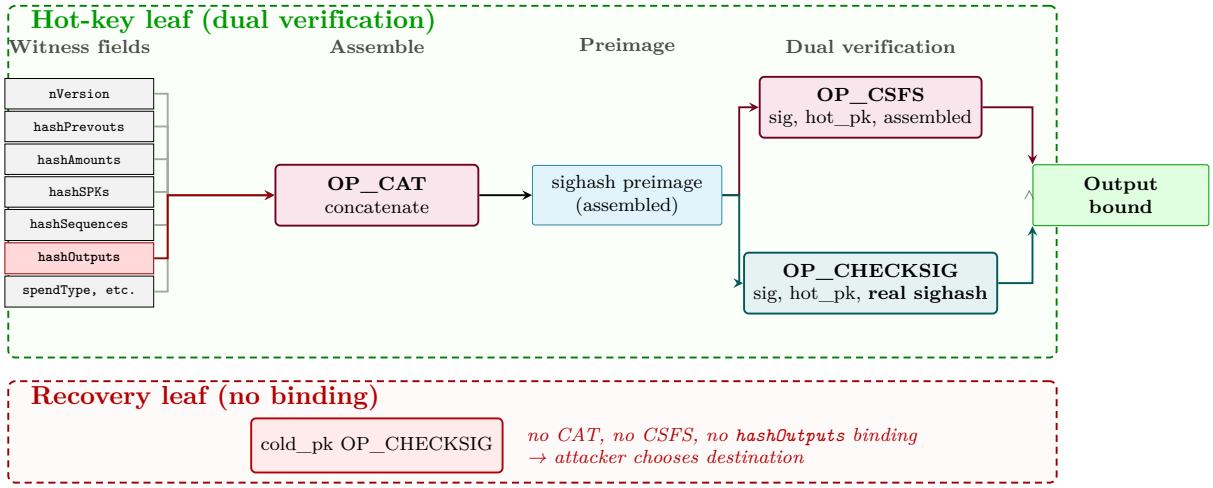

Figure 2.5: CAT+CSFS vault mechanism. The Taproot script has two leaves. **Hot-key leaf (top):** the spender provides sighash preimage fields; `OP_CAT` concatenates them; `OP_CSFS` verifies the Schnorr signature against this stack-assembled preimage; `OP_CHECKSIG` verifies the *same* signature against the real transaction sighash. Both succeed only if the two preimages agree, binding outputs through `hashOutputs`. **Recovery leaf (bottom):** bare `cold_pk OP_CHECKSIG`—no CAT, no CSFS, no output binding.

through the signature binding.

The CAT+CSFS vault uses `SIGHASH_SINGLE|ANYONECANPAY` (0x83), a sighash mode defined in BIP-342 [51] that commits to one input and one output. This allows external fee-paying inputs to be attached without breaking the covenant—a more flexible fee model than CTV's anchor-based CPFP.

Two properties of this design have direct security implications. First, the withdrawal destination is fixed at vault creation time (the `sha_single_output` constant is embedded in the script), making it the most rigid destination constraint of any design studied. Second, the recovery leaf uses bare `cold_pk OP_CHECKSIG` with no introspection, meaning cold key compromise grants the attacker unrestricted control over recovery—the weakest recovery path among the four Bitcoin designs.

2.9 Simplicity on Elements: An Alternative Script Primitive

The four proposals above extend Bitcoin Script by introducing new opcodes or composing existing ones under soft-fork consensus change. Simplicity takes a different path: it is an *alternative script primitive*, a typed combinator calculus with denotational semantics verified in the Coq proof assistant, designed by O'Connor [32]. Programs are directed acyclic graphs (DAGs) of combinators. The language runs on Elements/Liquid [16], a federated Bitcoin sidechain. Both Bitcoin Script and Simplicity are scripting languages; they differ in paradigm (stack-based vs. typed functional), formal foundation (informal specification vs. mechanised semantics), and operating substrate (Bitcoin L1 proof-of-work vs. Elements federated consensus).

Execution uses *jets*: when the validator encounters a known combinator subtree (identified by its canonical Merkle root), it executes a native C implementation instead of interpreting the combinators. Jets for transaction introspection (`jet::outputs_hash()`, `jet::sig_all_hash()`), timelocks (`jet::check_lock_distance()`), and cryptographic primitives (`jet::bip_0340_verify()` [49]) provide the building blocks for covenant enforcement.

The Simplicity vault enforces covenants via `jet::outputs_hash()`, which computes a Merkle hash over all outputs of the spending transaction. The expected hash is embedded as a compile-time parameter in the Simplicity program. At spend time, the jet computes the hash from the actual transaction outputs; if the values differ, the program fails at the consensus level. This is structurally analogous to CTV's template hash but operates over Elements' richer output model (explicit asset IDs, confidential values, range proofs).

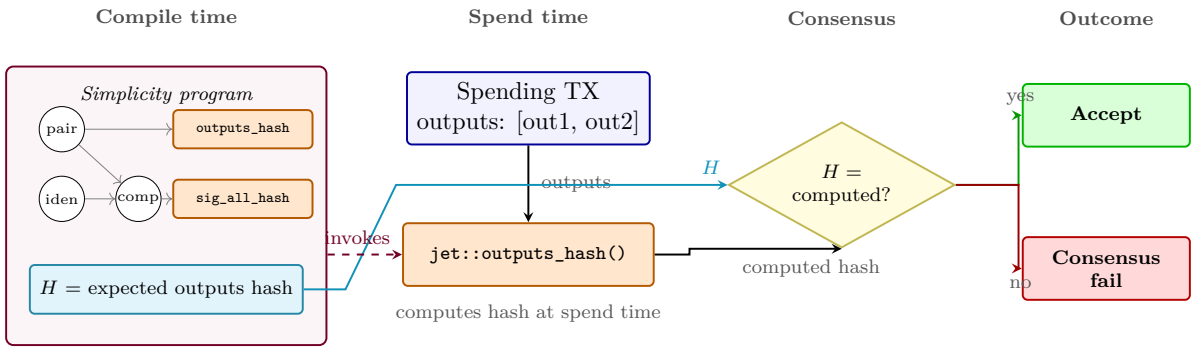

Figure 2.6: Simplicity vault mechanism on Elements. At compile time, the program (a DAG of combinators) embeds an expected outputs-hash H as a parameter. At spend time, `jet::outputs_hash()` computes the Merkle hash of the spending transaction's outputs. If the computed hash matches H, the program accepts; otherwise the program fails at the consensus level. Both trigger and recovery leaves enforce this jet—unlike CAT+CSFS, where recovery is unconstrained.

The Simplicity vault runs on Elements, a federated sidechain where block production is controlled by known functionaries rather than proof-of-work miners. This fundamentally changes the trust model: functionaries could censor attacker transactions or collude with an attacker, making the threat models (TM1–TM11) applicable only under the assumption of honest federation behavior. Measurements from Elements are presented separately from the four Bitcoin covenant measurements and are not directly comparable due to differences in transaction serialization (explicit fee outputs, confidential transaction fields add approximately 30–40 bytes per output).

2.10 The Design-Space Axes

The four Bitcoin covenant proposals differ along seven axes. Three carry over from the formal decomposition ([18, 43] and this work) and appear with Greek-letter symbols (f, a, g, b) in proofs (Chapter 5); the remaining four are introduced in this thesis and are used as descriptive labels. Every attack class in Chapter 5 derives from a specific axis-value; every design's

vulnerability profile is a lookup against Table 2.2. We define the axes here, in Background, so that Chapter 5's analysis reads as mechanical derivation rather than assertion.

2.10.1 Axis Definitions

$A1$ (**g**) — **Recovery authorisation.** Who may initiate the recovery path? Values: `keyless`, `recoveryauth-key`, `hot-key`, `cold-key`. Controls class Z2 (permissionless griefing).

$A2$ (**a**) — **Revault support.** Does the covenant allow partial withdrawal — splitting one vault UTXO into a withdrawn output and a remaining vault UTXO in one transaction? Values: `yes`, `no`. Controls class Z3 (per-UTXO recovery-cost scaling).

$A3$ (**f**) — **Fee-inclusion model.** Where does the spending transaction's fee come from? Values: `dynamic` (fee in the transaction), `static-precommit` (fee baked at creation), `out-of-band-anchor` (CPFP via a separate anchor key). Controls class Z1 (fee-channel pinning).

A4 — **Withdrawal-destination binding.** How is the withdrawal destination constrained? Values: `template-hash-digest`, `per-output-amount+script`, `dedicated-opcode`, `signature-dual-bind`. Controls class Z4 (hot-key theft via destination substitution).

A5 — **Recovery-destination binding.** Same question for the recovery path. Values: same as A4, plus `plain-signature` (no script-level constraint). Controls class Z5 (cold-key theft via unconstrained recovery). Kept separate from A4 because `CAT+CSFS` occupies different positions on the two sub-axes — strongest on A4, weakest on A5. A collapsed binding axis b is used in proofs that do not depend on this asymmetry.

A6 — Opcode surface. Is the covenant expressed by a single semantic opcode or a multi-mode polymorphic one with `OP_SUCCESSx` fallthrough on undefined modes? Values: `single-mode`, `multi-mode-with-OP_SUCCESS`. Controls class Z6 (mode/parameter bypass).

A7 — Introspection primitive. How does the script read transaction data? Values correspond to the four primitives enumerated in Section 2.2: `digest-equality` (CTV), `merkle-membership-and-amount` (CCV), `dedicated-opcode` (OP_VAULT), and `sighash-preimage-reconstruction` (CAT+CSFS). Descriptive; governs which of the other axes are simultaneously realisable.

For the formal decomposition in Chapter 5 we retain (f, a, g, b) as the axes of the design-space lattice used in Propositions 1 and 2: $f = A3$, $a = A2$, $g = A1$, and b is the joint $(A4, A5)$ sublattice. Axes A6 and A7 are additional dimensions orthogonal to the proofs; they are where the developer-footgun (class Z6) and design-implementability questions live.

2.10.2 Four-Covenant Position Table

Table 2.2 locates each of the four Bitcoin proposals on every axis. Every attack-class susceptibility claim in Chapter 5 is a mechanical lookup against this table; every immunity claim is a mechanical reading of the opposite axis-value. Simplicity on Elements is treated separately in Section 2.9 and (at greater length) in Appendix C; it occupies the strict-best position on A1, A2, A4, A5, and A7 simultaneously, and no Bitcoin proposal does.

Table 2.2: The four Bitcoin covenant proposals, positioned on the seven axes of Section 2.10.1. Every attack-class susceptibility and immunity claim in Chapter 5 is a lookup against this table; numerical thresholds are measured in Chapter 4.

Axis	`OP_CTV`	`OP_CCV`	`OP_VAULT`	`CAT+CSFS`
A1 (recovery auth)	hot-key	keyless	recoveryauth-key	cold-key
A2 (revault)	no	yes	yes	no
A3 (fee model)	out-of-band-anchor	dynamic	dynamic	dynamic
A4 (withdrawal binding)	template-hash-digest	per-output-amount+script	dedicated-opcode	signature-dual-bind
A5 (recovery binding)	template-hash-digest	per-output-amount+script	dedicated-opcode	plain-signature
A6 (opcode surface)	single-mode	multi-mode-with-OP_SUCCESS	single-mode	single-mode
A7 (introspection primitive)	digest-equality	merkle-membership-and-amount	dedicated-opcode	sighash-preimage-rec

2.10.3 Structural Dependencies Between Axes

Two spec-level dependencies between axes constrain the realisable combinations and are used throughout Chapter 5:

1. A3 = `out-of-band-anchor` implies A2 = `no`. A CPFP-anchor fee model relies on a template hash that commits to the whole output set; splitting the vault into [withdrawn] + [remaining] would require the template to anticipate the split amount, which the user cannot know at vault creation. CTV's position on these two axes is not coincidental; it is forced.

2. A7 = `sighash-preimage-reconstruction` is the only known introspection primitive that admits A4 = `signature-dual-bind`. The binding reduces to Schnorr message-equality under a single key, which requires reassembling the sighash preimage from stack contents and verifying two signatures under the same key — a construction that the whole-template-digest, per-output, and dedicated-opcode primitives do not support.

These dependencies reduce the effective design space from the naive $4 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 2 \cdot 4 = 1920$

cells to a much smaller set of realisable positions. Every Bitcoin proposal occupies a realisable position; the thesis contribution is to characterise every realisable position's attack surface.

Chapter 3

Measurement Methodology

This chapter describes how we measure and compare covenant vault designs. Section 3.1 presents the adapter-based framework architecture. Section 3.2 defines the primary metric (virtual size) and the fee projection model. Section 3.4 analyzes what regtest measurements do and do not tell us about mainnet behavior. Section 3.5 catalogs the 15 experiments and their classification. Section 3.6 documents the node variants required for each covenant.

3.1 Framework Architecture

The comparison framework uses an *adapter pattern* to decouple experiment logic from covenant-specific implementations. Each vault design is wrapped in an adapter class that implements a common interface (Figure 3.1). Experiments invoke adapter methods—`create_vault`, `trigger_unvault`, `complete_withdrawal`, `recover`—without knowledge of the underlying covenant mechanism. This design serves two purposes: it ensures that measurements across covenants use identical experiment logic, and it allows adding new covenant designs without modifying existing experiments.

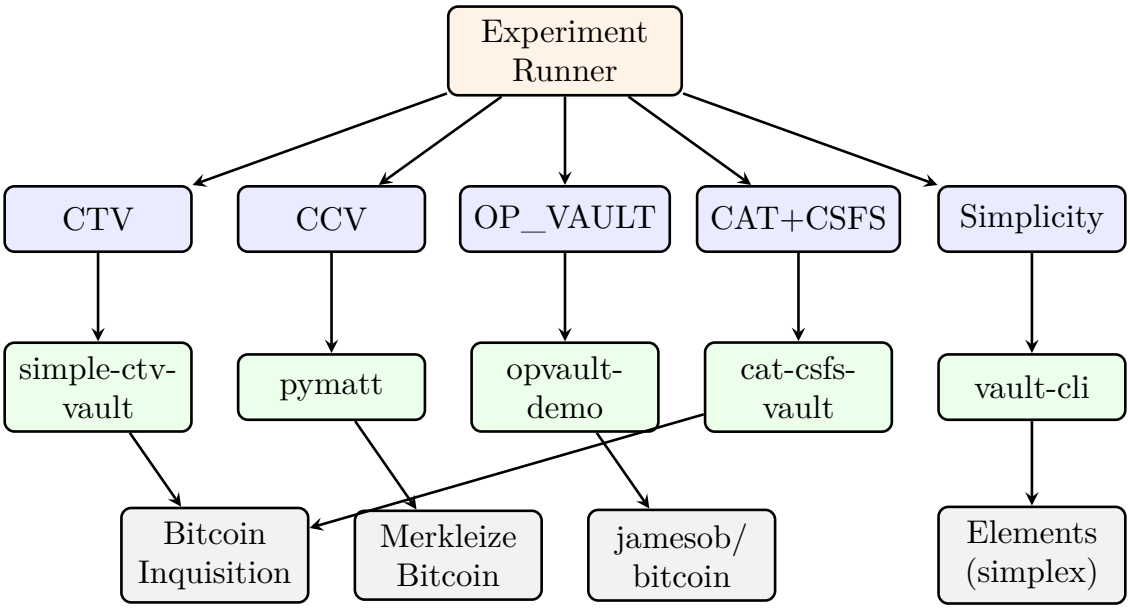

Figure 3.1: Framework architecture. The experiment runner invokes vault operations through the `VaultAdapter` interface. Each adapter wraps a covenant-specific implementation and communicates with its required Bitcoin or Elements node variant. CTV and CAT+CSFS share the Bitcoin Inquisition node.

3.1.1 Adapter Interface

The abstract `VaultAdapter` class defines the contract that every adapter must implement. The core lifecycle methods mirror the four-phase vault model from Section 2.4:

- `create_vault(amount_sats)` → `VaultState`: Fund a new vault with the specified amount.
- `trigger_unvault(vault)` → `UnvaultState`: Initiate a withdrawal using the hot key.
- `complete_withdrawal(unvault)` → `TxRecord`: Finalize the withdrawal after the CSV delay.
- `recover(state)` → `TxRecord`: Execute emergency recovery from either the Funded or Triggered state.

Table 3.1: Adapter capability matrix. Experiments dispatch on these capabilities rather than adapter names.

Capability	CTV	CCV	OPV	CAT	SIM
`create_vault`	✓	✓	✓	✓	✓
`trigger_unvault`	✓	✓	✓	✓	✓
`complete_withdrawal`	✓	✓	✓	✓	✓
`recover`	✓	✓	✓	✓	✓
`supports_revault`	—	✓	✓	—	—
`supports_batched_trigger`	—	✓	✓	—	—
`supports_keyless_recovery`	—	✓	—	—	—
Address reuse safe	No	Yes	Yes	Yes	Yes

Optional methods expose capabilities that not all covenants support: `trigger_revault` for partial withdrawals (CCV, OP_VAULT), `trigger_batched` for combining multiple vault UTXOs in one transaction (CCV, OP_VAULT), and `supports_keyless_recovery` for CCV's signature-free recovery path.

3.1.2 Capability-Based Dispatch

Experiments query adapter capabilities at runtime rather than checking adapter names. For example, the multi-input batching experiment calls `adapter.supports_batched_trigger()` to determine whether to execute the batched or linear code path. This design principle—dispatch on capabilities, not identities—means a new adapter automatically participates in all experiments whose required capabilities it declares. Table 3.1 shows the capability matrix for the five adapters.

3.1.3 Implementation Provenance

Three of the five vault implementations are third-party upstream repositories used as-is: `simple-ctv-vault` (jamesob), `pymatt` (Merkleize/Ingala), and `opvault-demo` (jamesob).

The remaining two—`cat-csfs-vault` and `simple-simplicity-vault`—are original implementations built for this research. This distinction matters for interpreting results: implementation bugs in upstream repositories are not attributable to the BIP design, while bugs in our own implementations are indistinguishable from design choices unless explicitly documented. Section 7.1.2 in Chapter 7 lists the specific design choices we made in the two original implementations.

3.2 Primary Metric: Virtual Size

The primary metric throughout this work is *virtual size* (vsize), measured in virtual bytes (vB). Virtual size is the segwit-discounted transaction size used by Bitcoin Core for relay policy, fee estimation, and block weight accounting:

$$\text{vsize} = \left\lceil \frac{\text{weight}}{4} \right\rceil \quad \text{where} \quad \text{weight} = 3 \times \text{base_size} + \text{total_size} \tag{3.1}$$

The base size excludes witness data; the total size includes it. The factor-of-4 discount on witness data reflects the segwit design decision to incentivize witness-heavy transaction formats (Taproot, multisig) without increasing the block size limit for non-witness data.

Virtual size is the correct metric for cross-covenant comparison because it is *structurally deterministic*: for a given script and witness structure, vsize is identical regardless of chain state, balance, block height, or execution environment. The per-UTXO recovery-cost scaling experiment (Chapter 4) confirmed this empirically: across 50 trigger-and-revault splits with vault balances ranging from 50 million to fewer than 1,000 satoshis, the measured vsize range was zero for both CCV and OP_VAULT transactions.

This determinism has a methodological consequence: single-run measurements are exact

values, not estimates. There is no variance to analyze and no confidence interval to compute. Repeated measurements would produce identical results because the measurement is a function of the script structure, not a stochastic process.

3.2.1 Fee Projection Model

Since regtest has no fee market, we project transaction costs into real-world fee environments using a linear model:

$$\text{fee}(r) = \text{vsize} \times r \tag{3.2}$$

where r is the fee rate in sat/vB, treated as an exogenous parameter. We evaluate all security experiments at six historically observed fee rates: 1 sat/vB (2021 low), 10 sat/vB (2022 average), 50 sat/vB (2023 moderate), 100 sat/vB (2024 inscriptions era), 300 sat/vB (2023 BRC-20 spike), and 500 sat/vB (stress scenario). This produces a cost sensitivity table for each attack, showing how the rationality condition shifts across fee environments.

The rationality condition for an attack is:

$$\text{attack_cost}(r) < \text{vault_value} \tag{3.3}$$

An attack is economically rational when its on-chain cost (transaction fees plus locked capital) is less than the value at risk in the vault. This condition is necessary but not sufficient: the attacker must also expect the attack to succeed, which depends on whether the defender's watchtower can respond within the CSV window.

3.3 Rationality and Scope Conventions

Every attack class derived in Chapter 5 is accompanied by a standardised *Rationality and Scope* block with seven fields. The fields are defined here, once, so that Chapter 5's derivations do not re-introduce the schema. Appendix D carries the full free-text form for every class; Chapter 5 references these blocks inline.

(1) Attacker capability. Keys held, capital on hand, mempool access, compute. Cites the Kerckhoffs assumption of Section 2.4.1 by reference; does not re-state it.

(2) Defender model. Watchtower authority and policy (proactive vs reactive, batched vs per-UTXO), user availability, detection-to-action latency. Where the attack depends on a specific defender choice, the block names it and explains why a rational defender might make that choice.

(3) Economic preconditions. Fee regime, vault-value range, and rationality threshold under which the attack yields positive adversary advantage.

(4) Protocol / network assumptions. Mempool policy parameters (descendant limits, dust thresholds, RBF rules), relay floor, TRUC/BIP-431 modifiers, cluster-mempool scope-limits.

(5) Scope of validity. Axis-values under which the class applies; equally important, the axis-values under which it does *not* apply.

(6) Rational-attacker check. Is execution economically rational? Payoff vs cost vs residual (theft, liveness denial, both, or neither). If the attack is only rational under external incentive (extortion, competition, state actor), the block says so explicitly.

(7) Counter-factual mitigations. Which single falsified assumption kills the attack. This field doubles as a seed for deployment recommendations in Chapter 7.

The template applies to attack classes (Z1–Z6) in Chapter 5 and, with "Attack" replaced by "Measurement" in field (1), to empirical-cost measurements in Chapter 4. Every field is populated, or explicitly marked "N/A — [reason]"; partial blocks are treated as an audit failure.

3.4 Regtest Validity

All experiments run on Bitcoin Core regtest (or Elements regtest for Simplicity). This is a deliberate methodological choice: regtest provides deterministic, reproducible execution without external dependencies. However, it limits external validity in four specific ways.

No mempool competition. Regtest mines transactions immediately on demand. Attacks that depend on mempool dynamics—fee pinning races, front-running via mempool observation, recovery races against timelocks—lose their temporal dimension. On mainnet, a recovery transaction competes against the attacker's withdrawal for confirmation within the CSV window.

No relay policy pressure. Bitcoin Core's relay policy (descendant limits at 25 transactions / 101 kvB, RBF rules, minimum relay fee) is enforced in regtest's mempool but never stressed by competing traffic. The fee pinning experiment constructs a 25-descendant chain that reaches the limit, but no concurrent transactions contest for mempool space.

Table 3.2: Regtest validity. Structural properties transfer to mainnet; temporal and economic dynamics do not.

Property	Valid on regtest?	Primary limitation
Transaction vsize and weight	Yes (exact)	—
Script execution semantics	Yes (identical)	—
Descendant chain limits	Yes (enforced)	No competing traffic
Contract state transitions	Yes (identical)	—
Cost asymmetries (trigger vs. recovery)	Yes (structural)	—
Absolute fee amounts	No (artifacts)	No fee market
Recovery race timing	No	Instant mining
Mempool front-running	No	No competing traffic
Fee market confirmation pressure	No	No congestion
CSV manipulation via mining power	No	No adversarial miners

No fee market. Fee amounts on regtest are artifacts of coin selection, not rational economic behavior. This is why we treat vsize as the primary metric and project fees via Equation 3.2 rather than measuring them directly.

Mining is instant. The CSV delay (e.g., 10 blocks $\approx$ 100 minutes on mainnet) resolves in milliseconds on regtest. Time-critical races between the attacker's withdrawal and the defender's recovery are instantaneous rather than uncertain.

Table 3.2 summarizes the validity status of each measurement category on regtest.

3.5 Experiment Catalog

The framework includes 15 experiments organized by scope and concern. Table 3.3 lists all experiments with their scope category, the covenants they test, and the threat model they address.

Experiments fall into five scope categories:

Table 3.3: Experiment catalog. Each experiment is identified by a letter (A–P, no G) and classified by scope and concern.

ID	Name	Category	Covenants	TM
A	`lifecycle_costs`	Comparative	All 5	—
B	`address_reuse`	Comparative	All 5	TM7
C	`fee_pinning`	Comparative	All 5	TM1
D	`revault_amplification`	Capability gap	CCV, OPV	—
E	`multi_input`	Capability gap	All 5	—
F	`recovery_griefing`	Comparative	All 5	TM2
H	`watchtower_exhaustion`	Comparative	CCV, OPV	TM4
I	`ccv_mode_bypass`	Verification	CCV	TM8
J	`fee_sensitivity`	Analytical	All	All
K	`opvault_recovery_auth`	Covenant-specific	OPV	TM2
L	`opvault_trigger_key_theft`	Covenant-specific	OPV	TM3,5
M	`cat_csfs_hot_key_theft`	Verification	CAT	TM9
N	`cat_csfs_witness_manip.`	Verification	CAT	TM10
O	`cat_csfs_dest_lock`	Covenant-specific	CAT	—
P	`cat_csfs_cold_key_recovery`	Covenant-specific	CAT	TM11

- **Comparative.** The same test runs on all covenants; findings emerge from the differences in outcomes (A, B, C, F, H).
- **Capability gap.** Tests a feature that some covenants support and others cannot (D, E).
- **Covenant-specific.** Measures economic costs of design tradeoffs unique to one covenant (K, L, O, P).
- **Verification.** Empirically confirms that specified opcode or cryptographic behavior holds in the implementation (I, M, N). These are implementation correctness checks, not security findings.
- **Analytical.** Fee projections and sensitivity analysis using structural vsize as input; no on-chain transactions produced (J).

Table 3.4: Node variants required for each covenant. Switching nodes wipes regtest state; experiments are isolated per-covenant.

Covenant	Node variant	Branch	Build
CTV	Bitcoin Inquisition	`v29.2-inq` release	Pre-built
CCV	Merkleize Bitcoin	`inq-ccv`	CMake
OP_VAULT	jamesob/bitcoin	`2023-02-opvault-inq`	Autotools
CAT+CSFS	Bitcoin Inquisition	`v29.2-inq` release	Pre-built
Simplicity	Elements (Simplex SDK)	`v0.0.2`	Pre-built

3.6 Node Infrastructure

Each covenant requires a specific Bitcoin (or Elements) node variant, because the opcodes are not yet activated on Bitcoin mainnet. CTV and CAT+CSFS share Bitcoin Inquisition, a soft-fork testing fork maintained by the Bitcoin Inquisition project. CCV requires a separate fork maintained by Merkleize (Ingala). OP_VAULT requires jamesob's fork with the `2023-02-opvault-inq` branch, which uses the older autotools build system. Simplicity runs on Elements via the Simplex toolchain, which bundles `elementsd` and an Esplora block indexer.

The framework's `switch-node.sh` script manages node lifecycle: starting the correct node variant, wiping regtest state between runs, and discovering ports for Elements (which assigns random RPC and Esplora ports at startup). A Docker image packages all five node variants with pre-built binaries for reproducible execution without local compilation.

Chapter 4

Empirical Results

This chapter presents the measured transaction sizes and cost data that form the empirical basis for the security analysis in Chapter 5. Section 4.1 reports the core lifecycle vsize measurements. Section 4.2 documents the correction of prior OP_VAULT estimates. Section 4.3 presents batching efficiency data. Section 4.4 measures revault amplification costs. Section 4.5 reports address reuse behavior. Section 4.8 presents the Simplicity vault measurements separately.

4.1 Lifecycle Cost Measurements

Table 4.1 shows the measured vsize for each transaction in the deposit–trigger–withdraw lifecycle across all five covenant designs. These measurements are the foundation for all subsequent cost analysis: every attack cost function, fee sensitivity projection, and cross-covenant comparison in this thesis derives from these structural vsize values.

CTV has the cheapest lifecycle at 368 vB, owing to its minimal script structure: the trigger (unvault) transaction is a bare CTV hash check with no signature, producing the smallest trigger of any design at 94 vB. CCV and OP_VAULT have nearly identical lifecycle totals (565 and 567 vB), but with opposite cost distributions: CCV's trigger is cheap (154 vB) and its deposit is expensive (300 vB due to the two-output contract instance structure), while OP_VAULT's trigger is expensive (292 vB due to the fee-wallet input) and its deposit is

Table 4.1: Lifecycle vsize measurements for the four Bitcoin covenant designs. Each value is regtest-measured and structurally deterministic (range = 0 across repeated measurements). Lifecycle total = tovault + trigger + withdraw (the happy-path cost). Simplicity (Elements) measurements appear in Appendix C. The *axis measured* column identifies the primary axis (Table 2.2) whose position determines the row's cost structure.

Covenant	tovault (vB)	trigger (vB)	withdraw (vB)	recover (vB)	**total (vB)**	axis measured
CTV	122	94	152	133	**368**	A3=anchor, A4=digest
CCV	300	154	111	122	**565**	A2=yes, A3=dynamic
OP_VAULT	154	292	121	246	**567**	A2=yes, A3=dynamic (wallet)
CAT+CSFS	122	221	210	125	**553**	A4=dual-bind, A3=ACP

moderate (154 vB). CAT+CSFS falls between CTV and CCV/OP_VAULT at 553 vB, with the dual CSFS+CHECKSIG introspection pattern adding witness overhead to both trigger (221 vB) and withdrawal (210 vB).

The lifecycle cost at a given fee rate r (in sat/vB) is:

$$C_{\text{lifecycle}}(r) = (\text{vsize}_{\text{tovault}} + \text{vsize}_{\text{trigger}} + \text{vsize}_{\text{withdraw}}) \times r \tag{4.1}$$

At the median 2024 fee rate of 50 sat/vB, a single vault lifecycle costs 18,400 sats (CTV), 28,250 sats (CCV), 28,350 sats (OP_VAULT), or 27,650 sats (CAT+CSFS).

4.2 OP_VAULT Vsize Correction

Prior to this work, the only published transaction size estimates for OP_VAULT were informal hand-calculations. Table 4.2 compares these estimates against our regtest measurements.

The discrepancy originates from the fee-input requirement described in Section 2.7. OP_VAULT's value preservation rule prohibits spending the vault UTXO alone (the fee

Table 4.2: OP_VAULT vsize corrections. Prior estimates are from informal community discussion; measured values are from regtest lifecycle runs on the `opvault-demo` reference implementation.

Transaction	Prior estimate	Measured	Delta
trigger (`start_withdrawal`)	∼200 vB	292 vB	+46%
recovery (`OP_VAULT_RECOVER`)	∼170 vB	246 vB	+45%

would violate input–output balance), so every trigger and recovery transaction includes a second input from a fee wallet. This adds a P2TR input witness (approximately 80–90 vB) that prior estimates omitted. The consequence is a 36% lifecycle cost premium over CCV: OP_VAULT's lifecycle (567 vB) versus CCV's (565 vB) appears similar in total, but OP_VAULT pays 80–90 vB more for every non-deposit transaction. Over time, the accumulated overhead is significant for vault operators who trigger frequently.

This measurement contributed to the economic case for withdrawing BIP-345 in favor of BIP-443 [30]: the purpose-built opcodes' fee-input overhead costs more than the generic CCV approach achieves with the same vault semantics.

4.3 Batching Efficiency

CCV and OP_VAULT can combine multiple vault UTXOs into a single trigger transaction. CTV and CAT+CSFS cannot batch because each vault's template hash (CTV) or embedded `sha_single_output` (CAT+CSFS) commits to a specific output structure that changes with each additional input.

Table 4.3 shows the measured trigger vsize at increasing batch sizes. Both CCV and OP_VAULT exhibit linear scaling with a fixed base cost and a constant marginal cost per additional vault.

Table 4.3: Batched trigger vsize measurements. CCV and OP_VAULT are measured from actual multi-input transactions on regtest. CTV and CAT+CSFS cannot batch.

	Batch size (N vaults)					
Covenant	N=1	N=2	N=3	N=5	N=10	N=20
CCV	154	254	355	555	1,056	2,059
OP_VAULT	292	385	479	667	1,135	2,073
CTV, CAT+CSFS: $N\times$ single trigger (154 vB, 221 vB respectively)						

The batching efficiency follows a linear model:

$$\text{vsize}_{\text{batch}}(N) = \text{base} + \text{marginal} \times N \tag{4.2}$$

For CCV, the marginal cost is ∼100 vB per additional vault (base ∼54 vB). For OP_VAULT, the marginal cost is ∼94 vB per vault (base ∼198 vB). The marginal cost per vault in a batch is substantially lower than the cost of individual triggers (154 vB for CCV, 292 vB for OP_VAULT), demonstrating the efficiency gain from amortizing the transaction overhead across multiple inputs.

The standardness ceiling—the maximum batch size that fits within Bitcoin Core's 400,000 WU transaction weight limit—can be projected from these measurements:

$$N_{\text{max}} = \left\lfloor \frac{400{,}000 - \text{base_weight}}{\text{marginal_weight}} \right\rfloor \tag{4.3}$$

For CCV, $N_{\text{max}} \approx 1{,}000$ vaults per transaction. For OP_VAULT, $N_{\text{max}} \approx 1{,}064$ vaults. In practice, batch sizes above 50–100 are rare in custodial operations, so the standardness limit does not constrain realistic deployments.

Table 4.4: Revault amplification. Per-step withdrawal vsize is constant across steps because the script structure does not change with balance. CTV and CAT+CSFS cannot revault natively; their cost is the full unvault–withdraw cycle per withdrawal.

Covenant	Per-step vsize	Cumulative (5 steps)	Cumulative (9 steps)
CCV	111 vB	555 vB	999 vB
OP_VAULT	121 vB	605 vB	1,089 vB
CTV (extrapolated)	246 vB	1,230 vB	2,214 vB
CAT+CSFS (extrapolated)	431 vB	2,155 vB	3,879 vB

4.4 Revault Amplification

CCV and OP_VAULT support partial withdrawals: the vault owner withdraws a portion of the vault balance and re-locks the remainder in a new vault UTXO. This process can be repeated, creating a chain of partial withdrawals from a single initial deposit.

The cost of each withdrawal step is the sum of the trigger (or trigger-and-revault) and the withdrawal completion. Table 4.4 shows the per-step and cumulative costs measured across 9 successive partial withdrawals of 5,000,000 sats each from an initial 49,999,900-sat vault.

The per-step vsize is constant across all 9 steps for both CCV and OP_VAULT, confirming the structural determinism of the measurement. CCV's smaller per-step cost (111 vB vs. 121 vB) reflects its lighter withdrawal transaction (no fee-input overhead).

For CTV and CAT+CSFS, which lack native revault, each partial withdrawal requires a full cycle: unvault the entire balance, withdraw the desired portion, and re-deposit the remainder in a new vault. The extrapolated per-step cost is the sum of the trigger and withdrawal vsizes for a single cycle. This is 2.2× (CTV) to 3.9× (CAT+CSFS) more expensive than CCV's native revault per step.

The cumulative cost has implications for the per-UTXO recovery-cost scaling attack analyzed in Chapter 5: an attacker chaining many small partial withdrawals forces the watchtower to

pay recovery fees proportional to the number of splits.

4.5 Address Reuse

CTV is the only design where address reuse causes permanent fund loss. A second deposit to the same CTV address creates a UTXO whose amount does not match the committed template hash. The template commits to output amounts derived from the *original* deposit, so the second UTXO is either permanently unspendable (if the deposit is smaller) or overpays miners by the difference (if larger).

CCV, OP_VAULT, CAT+CSFS, and Simplicity all handle multiple deposits to the same address safely. CCV and OP_VAULT use per-instance contract checking—each UTXO is independently validated against the covenant rules regardless of how many UTXOs share the same address. CAT+CSFS generates a unique vault plan per source coin, producing distinct P2TR addresses. Simplicity creates fresh plans with new addresses for each vault instance.

This is a design-level property, not an implementation artifact: CTV's template hash commits to input count and output amounts, making it structurally incompatible with variable deposit amounts. The other designs either validate per-input (CCV, OP_VAULT) or per-instance (CAT+CSFS, Simplicity).

4.6 Fee Sensitivity Projections

Experiment J projects the measured vsize data from the preceding sections into real-world fee environments using the linear fee model (Equation 3.2). Table 4.5 shows the lifecycle

Table 4.5: Fee sensitivity projections. Lifecycle cost = $\text{vsize}_{\text{total}} \times r$. Fee pinning cost is fee-invariant. Recovery griefing cost is the defender's per-round cost. Watchtower exhaustion shows the number of splits needed to exhaust a 0.5 BTC vault to dust.

	Lifecycle cost (sats)				Fee pin	Grief	WT exhaust
r (sat/vB)	CTV	CCV	OPV	CAT	(CTV)	(CCV)	splits
1	368	565	567	553	14,300	454	409,836
10	3,680	5,650	5,670	5,530	14,300	4,540	40,984
50	18,400	28,250	28,350	27,650	14,300	22,700	8,197
100	36,800	56,500	56,700	55,300	14,300	45,400	4,098
300	110,400	169,500	170,100	165,900	14,300	136,200	1,366
500	184,000	282,500	283,500	276,500	14,300	227,000	820

cost and key attack costs across six historically observed fee rates.

Three observations emerge from the projections:

- **Fee pinning is fee-invariant.** The 14,300-sat pinning cost (Section 5.3) is constant across all fee environments because the attacker pays dust-rate fees regardless of prevailing rates. At a 0.5 BTC vault value, pinning costs 0.03% of the vault—well below any rationality threshold at any historical fee rate.

- **Lifecycle cost rankings are stable.** CTV is the cheapest at every fee rate. CCV and OP_VAULT are nearly identical (within 0.4%). CAT+CSFS falls between CTV and CCV/OP_VAULT. The ranking does not change with fees because the vsize ratios are constant.

- **Watchtower exhaustion feasibility is fee-dependent.** At 1 sat/vB, exhausting a 0.5 BTC vault requires ∼410,000 splits (infeasible—66 blocks of maximum-rate splitting). At 300 sat/vB, it requires only 1,366 splits (feasible in a single block). This fee-dependent transition is the empirical basis for Proposition 5.9 in Chapter 5.

4.7 Synthesis: What the Measurements Imply

Three patterns in the data foreshadow the security analysis in Chapter 5:

1. **CTV is the cheapest lifecycle but the least flexible.** At 368 vB, CTV's lifecycle cost is 33–35% lower than CCV or OP_VAULT. But CTV cannot batch, cannot partially withdraw, and loses funds on address reuse. The cost advantage comes from CTV's minimal script structure—the same rigidity that creates its security vulnerabilities (fee pinning via anchors, stuck funds on reuse).

2. **OP_VAULT's fee-input overhead is structural, not incidental.** The 80–90 vB per non-deposit transaction reflects BIP-345's value preservation rule, not an implementation choice. This overhead makes OP_VAULT 36% more expensive than CCV per lifecycle—the economic basis for BIP-345's withdrawal in favor of BIP-443.

3. **Batching and revault create the per-UTXO recovery-cost scaling surface.** CCV and OP_VAULT's capability advantages (partial withdrawal, batched triggers) are also the attack surface for per-UTXO recovery-cost scaling. The per-step revault cost (111–121 vB) is cheap enough to chain thousands of splits, creating the fee-dependent vulnerability analyzed in Section 5.4. CTV and CAT+CSFS are immune precisely because they lack these capabilities.

4.8 Case Study: Simplicity on Elements

The preceding sections present the four-way Bitcoin covenant comparison—the core contribution of this thesis. This section reports measurements from a fifth vault built with the Simplicity language on Elements regtest. Because Elements uses different transaction

serialization (explicit fee outputs, confidential transaction fields, multi-asset model) and a federated consensus model, these measurements are **not directly comparable** with the Bitcoin results. We present them as a case study in what covenant enforcement looks like with full transaction introspection and no soft-fork dependency.

Lifecycle costs. The Simplicity vault lifecycle totals 865 vB (tovault 269, trigger 298, withdraw 298, recover 286). This is 49–135% larger than the Bitcoin designs, reflecting two factors: Elements serialization overhead (approximately 30–40 bytes per output for confidential transaction fields even in non-confidential mode) and Simplicity's bit-oriented DAG witness format (structurally different from Bitcoin Script's stack-based witnesses). A hypothetical Bitcoin-native Simplicity implementation would produce smaller transactions; the 865 vB figure includes both the language overhead and the chain overhead.

Output-constrained recovery. Unlike CAT+CSFS, the Simplicity vault enforces `jet::outputs_hash()` on *both* trigger and recovery paths. Cold key compromise can only recover funds to the pre-committed recovery address—matching CCV and OP_VAULT's output-constrained model and providing stronger recovery safety than CAT+CSFS's unconstrained `OP_CHECKSIG` (TM11, Section 5.7).

Fee pinning immunity. Elements uses explicit fee outputs (empty scriptPubKey) rather than Bitcoin's implicit fees. There are no anchor outputs to pin with descendant chains, eliminating the TM1 attack vector entirely. This immunity is a property of the Elements transaction format, not of Simplicity's covenant mechanism.

Chapter 5

Security Analysis

This chapter presents the core analytical contribution. We prove two results that constrain the design space of covenant vaults (Sections 5.1 and 5.2), then analyze each threat model using the measured vsize data from Chapter 4. Section 5.8 synthesizes the per-threat findings into a two-dimensional security tradeoff characterization. Section 5.9 presents the complete threat model matrix. Section 5.10 derives fee-conditional deployment guidance.

5.1 Griefing–Safety Impossibility

The covenant literature has long discussed the tension between keyless and key-gated recovery as a pragmatic tradeoff without formalising the underlying dichotomy. We make that dichotomy explicit: when recovery authorisation is expressed as a function of cryptographic key possession, any design must choose one side, and the choice carries inescapable consequences for both griefing surface and key-loss risk. The proposition below captures this as the structural root of the (g, b) projection (Lemma 5.11).

Definition 5.1 (Recovery Authorization)**.** A vault's recovery mechanism is characterized by an authorization function $\mathcal{A} : 2^{\mathcal{K}} \to \{0, 1\}$, where $\mathcal{K}$ is the set of all cryptographic keys and $\mathcal{A}(S) = 1$ if and only if the key set S is sufficient to invoke recovery. Recovery is *permissionless* if $\mathcal{A}(\emptyset) = 1$ (no keys required) and *key-gated* if $\mathcal{A}(\emptyset) = 0$ (at least one key is required).

Proposition 5.2 (Griefing–Safety Incompatibility)**.** *Under the assumption that recovery authorization is a function of key possession (Definition 5.1), no covenant vault simultaneously achieves:*

1. ***Permissionless recovery:*** $\mathcal{A}(\emptyset) = 1$ *(any party can recover without key material), and*
2. ***Griefing resistance:*** *no unauthorized party can invoke recovery to deny the owner's withdrawal.*

Proof. By exhaustive case analysis over the two possible values of $\mathcal{A}(\emptyset)$.

Case 1: $\mathcal{A}(\emptyset) = 1$ *(permissionless).* Any party can broadcast a recovery transaction without possessing key material. This maximizes fund safety under key loss—the owner can always recover, even if all hot keys are compromised—but any third party who can construct the recovery transaction can force-recover the vault, denying the owner access to the normal withdrawal path. Adding output constraints (recovery only to a pre-committed address) prevents fund redirection but does not prevent the denial of service: the attacker can still interrupt the owner's withdrawal by forcing recovery to the legitimate address. CCV implements this model.

Case 2: $\mathcal{A}(\emptyset) = 0$ *(key-gated).* Recovery requires a signature from a designated key (the `recoveryauth` key in OP_VAULT, or the cold key in CTV, CAT+CSFS, and Simplicity). This eliminates griefing by unauthorized parties—only the key holder can invoke recovery—but introduces key-loss risk: if the required key is lost or destroyed, $\mathcal{A}(S) = 0$ for all available key sets S, and recovery becomes permanently unavailable.

Since $\mathcal{A}(\emptyset) \in \{0, 1\}$, these two cases are exhaustive. In Case 1, griefing resistance fails. In Case 2, permissionless recovery fails. No vault achieves both. □

Table 5.1: Recovery model and griefing–safety tradeoff for each covenant design.

Covenant	Recovery model	Key required	Griefing	Key-loss risk
CCV	Keyless	None	Anyone can grief	None
OP_VAULT	Key-gated (auth)	recoveryauth	Auth key holder	Auth key loss
CTV	Key-gated (cold)	Cold key	Cold key holder	Cold key loss
CAT+CSFS	Key-gated (cold)	Cold key	Cold key holder	Cold key loss
Simplicity	Key-gated + constrained	Cold key	Cold key holder	Cold key loss

Scope of the assumption. The proposition assumes recovery authorization depends solely on cryptographic key possession. It does not model non-cryptographic gating mechanisms such as time delays on recovery, reputation systems, or out-of-band identity verification. All five covenant designs studied in this thesis fall within this assumption: recovery is gated by `OP_CHECKSIG` (key-gated) or ungated (permissionless).

The practical consequence is that vault designers must choose a point on the griefing–safety spectrum. Table 5.1 maps each covenant to its position.

5.2 Adversary Advantage and Structural Analysis

Proposition 5.2 establishes a structural tradeoff without reference to fee dynamics. To reason about how vault security varies across designs, threat models, and fee environments uniformly, we introduce a per-strategy loss framework. We then decompose the covenant-vault design space into four nearly-orthogonal dimensions, derive feature-by-feature implications for defender loss, and obtain as a structural consequence that no proposed Bitcoin Script extension occupies the dominant corner of the design space.

5.2.1 Framework

We parameterise each vault scenario by a threat model $\mathrm{TM} \in \mathcal{T}$, a design $D \in \mathcal{D}$, a vault value $V > 0$, a prevailing fee rate $r \geq 0$, and a boolean defender strategy $b \in \{0, 1\}$: $b = 1$ indicates the defender batches recovery transactions across available UTXOs; $b = 0$ indicates single-input recovery. An adversary strategy $\mathcal{A}$ consists of a (possibly unbounded) sequence of on-chain transactions broadcast in pursuit of TM.

Definition 5.3 (Per-strategy cost and outcome)**.** For an adversary strategy $\mathcal{A}$ at fee rate r we write: $g(\mathcal{A}) \in [0, V]$ for the funds the adversary extracts to an attacker-controlled output; $c_A(\mathcal{A}, r)$ for the on-chain fees paid by the adversary; $c_D(\mathcal{A}, r, b)$ for the on-chain fees paid by the defender responding optimally to $\mathcal{A}$ under batching strategy b; and $u(\mathcal{A}) \in [0, V]$ for funds rendered unrecoverable (dust-locked) by $\mathcal{A}$.

Definition 5.4 (Defender loss and adversary advantage)**.** Fix $(\mathrm{TM}, D, V, r, b)$ and an attacker budget B_A. The *worst-case defender loss* is

$$\mathcal{L}(\mathrm{TM}, D, V, r, b) \;=\; \sup_{\mathcal{A}:\, c_A(\mathcal{A}, r) \leq B_A} \big[\, g(\mathcal{A}) + u(\mathcal{A}) + c_D(\mathcal{A}, r, b) \,\big],$$

taken over strategies realising TM against D with bounded attacker fee expenditure. The budget constraint formalises the rational-adversary assumption; without it, unbounded griefing rounds make the supremum diverge. In the per-threat analyses of Sections 5.3–5.7 we take B_A to be the vault value V (an attacker will not pay more in fees than the funds at stake); the measured $\mathcal{L}$ values are insensitive to larger choices of B_A. The *worst-case adversary advantage* is $\mathrm{Adv}(\mathrm{TM}, D, V, r) = \sup_{\mathcal{A}}[\, g(\mathcal{A}) - c_A(\mathcal{A}, r) \,]$, which is invariant in b. Design D_1 is *safer than* D_2, written $D_1 \prec_{\mathrm{TM},V,r,b} D_2$, iff $\mathcal{L}(\mathrm{TM}, D_1, V, r, b) < \mathcal{L}(\mathrm{TM}, D_2, V, r, b)$.

$\mathcal{L}$ captures theft ($g > 0$), griefing ($c_D > 0$), and dust-lockup ($u > 0$) in a single quantity comparable across threat classes; Adv is positive when the attack is profit-bearing and

negative when the attacker pays for a denial-of-service outcome. Per-threat expressions for $\mathcal{L}$ appear in Sections 5.3–5.7.

5.2.2 Design-Space Decomposition

Every covenant vault design D makes a choice in four nearly-orthogonal dimensions that together determine its on-chain attack surface.

Fee model (f) How covenant transactions pay miner fees. Observed values across the four proposed BIPs:

- f_{anchor}: CPFP-bumpable anchor output (CTV);
- f_{wallet}: separate fee-wallet input (OP_VAULT);
- f_{inval}: fees deducted from input value via introspection (CCV);
- f_{acp}: `SIGHASH_SINGLE|ANYONECANPAY` permits external fee inputs (CAT+CSFS).

Amount flexibility (a) Whether a single trigger can spend less than the vault's full value. a_{atomic}: the entire vault must unvault in one trigger. a_{partial}: trigger-and-revault splits input value across two outputs.

Recovery gating (g) Who may invoke the recovery path. g_{keyless}: any party (CCV). g_{key}: an authorised key is required (cold key in CTV, CAT+CSFS; recoveryauth key in OP_VAULT).

Recovery binding (b) Whether the recovery output is script-constrained. b_{bound}: destination fixed at vault creation (CTV template hash; OP_VAULT Taproot-tweak; Simplicity `jet::outputs_hash()`). b_{unbound}: recovery leaf is bare `OP_CHECKSIG` and admits any destination (CAT+CSFS cold leaf).

We write $D = (f, a, g, b)$. Two structural dependencies hold among the dimensions: $f = f_{\text{acp}}$ forces $a = a_{\text{atomic}}$ because `SIGHASH_SINGLE` commits to exactly one output, which forbids the two-output `trigger_and_revault` pattern; $a = a_{\text{partial}}$ requires $f \in \{f_{\text{wallet}}, f_{\text{inval}}\}$ because per-output introspection is needed to deduct the withdrawal amount while preserving the remainder.

5.2.3 Feature–Vulnerability Implications

Each feature choice deterministically opens or closes a vulnerability surface. The implications below are consequences of the feature's script semantics, not empirical observations about specific BIPs.

Lemma 5.5. *Under Bitcoin Core 28.x default relay policy (25-descendant mempool cluster limit, 3 sat/vB dust relay rate, 3,300-sat anchor output size in the reference CTV implementation), $f = f_{anchor}$ implies the existence of a TM1 fee-pinning attack strategy with $\mathcal{L}(\text{TM1}) = V$. Under TRUC/v3 relay [10] or a future cluster-mempool policy with single-descendant opt-in, the surface closes, and the lemma's premise no longer entails the vulnerability. The constants are relay-policy parameters of the Bitcoin Core reference software, not constants in BIP-119 itself.*

Lemma 5.6. *$a = a_{partial}$ implies the existence of a class-Z3 per-UTXO recovery-cost scaling strategy with $\mathcal{L}(\text{TM4}) > 0$ for all $r > r^{\dagger}_{\text{TM4}}(V, D, b)$. Conversely, $a = a_{atomic}$ implies $\mathcal{L}(\text{TM4}) = 0$ for every r.*

Protocol level vs. user parameters. The *existence* of a finite threshold $r^{\dagger}_{\text{TM4}}(V, D, b)$ is a protocol-level consequence of Lemma 5.6: the dust-threshold attack is definable whenever $a = a_{\text{partial}}$ holds, independent of implementation. The numeric *value* of $r^{\dagger}$ depends on three

parameters, none of them implementation artifacts: V is the vault value (chosen by the user at creation), N_{budget} is the defender's block budget (chosen by the watchtower operator), and $\text{vsize}_{\text{rec}}$ is determined by the BIP specification (script semantics fix the minimum witness program size; signature scheme fixes witness overhead; transaction format fixes serialisation overhead). Spec-minimal implementations produce the same $\text{vsize}_{\text{rec}}$ up to small encoding choices (Taproot control-block variants, nonce formats); our measurements on the upstream reference implementations differ by less than 5 vB from spec-minimal estimates. Readers reproducing the numeric $r^\dagger$ for different vault values or defender budgets may scale our measured values linearly without re-running the regtest harness.

Lemma 5.7. *$g = g_{keyless}$ implies a third party can force $\mathcal{L}(\text{TM2}) > 0$ with attacker cost $\gamma \cdot c_D$ for griefing ratio $\gamma < 1$ (Proposition 5.2). $g = g_{key}$ restricts the attacker to the designated key holder.*

Lemma 5.8. *$b = b_{unbound}$ implies $\mathcal{L}(\text{TM11}) = V$. $b = b_{bound}$ implies $\mathcal{L}(\text{TM11}) = 0$.*

Proofs. Each claim is direct from the feature's semantics. Lemma 5.5: descendant-chain pinning requires an externally-spendable output under the same ancestor-descendant mempool cluster as the unvault transaction; f_{anchor} is the only choice producing such an output. Lemma 5.6: `trigger_and_revault` is the mechanism by which the splitting attack decreases UTXO values below the dust threshold; it is defined only under $a = a_{\text{partial}}$. Lemma 5.7: restatement of the forward direction of Proposition 5.2 under fee-rate parameterisation. Lemma 5.8: b_{unbound} is precisely the condition "recovery output is attacker-chooseable," which is the content of TM11. □ □

Table 5.2: The four proposed Bitcoin Script extensions located in the (f, a, g, b) design space. The column "dist." gives Hamming distance to the dominant corner $D^\star = (f_{\neq\text{anchor}}, a_{\text{atomic}}, g_{\text{key}}, b_{\text{bound}})$.

Design	f	a	g	b	dist.
CTV	f_{anchor}	a_{atomic}	g_{key}	b_{bound}	1
CCV	f_{inval}	a_{partial}	g_{keyless}	b_{bound}	2
OP_VAULT	f_{wallet}	a_{partial}	g_{key}	b_{bound}	1
CAT+CSFS	f_{acp}	a_{atomic}	g_{key}	b_{unbound}	1
$D^\star$	$\neq f_{\text{anchor}}$	a_{atomic}	g_{key}	b_{bound}	—

5.2.4 Structural Consequence

Define the design point

$$D^\star = (f_{\neq\text{anchor}},\ a_{\text{atomic}},\ g_{\text{key}},\ b_{\text{bound}}).$$

By Lemmas 5.5–5.8, $D^\star$ has $\mathcal{L}(\text{TM}_k) = 0$ for every $\text{TM}_k \in \{\text{TM1}, \text{TM2}, \text{TM4}, \text{TM11}\}$. Table 5.2 locates each proposed BIP in the (f, a, g, b) lattice together with its Hamming distance to $D^\star$.

Proposition 5.9. *No proposed Bitcoin Script extension occupies $D^\star$. Each of CTV, CCV, OP_VAULT, and CAT+CSFS differs from $D^\star$ in at least one dimension, and that difference directly entails $\mathcal{L}(\text{TM}_k) > 0$ for the threat $\text{TM}_k \in \{\text{TM1}, \text{TM2}, \text{TM4}, \text{TM11}\}$ corresponding to the distinguishing dimension via Lemmas 5.5–5.8.*

Proof. Read Table 5.2 against the lemmas. CTV differs in f (f_{anchor} rather than $f_{\neq\text{anchor}}$), so by Lemma 5.5 $\mathcal{L}(\text{TM1}) = V > 0$. CCV differs in a and g, so by Lemmas 5.6 and 5.7 both $\mathcal{L}(\text{TM4}) > 0$ (at sufficiently high r) and $\mathcal{L}(\text{TM2}) > 0$. OP_VAULT differs in a, so $\mathcal{L}(\text{TM4}) > 0$ at high r. CAT+CSFS differs in b, so $\mathcal{L}(\text{TM11}) = V$. No proposal matches all four coordinates of $D^\star$. □ □

Corollary 5.10. *The design point $D^\star$ is unoccupied by the current BIP corpus. A specification adopting $(f_{\neq anchor}, a_{atomic}, g_{key}, b_{bound})$ would achieve $\mathcal{L} = 0$ simultaneously against* $\{\mathrm{TM1}, \mathrm{TM2}, \mathrm{TM4}, \mathrm{TM11}\}$.

This corollary identifies a concrete direction for future covenant proposals. Two current proposals are one dimension away: CTV (by replacing its anchor-based fee model with a non-anchor alternative, which TRUC deployment approximates) and CAT+CSFS (by replacing its bare cold-key recovery leaf with an output-bound recovery path, as sketched by Poelstra [34] through recursive staging). TM3 (trigger key theft) is universal across the four proposals and is not addressed by the (f, a, g, b) decomposition; we discuss its incorporation in a future framework (Chapter 7).

5.2.5 Relationship to Proposition 5.2

Proposition 5.2 is the projection of the design-space analysis onto the (g, b) sublattice. Lemma 5.7 is its formal restatement under the present framework. Where Proposition 5.2 captures a single structural tradeoff (recovery-gating against griefing), the 4-dimensional decomposition captures the full set of tradeoffs visible at the covenant Script layer. Future BIPs can be localised in the lattice by reading their specification; their vulnerability profile follows from Lemmas 5.5–5.8 without further measurement.

5.2.6 Fee-Rate Sensitivity of TM4

The TM4 counterexample in the proof of Proposition 5.9 is the only one whose existence depends on the fee rate. We characterise the dependence because the threshold $r^\dagger$ is the most operationally consequential parameter for practitioners deploying CCV or OP_VAULT

vaults. For the remainder of this subsection, fix TM = TM4 and consider the attacker classes CTV (immune) versus CCV and OP_VAULT (vulnerable).

Fee pinning cost. The fee pinning attack against CTV requires constructing 25 descendant transactions off the unvault anchor output. The attack cost is:

$$C_{\text{pin}} = 25 \times (\text{dustRelay} + 1) \times \text{vsize}_{\text{desc}} + \text{anchor_sats} \tag{5.1}$$

where dustRelay = 3 sat/vB is the dust relay fee, $\text{vsize}_{\text{desc}} \approx 110\,\text{vB}$ per descendant, and anchor_sats = 3,300. This yields $C_{\text{pin}} \approx 14{,}300$ sats, independent of the prevailing fee rate r. Fee pinning cost is *fee-invariant*: the attacker pays dust-rate fees regardless of the fee environment.

Watchtower exhaustion cost. The per-UTXO recovery-cost scaling attack against CCV or OP_VAULT requires the attacker to repeatedly split the vault into smaller UTXOs via `trigger_and_revault`. Each split costs the attacker $\text{vsize}_{\text{trigger}} \times r$ in fees. The number of splits needed to exhaust the vault to dust-unrecoverable levels. A UTXO becomes *dust* (uneconomical to recover) when its value falls below the recovery transaction fee $\text{vsize}_{\text{recover}} \times r$. The number of splits needed to reduce every fragment below this dust threshold is:

$$N_{\text{dust}}(r) = \left\lfloor \frac{V}{\text{vsize}_{\text{recover}} \times r} \right\rfloor \tag{5.2}$$

where V is the vault value in sats, $\text{vsize}_{\text{recover}}$ is the recovery transaction size, and r is the fee rate. N_{dust} gives the number of fragments at which each fragment's value equals the cost to recover it. As r increases, the dust threshold rises, so fewer splits are needed to make individual UTXOs unrecoverable. The total attacker cost is:

$$C_{\text{exhaust}}(r) = N_{\text{splits}}(r) \times \text{vsize}_{\text{trigger}} \times r \tag{5.3}$$

Dust-threshold fee rate. The defender-loss under TM4 is dominated by whether individual dust UTXOs can be economically recovered. Define the single-block feasibility threshold $r^{\dagger}_{\text{TM4}}(V, D, b)$ as the fee rate at which an attacker can fully exhaust a vault of value V within one block under defender batching strategy b. Using measured vsize values from Chapter 4, the unbatched thresholds for a 0.5 BTC vault are approximately $r^{\dagger}(V, \text{CCV}, 0) \approx 66\,\text{sat/vB}$ and $r^{\dagger}(V, \text{OP_VAULT}, 0) \approx 59\,\text{sat/vB}$. Below these rates, the TM4 attack is infeasible; above them, $\mathcal{L}(\text{TM4})$ approaches V and the design becomes vulnerable. The CTV fee-pinning cost of Equation 5.1 is negligible relative to V for any practical vault size, so CTV dominates CCV and OP_VAULT under TM4 at any r, consistent with the TM4 counterexample in the proof of Proposition 5.9.

Scope of the threshold analysis. Equations 5.2–5.3 assume single-input recovery by the defender ($b = 0$). BIP-345 and BIP-443 both permit batched recovery in which N Unvaulting UTXOs are spent into a single recovery transaction, amortising fixed overhead across inputs. Section 5.4.1 revisits the threshold under the batched strategy ($b = 1$); the batched thresholds are approximately 1.8× (CCV) and 2.7× (OP_VAULT) the unbatched values and reverse the two designs' relative vulnerability under TM4.

Sensitivity to vault value. The crossover fee rate r^* depends on the vault value V. From Equation 5.2, the number of blocks needed for per-UTXO recovery-cost scaling is:

$$B(r, V) = \frac{V}{\text{vsize}_{\text{recover}} \times r \times S} \tag{5.4}$$

Table 5.3: Crossover fee rate r^* as a function of vault value V for CCV. Below r^*, per-UTXO recovery-cost scaling requires more than $B_{\max}$ blocks (infeasible). Above r^*, exhaustion completes within $B_{\max}$ blocks.

Vault value V (BTC)	r^* at $B_{\max} = 10$ (sat/vB)	r^* at $B_{\max} = 1$ (sat/vB)
0.01	0.1	1.3
0.10	1.3	13.3
0.50	6.6	66.4
1.00	13.3	132.8
5.00	66.4	664.0
10.00	132.8	1,328

where S is the splits per block (6,172 for CCV). Setting $B = B_{\max}$ (a practical feasibility threshold) and solving for r:

$$r^*(V) = \frac{V}{\text{vsize}_{\text{recover}} \times B_{\max} \times S} \tag{5.5}$$

Table 5.3 shows r^* across vault values at two thresholds: $B_{\max} = 10$ blocks (∼100 minutes, a sustained but feasible attack) and $B_{\max} = 1$ block (single-block exhaustion).

Defender-budget interpretation. Equation 5.5 parameterises r^* by the attacker's block-budget $B_{\max}$. The equivalent defender-side parameter is $N_{\text{budget}} = B_{\max} \cdot S_{\text{recover}}$, the number of Unvaulting UTXOs the watchtower can recover within $B_{\max}$ blocks at recovery throughput S_{recover} (blocks-worth of recoveries per block, $S_{\text{recover}} = \lfloor 4{,}000{,}000/(4 \cdot \text{vsize}_{\text{recover}}) \rfloor$ under single-input recovery, larger under batching per Section 5.4.1). The threshold $r^\dagger$ scales linearly in N_{budget}: doubling the defender's recovery capacity doubles the fee rate at which the attack becomes feasible.

The crossover scales linearly with vault value: larger vaults require higher fee rates before exhaustion becomes feasible (more splits needed). A 0.1 BTC vault is vulnerable to single-

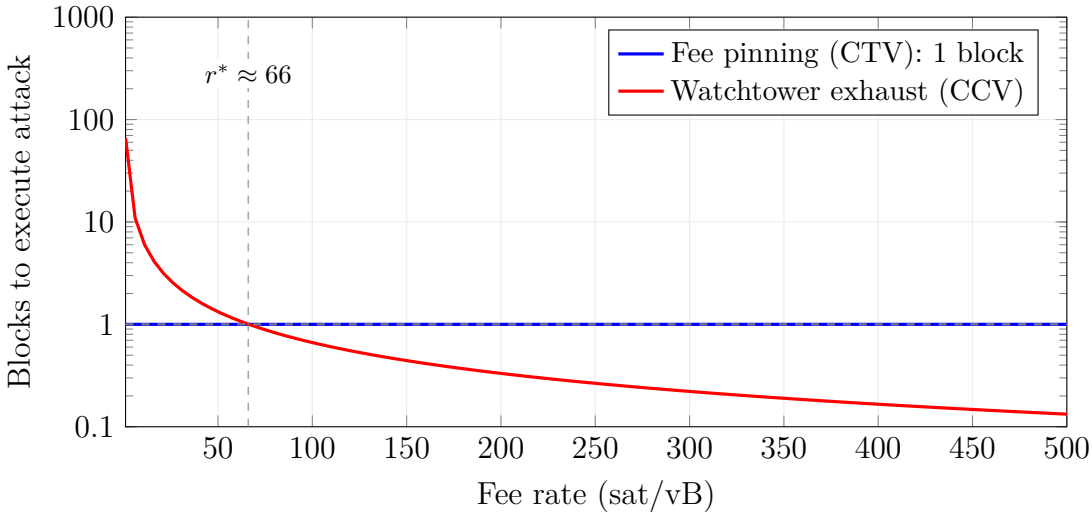

Figure 5.1: Fee-dependent security inversion. CTV fee pinning (blue) executes in one block regardless of fee rate. CCV per-UTXO recovery-cost scaling (red) requires progressively fewer blocks as fees rise, because each split creates a larger dust threshold. Below ∼50 sat/vB, exhaustion requires hundreds of blocks (infeasible). Above $r^* \approx 66$ sat/vB, exhaustion completes within a few blocks, making it practical. The crossover defines where the safety ranking inverts.

block exhaustion at 13 sat/vB—well within normal fee ranges. A 5 BTC vault requires 664 sat/vB for single-block exhaustion, which has occurred only during extreme congestion events. This parameterization gives practitioners a direct tool: given their vault value and risk tolerance, they can determine the fee regime where CCV/OP_VAULT's watchtower exposure exceeds CTV's fee pinning risk.

Figure 5.1 illustrates the crossover for a 0.5 BTC vault. The fee pinning cost curve is flat; the per-UTXO recovery-cost scaling curve decreases hyperbolically as fees rise.

5.3 Class Z1: Fee-Channel Pinning (TM1)

Fee pinning exploits CTV's anchor-based fee management. The unvault transaction includes an anchor output at `vout=1` (3,300 sats) to enable CPFP fee bumping. An attacker with the

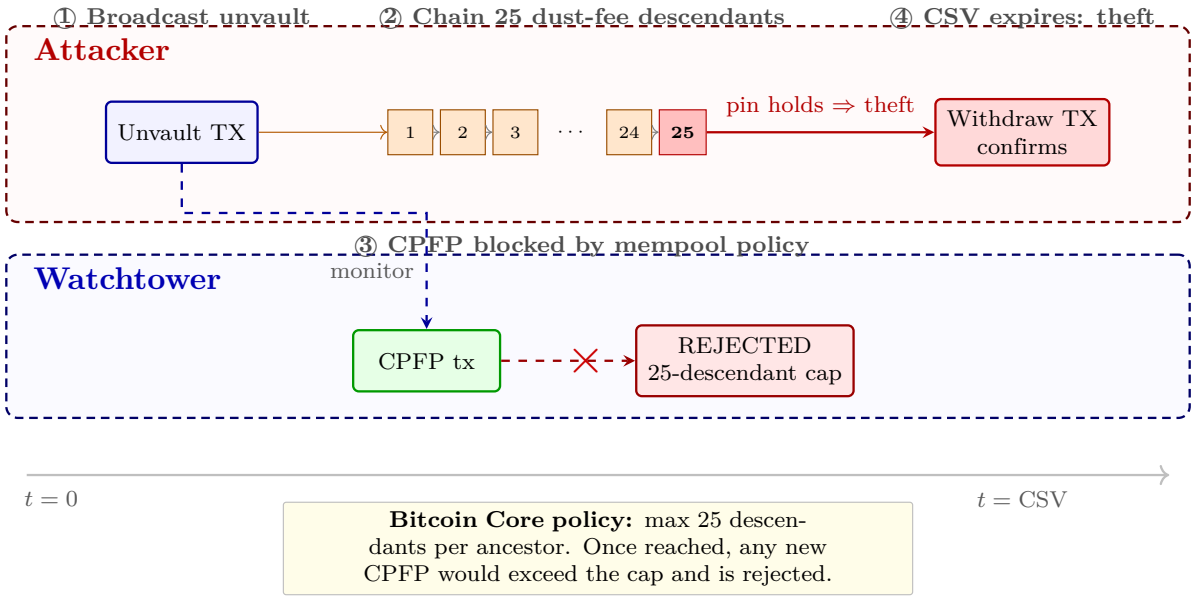

Figure 5.2: Fee pinning attack against CTV (**TM1**). ① Attacker broadcasts the unvault, starting the CSV window. ② Attacker chains 25 dust-rate descendants off the anchor output, filling the mempool descendant limit. ③ The watchtower's CPFP recovery is rejected by mempool policy (adding it would exceed the 25-descendant cap). ④ When CSV expires, the attacker's pre-committed withdrawal confirms.

hot key broadcasts the unvault, then immediately constructs 25 low-fee descendant transactions spending the anchor output. Bitcoin Core's mempool policy limits descendant chains to 25 transactions; once the limit is reached, the watchtower's CPFP recovery transaction is rejected because it would exceed the descendant count.

The attack cost function (Equation 5.1) evaluates to approximately 14,300 sats at dust-rate fees, regardless of the prevailing fee environment. At a vault value of 0.5 BTC (50,000,000 sats), the attack costs 0.03% of the vault value—well below any economic rationality threshold.

CCV, OP_VAULT, and CAT+CSFS are immune to this specific attack. CCV and OP_VAULT do not use anchor outputs (fees are paid via separate mechanisms). CAT+CSFS uses `SIGHASH_SINGLE|ANYONECANPAY`, which allows external fee inputs without an anchor

output.

Timing assumption. The attack requires the attacker to broadcast all 25 descendant transactions and have them accepted into mempools *before* the watchtower broadcasts its CPFP recovery transaction. On mainnet, a well-configured watchtower would have its CPFP transaction pre-constructed and ready to broadcast immediately upon detecting the unvault. The attack therefore depends on a propagation race: the attacker must fill the descendant chain across a sufficient fraction of the network's mempools before the watchtower's single CPFP transaction propagates. Our regtest experiment (Chapter 4) demonstrates that the descendant chain construction is structurally valid—the 25-transaction chain is accepted and the CPFP is rejected—but cannot test the propagation race, because regtest has a single mempool with no network latency. The practical feasibility of this attack on mainnet depends on the relative propagation speeds of the attacker's 25 transactions versus the watchtower's single transaction, which is a network-layer question outside the scope of this work (see Section 7.1 for further discussion).

TRUC/v3 transactions (merged in Bitcoin Core 28.0) limit descendant chains to a single transaction. If CTV vault transactions opt into TRUC, the 25-descendant pinning vector is eliminated at the relay policy level, potentially changing the crossover result of Proposition 5.9.

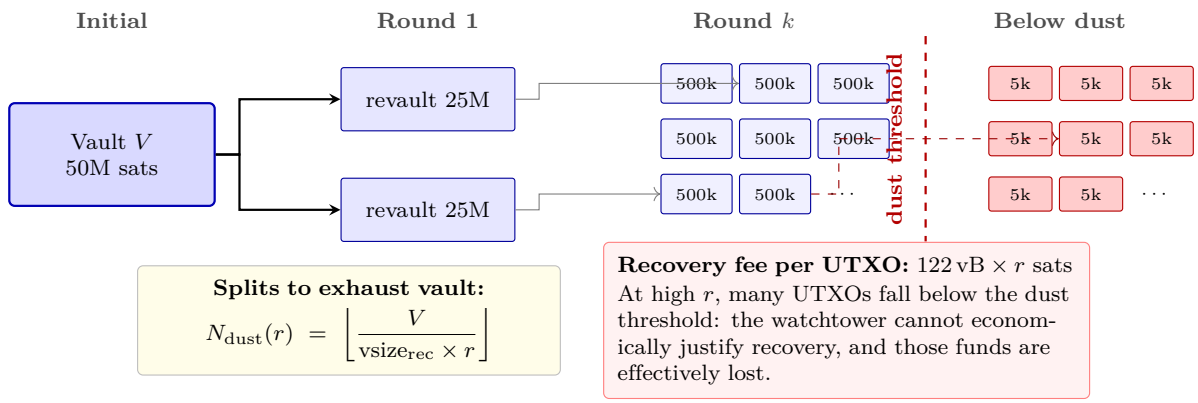

Figure 5.3: Watchtower exhaustion via `trigger_and_revault` (**TM4**). The attacker (holding the trigger key) repeatedly splits the vault into smaller UTXOs. As UTXOs shrink, the per-recovery fee ($\text{vsize}_{\text{rec}} \times r$) consumes an increasing fraction of each UTXO's value. Once a UTXO's value falls below $\text{vsize}_{\text{rec}} \times r$, the watchtower cannot economically justify recovery—those funds are effectively lost.

5.4 Class Z3: Per-UTXO Recovery-Cost Scaling (TM4)

CCV and OP_VAULT support partial withdrawals via `trigger_and_revault`, which splits the vault into a withdrawal portion and a re-locked remainder. An attacker with the trigger key can repeatedly invoke this operation, splitting the vault into progressively smaller UTXOs. When individual UTXOs become small enough that the recovery transaction fee exceeds the UTXO value, those funds are effectively lost—the watchtower cannot economically justify recovering them.

Using the measured transaction sizes from Chapter 4, we compute the splitting capacity per block. Each `trigger_and_revault` transaction weighs approximately 648 WU for CCV and 1,168 WU for OP_VAULT. The maximum block weight is 4,000,000 WU, yielding:

$$\text{CCV splits/block} \approx \frac{4{,}000{,}000}{648} \approx 6{,}172 \tag{5.6}$$

$$\text{OP_VAULT splits/block} \approx \frac{4{,}000{,}000}{1{,}168} \approx 3{,}427 \tag{5.7}$$

Harding [18] estimated approximately 3,000 splits per block based on assumed OP_VAULT-sized transactions. Our measurement confirms this for OP_VAULT (3,427, within 14% of the estimate) but reveals that CCV achieves approximately 80% more splits per block due to its smaller trigger transaction (162 vB vs. 292 vB). This correction matters because CCV (BIP-443) is the active proposal that replaced OP_VAULT (BIP-345): the design that Bitcoin is most likely to adopt has a larger per-UTXO recovery-cost scaling surface than previously estimated.

The watchtower's recovery budget determines how many splits it can absorb. At fee rate r, recovering a single split costs $\text{vsize}_{\text{recover}} \times r$ sats ($122r$ for CCV, $246r$ for OP_VAULT). The total recovery budget needed to defend against N splits is $N \times \text{vsize}_{\text{recover}} \times r$. For a 0.5 BTC vault at 100 sat/vB, defending against 1,000 splits costs $1{,}000 \times 122 \times 100 = 12{,}200{,}000$ sats (0.122 BTC)—roughly 24% of the vault value.

5.4.1 Batched Defender Strategy

The analysis above assumes single-input recovery: one transaction per recovered Unvaulting UTXO. BIP-345's "Batching" section (authorized recovery, lines 613–621) and BIP-443's default aggregation mode both permit the defender to recover N UTXOs in a single transaction, with vsize scaling as $\alpha + \beta \cdot N$ where α is fixed overhead and β is the marginal per-input cost. We measure both constants empirically on regtest, sweeping $N \in \{2, 5, 10, 25, 50\}$ and

extracting (α, β) by linear regression:

Table 5.4: Measured batched-recovery constants on regtest. The fit $\text{vsize}(N) = \alpha + \beta \cdot N$ replaces the single-input $\text{vsize}_{\text{recover}}$ with an amortized per-UTXO cost $\alpha/N + \beta \to \beta$ for large N.

Design	α (vB)	β (vB)	Unbatched $r^\dagger$	Batched $r^\dagger$	Factor
CCV	54	68	66 sat/vB	119 sat/vB	1.8×
OP_VAULT	154	92	59 sat/vB	159 sat/vB	2.7×

Substituting β for $\text{vsize}_{\text{recover}}$ in Equation 5.5 yields the batched dust-threshold $r^\dagger_{\text{batched}}(V, D) = V/(\beta(D) \times S(D))$. For a 0.5 BTC vault the batched thresholds rise to 119 sat/vB for CCV and 159 sat/vB for OP_VAULT—roughly 1.8× and 2.7× the unbatched values. The larger multiplier for OP_VAULT reflects its larger unbatched recovery transaction (246 vB): batching amortizes that overhead more aggressively than CCV's already-small 122 vB recovery.

Batching flips the TM4 ordering. Unbatched, CCV is safer than OP_VAULT against per-UTXO recovery-cost scaling ($r^\dagger$ 66 vs 59). Once the defender batches, the ordering flips: OP_VAULT becomes safer ($r^\dagger$ 159 vs 119). The intuition is that CCV's lead under single-input recovery comes from a smaller $\text{vsize}_{\text{recover}}$, but under batching the relevant cost is the marginal per-input β, which is nearly equal across the two designs (68 vs 92 vB). OP_VAULT's lower splits-per-block (S = 3,427 vs 6,172) then dominates, yielding the higher $r^\dagger$. The operational implication is that the "CCV has a larger per-UTXO recovery-cost scaling surface" conclusion of the unbatched analysis reverses once watchtower operators adopt batched recovery—a non-trivial finding for vault designers choosing between BIP-443 and BIP-345.

Constraints on batched recovery. BIP-345's authorized recovery permits batching across vaults with distinct `<recovery-sPK-hash>` values (§"Batching" lines 618–621), allowing a single watchtower to serve multiple independent users in one transaction. Unauthorized recovery is capped at two outputs (lines 389–391) and thus limited to same-`recovery-sPK` batches. BIP-443's default aggregation has no taptree-equivalence requirement and is strictly more permissive. The attacker's `trigger_and_revault` sequence is inherently serial—each revault consumes the previous output—so the attacker does not benefit from batching. The measured savings accrue asymmetrically to the defender.

CTV, CAT+CSFS, and Simplicity are immune to per-UTXO recovery-cost scaling because they do not support partial withdrawals. The entire vault balance must be unvaulted atomically, so the watchtower needs only one recovery transaction regardless of attack persistence.

5.5 Class Z2: Permissionless Griefing (TM2)

Recovery griefing is a denial-of-service attack where the adversary repeatedly invokes recovery to prevent the vault owner from completing withdrawals. The economic asymmetry between attacker and defender costs determines the severity.

Table 5.5 compares the per-round costs. The griefing ratio $\gamma = \text{vsize}_{\text{attack}}/\text{vsize}_{\text{defend}}$ measures the cost asymmetry: $\gamma < 1$ means the attacker pays less than the defender per round (favorable to the attacker).

$$\gamma = \frac{\text{vsize}_{\text{attack}}}{\text{vsize}_{\text{defend}}} \tag{5.8}$$

CCV has the most severe griefing profile: the attack is keyless (anyone can invoke recovery) and the cost asymmetry strongly favors the attacker ($\gamma = 0.27$). OP_VAULT eliminates

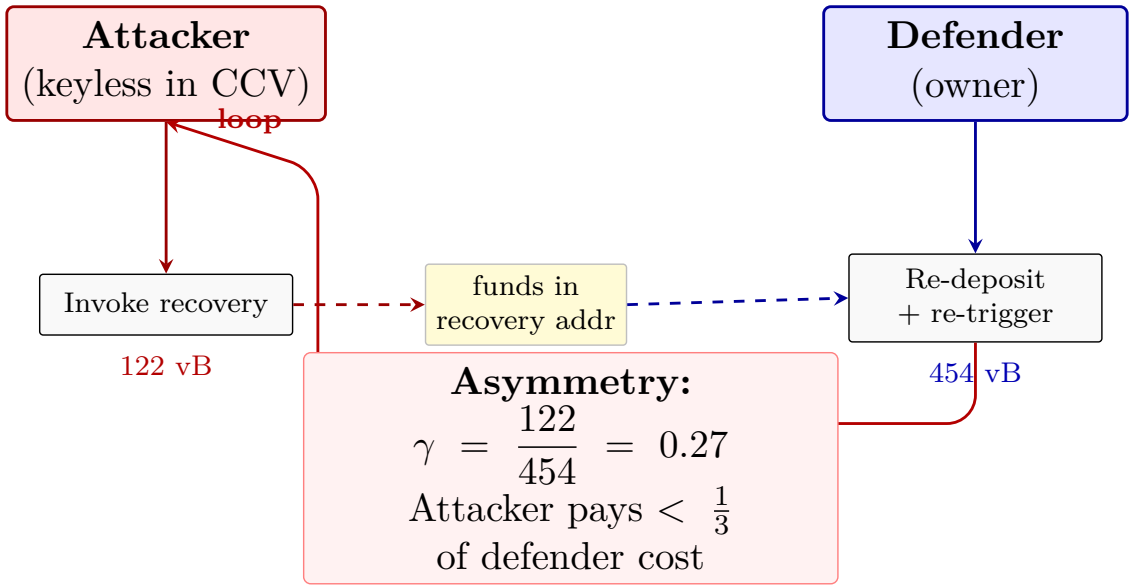

Figure 5.4: Recovery griefing loop (**TM2**) for CCV. The attacker triggers permissionless recovery (122 vB); the owner must re-deposit and re-trigger (454 vB) to resume withdrawal; the attacker invokes recovery again. The per-round cost asymmetry $\gamma = 0.27$ strongly favors the attacker.

anonymous griefing by requiring the `recoveryauth` key, raising the attacker bar substantially. CTV's griefing direction is reversed: the attacker triggers an unvault (94 vB) and the defender sweeps to cold (133 vB). CAT+CSFS griefing requires the cold key, but cold key compromise already constitutes total fund theft (TM11), making the griefing scenario academically moot.

5.6 Class Z4: Hot-Key Theft via Destination Substitution (TM3)

Trigger key compromise has different consequences across designs because the vault's output constraints determine what the attacker can do with the triggered funds.

Table 5.5: Recovery griefing cost comparison. The attacker invokes recovery; the defender re-deposits and re-triggers. $\gamma < 1$ favors the attacker.

Covenant	Key required	Attacker (vB/round)	Defender (vB/round)	γ	Assessment
CCV	None (keyless)	122	454	0.27	Severe (anyone)
OP_VAULT	recoveryauth	246	446	0.55	Low (key needed)
CTV	Hot key	94	133	0.71	Moderate (reversed)
CAT+CSFS	Cold key	125	343	0.36	Irrelevant (TM11)

CTV. The hot key can trigger the unvault transaction and, after the CSV delay, complete the withdrawal to the pre-committed destination. If the attacker also holds the fee key (enabling fee pinning to block recovery), this escalates to fund theft. Hot key alone produces liveness denial only—the defender sweeps to cold storage.

CCV. The trigger key can direct the withdrawal to an attacker-controlled address (the destination is chosen at trigger time, not fixed at creation). The watchtower races to invoke keyless recovery (122 vB). Since recovery is permissionless, the watchtower needs no additional key material to respond.

OP_VAULT. The trigger key can redirect withdrawals, similar to CCV. The watchtower races to invoke authorized recovery (246 vB), which requires the `recoveryauth` key. If the attacker compromises both the trigger key and the recoveryauth key, the result is persistent liveness denial (not theft), because the recovery address is pre-committed in the vault's Taproot internal key tweak and cannot be changed.

CAT+CSFS. The hot key can only trigger to the pre-committed vault-loop output. The dual CSFS+CHECKSIG verification binds the output to the embedded `sha_single_output` constant—changing the output changes the sighash, invalidating the signature. The hot key

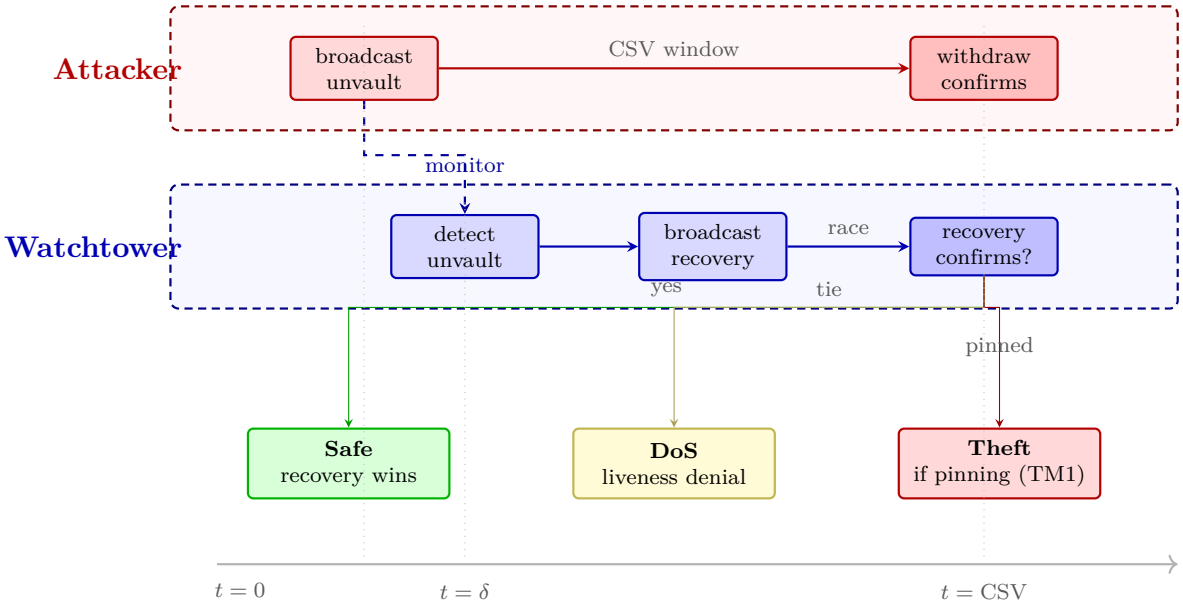

Figure 5.5: Trigger key theft race (**TM3**). The attacker broadcasts an unvault (upper lane). The watchtower detects it at time δ and broadcasts recovery (lower lane). The outcome hinges on whether recovery confirms before CSV expiry: safe if recovery wins, DoS if tied, theft if the attacker successfully pins the watchtower's transaction (combining TM3 with TM1).

holder is limited to griefing (triggering unwanted unvaults that the defender can recover from). This is the strongest hot-key theft resistance of any design studied.

5.7 Class Z5: Cold-Key Theft via Unconstrained Recovery (TM11)

Cold key compromise has asymmetric consequences depending on whether the recovery path constrains outputs.

CAT+CSFS. The recovery leaf is `cold_pk OP_CHECKSIG` with `SIGHASH_DEFAULT`: no introspection, no timelock, no output constraint. Cold key compromise grants immediate,

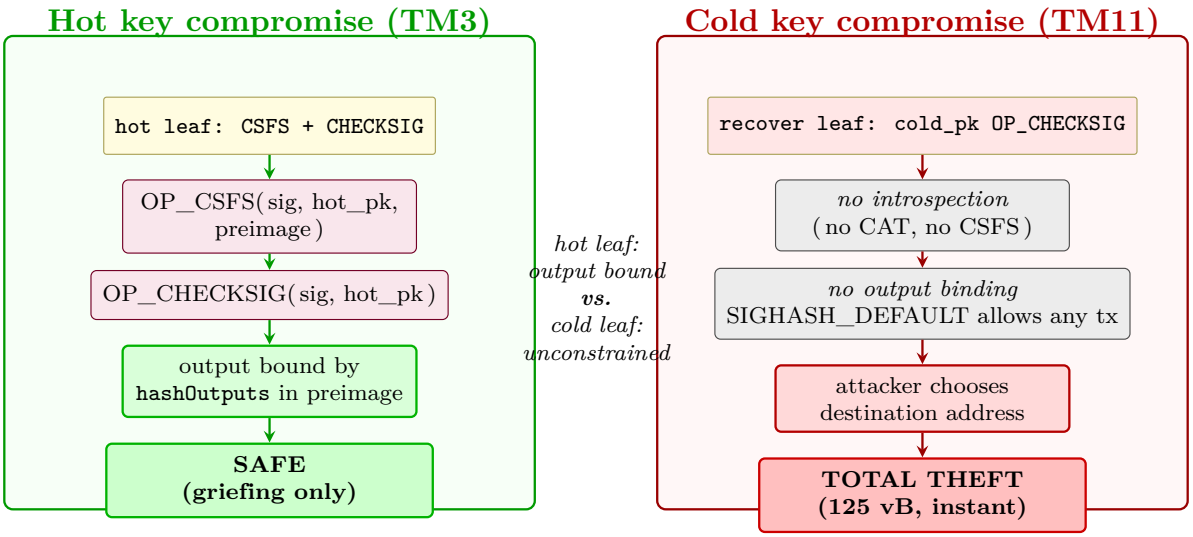

Figure 5.6: CAT+CSFS asymmetric key safety. **Left:** hot key compromise leaks only the trigger authority—the dual CSFS + CHECKSIG binding forces outputs to match the embedded `sha_single_output` constant (**TM3** reduces to griefing). **Right:** cold key compromise yields total theft (**TM11**): the recovery leaf is bare `OP_CHECKSIG`—no timelock, no introspection, no destination constraint. The attacker sweeps funds to any address in one 125 vB transaction.

unrestricted theft—the attacker sends all funds to an arbitrary address in a 125 vB transaction indistinguishable from legitimate recovery on-chain. Of the four designs, this is the worst-case TM11 profile.

CTV. The cold sweep is CTV-committed, sending funds to the pre-committed cold address. Cold key compromise allows forced recovery but only to that address. The attacker cannot redirect funds.

OP_VAULT. Recovery is output-constrained to the pre-committed recovery address (embedded in the Taproot internal key tweak). Cold key compromise plus recoveryauth key compromise yields persistent denial of service but not theft.

CCV. Recovery is keyless and output-constrained. There is no cold key to compromise in the recovery path.

Simplicity. Both trigger and recovery enforce `jet::outputs_hash()`, constraining recovery to the pre-committed address. Cold key compromise forces recovery but cannot redirect funds—matching CCV and OP_VAULT's output-constrained model.

The recovery safety ranking is: CCV (keyless, constrained) > OP_VAULT (key-gated, constrained) > CTV (key-gated, constrained) > CAT+CSFS (key-gated, *unconstrained*). Poelstra [34] proposed an alternative CAT+CSFS recovery design using recursive staging resets, where cold key compromise leads to a liveness battle rather than immediate theft. Under that design, the attacker and owner take turns resetting the vault state, each paying transaction fees per round. This would upgrade CAT+CSFS's recovery safety to match OP_VAULT's (liveness denial, not theft), but at the cost of a larger recovery script and additional witness overhead.

5.8 Projection onto the (g, b) Sublattice

The design-space decomposition of Section 5.2.2 induces a natural projection onto the (g, b) sublattice—the two recovery-path dimensions—whose structure captures the recovery-related tradeoffs without appeal to per-design measurement.

Lemma 5.11. *Let $D = (f, a, g, b)$ be a design with uniform leaf binding (hot and cold paths share the same b). Then:*

1. *As g weakens from g_{key} to $g_{keyless}$, $\mathcal{L}(\mathrm{TM2})$ increases by Lemma 5.7, while the key-loss attack surface vanishes.*

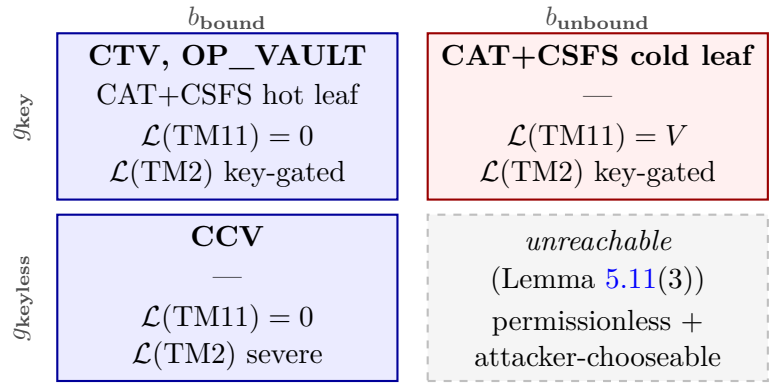

Figure 5.7: Projection of the covenant design space onto the (g, b) sublattice. Three cells are occupied by the four proposed Bitcoin Script extensions; the fourth is structurally unreachable. CAT+CSFS appears in two cells by Remark 5.12.

2. *As b weakens from b_{bound} to $b_{unbound}$, $\mathcal{L}(\text{TM11})$ increases from 0 to V by Lemma 5.8.*

3. *The cell $(g_{keyless}, b_{unbound})$ is unreachable: under Proposition 5.2, a keyless vault with unbound recovery admits any party to invoke recovery to any destination, which is vacuous as a custody construction. No proposed BIP occupies this cell.*

Proof. Parts 1 and 2 are direct reinstatements of Lemmas 5.7 and 5.8 with the g and b axes read individually. Part 3: if $\mathcal{A}(\emptyset) = 1$ and the recovery output is attacker-chooseable, any third party can sweep the vault to their own address, reducing the construction to a fee-free taproot `OP_CHECKSIG` on a keyed path with no custody benefit. □ □

Remark 5.12. Designs with *asymmetric* leaf binding—hot leaf output-bound and cold leaf unbound, as in CAT+CSFS—occupy two distinct cells in this projection, one per leaf. The hot leaf sits at $(g_{\text{key}}, b_{\text{bound}})$; the cold leaf sits at $(g_{\text{key}}, b_{\text{unbound}})$. The "off-axis" position of CAT+CSFS in a naive scatter visualisation is a direct consequence of this leaf-level asymmetry, not an anomaly requiring a separate axis.

A practitioner choosing between designs must evaluate both dimensions relative to their threat model. An exchange prioritizing hot-key theft resistance (the most likely attack vector

for an online system) would favor CAT+CSFS, accepting the cold-key risk and mitigating it through hardware security modules. A custodian prioritizing recovery flexibility (e.g., an estate planning trust that must recover under uncertain key availability) would favor CCV's keyless recovery, accepting the griefing surface.

5.9 Threat Model Matrix

Table 5.6 presents the complete threat model matrix across all eleven attack classes and four Bitcoin covenant designs. Each cell summarizes the attack's severity and the measured vsize cost. The Simplicity mapping is omitted from the matrix because it runs on a federated sidechain with fundamentally different trust assumptions (Section 2.9); its per-TM mapping appears in Section 4.8.

5.10 Deployment Guidance

Proposition 5.9 and the per-threat analysis yield fee-conditional deployment recommendations:

- **Low-fee environments** ($r < 10$ sat/vB). Watchtower exhaustion is economically infeasible (too many splits needed, each recovery cheap). Fee pinning is cheap but CTV's exposure is small relative to vault value. CCV or OP_VAULT are preferred because their additional capabilities (partial withdrawal, batching) come at negligible security cost.
- **High-fee environments** ($r > 100$ sat/vB). Watchtower exhaustion becomes viable against CCV and OP_VAULT—the watchtower's recovery budget scales linearly with

Table 5.6: Threat model matrix (TM1–TM11). Severity labels are bolded uniformly; every "N/A" cell carries a one-phrase reason. Severity scale: **N/A** = structurally impossible; **Low** = requires difficult preconditions; **Moderate** = feasible under specific conditions; **High** / **Severe** = significant risk under realistic assumptions; **Safe** = correct behaviour by design.

TM	CTV	CCV	OP_VAULT	CAT+CSFS
1	**High.** 25-desc chain blocks CPFP (14,300 sats).	**N/A** (no anchor).	**N/A** (fee wallet).	**N/A** (external fee input).
2	**Moderate.** Hot key triggers; defender sweeps cold (133 vB).	**Severe.** Keyless, γ=0.27.	**Low.** Auth key needed, γ=0.55.	**Low.** Cold key needed; moot if TM11.
3	**High.** Hot+fee = theft; hot alone = DoS.	**High.** Redirect withdrawal; watchtower races (122 vB).	**Moderate.** Redirect; authorised recovery races (246 vB).	**Low.** Grief only (dual verification).
4	**N/A** (no revault).	**Severe.** 6,172 splits/block.	**Severe.** 3,427 splits/block.	**N/A** (no revault).
5	**N/A** (no xpub derivation).	**N/A** (no xpub derivation).	**Moderate.** xpub expansion leaks children.	**N/A** (no xpub derivation).
6	**High.** Hot+fee = theft.	**N/A** (single key).	**High.** Trigger+auth = persistent DoS.	**High.** Cold key alone = theft (TM11).
7	**High.** Stuck funds on address reuse.	**Safe.**	**Safe.**	**Safe.**
8	**N/A** (no mode byte in CTV).	**Developer footgun.** Undefined mode → `OP_SUCCESS`.	**N/A** (no mode byte in `OP_VAULT`).	**N/A** (no mode byte in CAT+CSFS).
9	**N/A** (no CSFS path).	**N/A** (no CSFS path).	**N/A** (no CSFS path).	**Impossible.** Dual verification.
10	**N/A** (no CSFS witness to manipulate).	**N/A** (no CSFS witness to manipulate).	**N/A** (no CSFS witness to manipulate).	**Impossible.** SHA-256 binding.
11	**N/A** (CTV-committed recovery).	**N/A** (keyless recovery; no cold key).	**N/A** (Taproot-tweak-constrained recovery).	**High.** Unconstrained CHECKSIG = immediate theft.

fee rate, and dust-sized UTXOs become unrecoverable faster. CTV's fee pinning cost remains negligible. CTV is preferred in high-fee environments for vaults that do not require partial withdrawal or batching.

- **Mixed or uncertain fee environments.** OP_VAULT offers the best compromise: it has the strongest griefing resistance (authorized recovery), supports partial withdrawal and batching, and its per-UTXO recovery-cost scaling exposure is partially mitigated by the higher per-split cost that slows the attacker. However, BIP-345 has been withdrawn; the closest active proposal with similar properties is CCV (BIP-443) with application-level recovery authorization.

- **Hot-key theft priority.** If the primary threat is hot-key compromise (e.g., an exchange or online service), CAT+CSFS provides the strongest resistance: the dual verification binding limits a hot-key attacker to griefing, with no path to fund redirection. The tradeoff is the cold-key vulnerability (TM11), which must be mitigated through operational security (hardware security modules, geographic distribution, multisig).

5.11 Class Z6: Mode/Parameter Bypass (TM8)

Class Z6 is enabled by axis A6 = `multi-mode-with-OP_SUCCESS`: a polymorphic covenant opcode takes a stack-provided mode selector, and the taproot `OP_SUCCESSx` upgrade mechanism promotes undefined mode values to unconditional script success. A developer or construction bug that selects an undefined mode silently disables the covenant check.

5.11.1 Susceptibility

CCV is susceptible and is the only proposal with A6 = `multi-mode-with-OP_SUCCESS`. BIP-443 defines four valid modes $\{-1, 0, 1, 2\}$; any other value falls through to `OP_SUCCESSx`, which under BIP-341/BIP-342 taproot rules terminates script execution with success regardless of stack state. Five undefined modes $\{3, 4, 7, 128, 255\}$ were measured on regtest; each produces full covenant bypass at 110 vB per spend. $\mathcal{L}(\text{TM8}) = V$.

5.11.2 Immunity Derivations

CTV, OP_VAULT, and CAT+CSFS have A6 = `single-mode`: each exposes a single semantic opcode with no mode selector, so the `OP_SUCCESSx` fallthrough surface does not arise.

5.11.3 Framing: Specified Behaviour, Not a Novel Vulnerability

The class is *specified behaviour* under BIP-443's forward-compatibility design (itself inherited from BIP-341/BIP-342), not a newly-discovered vulnerability. The contribution here is the systematic mode sweep and the class-level framing that ties the surface to axis A6, not the discovery of `OP_SUCCESSx` semantics. In practice, the pymatt reference library mitigates the risk by providing named constants (`CCV_FLAG_*`); the residual risk is in independent implementations that hand-write the mode byte. We include this class in the security analysis because the framework predicts the vulnerability surface from axis position; omitting it would leave axis A6 under-motivated.

5.11.4 Rationality and Scope

(1) Attacker capability. Any observer of a vault whose script contains an undefined mode byte; no keys required.

(2) Defender model. Developer discipline around mode-byte selection; library-provided named constants; code review.

(3) Economic preconditions. None applicable — full vault bypass at constant 110 vB/spend.

(4) Protocol / network assumptions. BIP-341/BIP-342 `OP_SUCCESSx` taproot upgrade semantics.

(5) Scope of validity. Applies iff A6 = `multi-mode-with-OP_SUCCESS`. The other three Bitcoin proposals are immune by construction.

(6) Rational-attacker check. Rational whenever a misconfigured vault exists on-chain; the attacker's payoff is V, the cost is a single spend of $110\,\text{vB} \times r$.

(7) Counter-factual mitigations. (a) A6 = `single-mode` (one opcode per primitive, as in the other three proposals); (b) bitmask-gated mode check at script-parse time (reject undefined modes before execution) — would eliminate the class without revising soft-fork semantics but would forfeit the forward-compatibility benefit; (c) pymatt-style named constants (`CCV_FLAG_*`) that make the undefined-mode surface unreachable from application code.

5.12 Rationality Blocks for Classes Z1–Z5

Each of the attack classes analysed in Sections 5.3–5.7 admits a seven-field *Rationality and Scope* block per the schema of Section 3.3. We compress the blocks below; Appendix D carries the uncompressed free-text form. Each field implicitly cites the Kerckhoffs threat model of Section 2.4.1.

5.12.1 Class Z1 (Fee-Channel Pinning)

(1) Attacker capability. Hot key; capital for the anchor-descendant chain ($\approx$ 14,300 sats at $\rho = 3\,\text{sat/vB}$).

(2) Defender model. Online watchtower; Bitcoin Core default mempool policy with 25-descendant limit (BIP-125).

(3) Economic preconditions. $\mathcal{L}$ and Adv are r-invariant; rational at any fee regime.

(4) Protocol / network assumptions. BIP-125 Rule 3 requires the watchtower's CPFP replacement to pay for evicting every descendant; 3 sat/vB Core default dustRelay is policy not consensus.

(5) Scope of validity. Applies iff A3 = `out-of-band-anchor`. N/A for dynamic fee models (CCV, OPV, CAT+CSFS).

(6) Rational-attacker check. Rational at any r given hot key; yield $\approx V$ at marginal cost.

(7) Counter-factual mitigations. (a) TRUC/BIP-431 collapses the descendant limit to 1, eliminating the class at the relay-policy level; (b) cluster-mempool redesign qualitatively changes the replacement calculus (scope-limit); (c) P2A-style smaller anchor output reduces the absolute cost of the attack but does not eliminate the structural surface.

5.12.2 Class Z2 (Permissionless Griefing)

(1) Attacker capability. Zero keys (for CCV). Observation of mempool plus the public recovery address.

(2) Defender model. Any; the attack is pure liveness denial and ends only when the defender abandons or rotates the vault.

(3) Economic preconditions. $\mathcal{L}$ scales linearly in r; $\gamma = \text{vsize}_{\text{atk}}/\text{vsize}_{\text{def}}$ is r-invariant.

(4) Protocol / network assumptions. Mempool-observable trigger tx; no RBF dependence.

(5) Scope of validity. Applies iff A1 = `keyless`. Reduces to a key-compromise attack (TM6) otherwise.

(6) Rational-attacker check. Irrational without external incentive (no financial gain). Rational only under extortion, competitor disruption, or state-actor threat.

(7) Counter-factual mitigations. A1 = `recoveryauth-key` (as in OP_VAULT) or key-gated recovery (CTV, CAT+CSFS) eliminates the class by forcing a signature on the recovery leaf.

5.12.3 Class Z3 (Per-UTXO Recovery-Cost Scaling)

(1) Attacker capability. Trigger key; capital for N partial unvaults.

(2) Defender model. Watchtower with recovery authority; batching flag b; minimal-disruption policy (reactive per-UTXO recovery) maximises attacker leverage. A full-sweep-on-first-alert policy collapses the class to one round at the cost of operational disruption.

(3) Economic preconditions. Rate-dependent; threshold $r^{\dagger}_{\text{TM4}}$ per Equation 5.5. Below the threshold the defender can absorb the per-UTXO liability before dust economics kick

in.

(4) Protocol / network assumptions. Bitcoin dust thresholds (546 sats P2WPKH, 330 sats P2TR). RBF rule 3 does not permit sibling-eviction, so the attacker's partial-withdrawal chain is single-chain. Minimum relay fee $\approx 1\,\text{sat/vB}$.

(5) Scope of validity. Applies iff A2 = `yes`. The class is a structural cost tail of partial withdrawal, independent of adversary behaviour; an attacker accelerates the distribution but is not required.

(6) Rational-attacker check. Rational only above $r^\dagger$ with external incentive (competitor, extortion). Pure liveness attacker is irrational at any r (no gain). Attacker with hot key can extract value from abandoned UTXOs post-CSV.

(7) Counter-factual mitigations. (a) Defender batching raises $r^\dagger$ by 1.8–2.7×; (b) full-sweep-on-first-alert collapses the loop; (c) operational limits on per-epoch trigger count.

Reframing note. Earlier framings of this class as "watchtower exhaustion" cast it as a multi-round attacker-vs-watchtower game. That framing does not survive a defender with full recovery authority who can full-sweep on first alert, collapsing the multi-round narrative to one round. The reframe above identifies the class as a structural cost tail of partial withdrawal — attacker-accelerable but not attacker-required — with the defender's degree of freedom in recovery batching. The multi-round attacker scenario remains one *threat-model instantiation* among several, documented in Appendix D.

5.12.4 Class Z4 (Hot-Key Theft via Destination Substitution)

(1) Attacker capability. Hot key; mempool access; optional fee-pinning capital (if class Z1 composes).

(2) Defender model. Online watchtower able to race during the CSV delay; race bias

toward attacker under class-Z1 composition (CTV).

(3) Economic preconditions. Race outcome r-independent except via class Z1.

(4) Protocol / network assumptions. CSV delay length; mempool propagation dynamics (Riard's cycling attack is out of scope).

(5) Scope of validity. Applies iff A4 $\neq$ `signature-dual-bind`.

(6) Rational-attacker check. Rational only paired with fee-pinning budget (composition with class Z1); otherwise the defender wins the race generically.

(7) Counter-factual mitigations. A4 = `signature-dual-bind` (CAT+CSFS) eliminates the class by cryptographic construction: the Schnorr EUF-CMA reduction forces the assembled preimage to equal the consensus preimage, binding `hashOutputs` and foreclosing destination substitution.

5.12.5 Class Z5 (Cold-Key Theft via Unconstrained Recovery)

(1) Attacker capability. Cold key (Kerckhoffs: the attack assumes compromise, not inference).

(2) Defender model. Any; the attack is instant with no CSV window, no watchtower intervention window.

(3) Economic preconditions. r-invariant.

(4) Protocol / network assumptions. N/A — pure signature check; no mempool dynamics.

(5) Scope of validity. Applies iff A5 = `plain-signature`.

(6) Rational-attacker check. Rational whenever cold key is compromised; yield V.

(7) Counter-factual mitigations. A5 = `signature-dual-bind` on the recovery leaf (Poelstra's sketch for CAT+CSFS-based bound recovery) would move the de-

sign to an immune axis position, eliminating the class. Alternative A5 values (`template-hash-digest`, `per-output-amount+script`, `dedicated-opcode`) as in CTV, CCV, OPV also produce immunity.

Chapter 6

Formal Verification

This chapter describes the Alloy bounded model checking methodology used to verify structural properties of the five covenant vault designs. Section 6.1 introduces Alloy and its applicability to protocol verification. Section 6.2 defines the properties checked and reports results. Section 6.3 describes how the formal, empirical, and economic evidence layers reinforce each other. The full model architecture and per-covenant detailed results appear in Appendix A.

6.1 Alloy and Bounded Model Checking

Alloy [20] is a declarative modeling language with automatic bounded analysis. An Alloy model specifies a universe of *signatures* (types), *relations* (fields), and *constraints* (facts and predicates). The Alloy Analyzer exhaustively searches all instances within a user-specified *scope* (maximum number of atoms per signature) to find counterexamples to assertions or satisfying instances for predicates.

Alloy's bounded model checking is well-suited to protocol verification for two reasons. First, the small-scope hypothesis—if a property has a counterexample, it typically has one within a small bound—means that modest scopes (6–10 atoms) provide high confidence without exploring astronomically large state spaces. Second, Alloy's relational logic naturally expresses state machines, ownership relations, and authorization policies that characterize vault pro-

tocols.

dgpv [15] applied Alloy to verify a single OP_CAT vault prototype (Rijndael's design [37]), checking fund conservation and covenant enforcement within that one design. Our work extends this approach to all four BIP-specified covenant protocols plus Simplicity, using a shared base model to ensure uniform property definitions across designs. The models comprise 9 Alloy files: two base modules (`btc_base`, `vault_base`), five concrete vault models, a threat model module, and a cross-covenant interaction module.

6.2 Properties and Results

We verify three categories of structural properties across the five vault designs.

Fund conservation. No transaction in the vault lifecycle creates value:

$$\forall\, t \in \text{Transactions} : \sum_{\text{out} \in t.\text{outputs}} \text{out.amount} \leq \sum_{\text{in} \in t.\text{inputs}} \text{in.amount} \tag{6.1}$$

No-extraction. An attacker with a specified key set cannot increase their holdings through any sequence of vault transitions:

$$\forall\, \text{path} \in \text{Executions},\ \forall\, a \in \text{Attackers} : a.\text{holdings}_{\text{final}} \leq a.\text{holdings}_{\text{initial}} \tag{6.2}$$

We check this under three attacker profiles per design: no keys, hot key only, and cold key only. The one expected failure is CAT+CSFS under cold-key-only compromise: the Alloy model finds a counterexample where the cold key holder redirects recovery funds to an arbitrary address, structurally confirming TM11 (Section 5.7).

Table 6.1: Alloy verification results. 34 of 35 safety checks pass; the one failure confirms cold key extraction in CAT+CSFS (TM11).

Model	Checks	Scenarios	Key findings
CTV	6 pass	3 instances	Address reuse produces stuck UTXO
CCV	7 pass	5 instances	Griefing reachable; mode confusion contained
OP_VAULT	8 pass	6 instances	Dual-key = DoS not theft; key-loss = permanent
CAT+CSFS	5 pass, 1 fail*	3 instances	Cold key extraction (TM11) confirmed
Simplicity	6 pass	3 instances	Recovery constrained; hot key = grief only
Cross	2 pass	5 instances	No cross-vault injection
Total	**34 pass, 1 fail**	**25 instances**	

*Expected failure: Alloy finds a counterexample confirming that cold key compromise enables fund extraction (TM11).

Recovery liveness. From any reachable vault state, a recovery transition exists:

$$\forall\, s \in \{\text{Funded}, \text{Triggered}\} : \exists\, t \in \text{Transitions} :\ t.\text{source} = s \ \wedge\ t.\text{target} = \text{Recovered} \tag{6.3}$$

This holds for all five designs. In OP_VAULT, recovery requires the `recoveryauth` key—the transition exists but is gated by key possession.

Summary of results. The Alloy Analyzer checked 41 assertions and found satisfying instances for 25 scenario predicates. Table 6.1 summarizes the results.

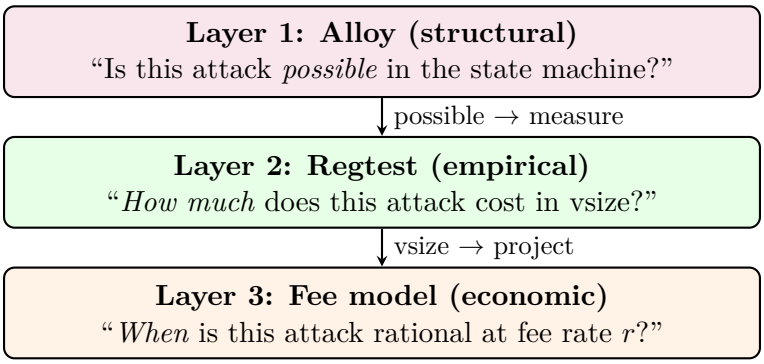

Figure 6.1: Three-layer evidence methodology. Alloy determines structural possibility. Regtest measures concrete costs. The fee model projects costs into real-world fee environments to determine economic rationality.

6.3 Three-Layer Evidence Methodology

The formal verification complements the empirical measurements and economic projections to form a three-layer evidence methodology (Figure 6.1). Each layer answers a different question about the same attack:

- **Layer 1 (Alloy): Is the attack structurally possible?** If the Alloy check passes (no counterexample), the attack is structurally impossible within the modeled scope. If a counterexample is found, the attack is possible and proceeds to Layer 2.
- **Layer 2 (Regtest): How much does it cost?** The regtest experiments measure the concrete vsize of the attack and defense transactions, converting a structural possibility into a quantitative cost.
- **Layer 3 (Fee model): When is it rational?** The fee projection model (Equation 3.2) converts vsize into sats at each fee rate, and the rationality condition (Equation 3.3) determines when the attack is economically justified.

Table 6.2 illustrates the alignment for three representative attacks.

Table 6.2: Three-layer evidence alignment for representative attacks.

Attack	Alloy (Layer 1)	Regtest (Layer 2)	Fee model (Layer 3)
CCV griefing (TM2)	Reachable (keyless recovery exists)	122 vB/round attacker, 454 vB/round defender	Rational at all fee rates; $\gamma = 0.27$
CTV fee pinning (TM1)	Reachable (anchor output spendable)	14,300 sats total	Rational whenever vault > 20,000 sats
CAT+CSFS cold theft (TM11)	Counterexample found (cold key extracts)	125 vB recovery tx	Rational at any fee rate (immediate theft)

The three layers reinforce each other: Alloy gives confidence that the regtest experiments measure the right attacks (no missed paths), the regtest measurements give concrete costs to the Alloy-identified possibilities, and the fee model transforms those costs into deployment guidance.

Chapter 7

Discussion

This chapter addresses limitations of the methodology, discusses mitigations for the vulnerabilities identified, and identifies open problems for future work.

7.1 Limitations

7.1.1 Regtest Environment

The regtest limitations detailed in Section 3.4 bound the external validity of all measurements. The most significant gap is the absence of temporal dynamics: recovery races between the attacker's withdrawal and the watchtower's response are instantaneous on regtest but uncertain on mainnet, where block confirmation times are stochastic and mempool congestion affects transaction propagation. The No Pareto-Optimal Design theorem (Proposition 5.9) is robust to this gap because its per-threat orderings rest on structural vsize measurements rather than timing—but the *practical feasibility* of per-UTXO recovery-cost scaling at a given fee rate depends on how quickly the attacker can fill blocks with split transactions, which regtest cannot test.

Table 7.1: Implementation choices in author-built vaults. These are design decisions, not BIP requirements; alternative designs would produce different measurements.

Choice	Covenant	BIP-required?	Alternative
Fixed destination	CAT+CSFS	No	Poelstra's dynamic destination [34]
Unconstrained recovery	CAT+CSFS	No	Output-constrained (like Simplicity)
500-sat fixed fee	Simplicity	No	Dynamic fee estimation
3,300-sat anchor	CTV (upstream)	No	Different anchor size

7.1.2 Reference Implementations

This work studies covenant *designs* (BIPs), not implementations. However, the measurements come from specific reference implementations, and implementation choices affect the measured vsize. The most significant example is OP_VAULT's fee-input requirement: the 292 vB trigger measurement includes a fee-wallet input that is inherent to BIP-345's value preservation rule, not an implementation artifact. But CTV's 3,300-sat anchor output and CAT+CSFS's unconstrained recovery leaf are implementation choices, not BIP requirements. Table 7.1 lists the design choices in the two author-built implementations (CAT+CSFS and Simplicity) that could affect measured properties.

7.1.3 Rational Adversary Assumption

The economic analysis assumes a rational adversary who mounts an attack only when attack_cost < vault_value. This excludes irrational adversaries (state actors, ideologically motivated attackers) who may attack regardless of cost. It also excludes sophisticated attackers who combine covenant-level attacks with network-level attacks (eclipse, time-dilation) to improve their success probability.

7.1.4 Threats Not Modeled

The TM1–TM11 threat taxonomy covers covenant-specific attack classes but excludes several categories that apply equally to all vault designs:

- **TM3 race dynamics are network-layer, not Script-layer.** Our design-space decomposition (Section 5.2.2) does not model the race between the attacker's unvault-plus-withdrawal sequence and the watchtower's recovery broadcast. Formalising $\Pr_D$[attacker wins race] would require a separate probability space over block-arrival times, mempool propagation delays, and detection latency—parameters that depend on the deployment environment (mainnet, regtest, signet) rather than on BIP script semantics. Existing watchtower and payment-channel formalisations in the state-channel literature treat confirmation as atomic or assume bounded-delay synchrony; none provides off-the-shelf machinery for a pinning-composed race against a Bitcoin mempool. We therefore treat TM3 as uniformly applicable to all four BIPs at the Script layer. CAT+CSFS's dual-binding on the hot leaf is the one design-level mitigation in our taxonomy: it reduces the worst-case outcome from theft to griefing (see Section 5.6). A formal network-layer race model is left for future work.

- **Mining-level attacks.** An attacker with significant hash power could withhold blocks to compress CSV timelocks, reducing the watchtower's response window. This interacts with TM3 and TM4 but is not covenant-specific.

- **Network-level attacks.** Eclipse attacks could prevent a watchtower from observing unvault transactions, giving the attacker a head start on the CSV race.

- **Watchtower availability.** All vault designs assume the watchtower is online during the CSV window. DDoS attacks targeting the watchtower infrastructure are a

deployment concern orthogonal to covenant design.

- **Multisig key management.** Production vaults would use multisig (e.g., 2-of-3 cold keys). The interaction between threshold signing schemes and covenant enforcement is not addressed.
- **Dust limit changes.** The per-UTXO recovery-cost scaling analysis assumes current dust limits (∼546 sats). Changes to dust relay policy would shift the economic floor at which splits become unrecoverable.

These threats are orthogonal to the covenant-specific properties measured in this work. They represent important deployment considerations but do not affect the relative ranking of covenant designs.

7.2 Mitigations

7.2.1 TRUC/v3 Transactions

TRUC (Topologically Restricted Until Confirmation), also known as v3 transactions, limits descendant chains to a single transaction. TRUC was merged into Bitcoin Core 28.0 [10] and is available on mainnet as an opt-in relay policy. If CTV vault transactions adopt TRUC, the 25-descendant fee pinning vector (TM1) is eliminated at the relay policy level. This would remove CTV's primary vulnerability, potentially shifting its security profile: without fee pinning, CTV's worst-case attack becomes address reuse (TM7), which is a user error preventable by wallet software rather than an adversarial exploit. The TM1 row of the master threat matrix (and hence the CTV branch of Proposition 5.9's counterexample enumeration) would need re-evaluation under TRUC, as CTV would no longer have a fee-invariant worst

case under TM1.

7.2.2 Batched Recovery

Class Z3 (TM4) is materially mitigated by batching recovery transactions. Instead of recovering each split UTXO individually, the watchtower combines multiple UTXOs into a single recovery transaction, amortising fixed overhead across inputs. We measure batched recovery on regtest for both CCV and OP_VAULT (Section 5.4.1); the fitted linear scaling is $54 + 68N$ vB for CCV and $154 + 92N$ vB for OP_VAULT. The dust-threshold fee rate $r^{\dagger}$ rises by 1.8× (CCV) and 2.7× (OP_VAULT), and the CCV/OP_VAULT ordering under TM4 reverses relative to the single-input case.

7.2.3 Poelstra Reset Model

CAT+CSFS's cold key vulnerability (TM11) can be mitigated by replacing the bare `OP_CHECKSIG` recovery leaf with a recursive staging reset, as proposed by Poelstra [34]. Under this design, cold key compromise initiates a liveness battle: the attacker resets the vault, the owner triggers a new withdrawal, the attacker resets again. Each round costs both parties transaction fees. This converts immediate theft into sustained denial of service, matching OP_VAULT's security profile under dual-key compromise. The tradeoff is a larger recovery script and additional witness overhead.

7.3 Open Problems

Temporal verification. The Alloy models verify structural reachability but not timing properties. A TLA+ or UPPAAL model could capture the recovery race dynamics: given a

CSV delay of d blocks, a block interval distribution, and network propagation delays, what is the probability that the watchtower's recovery confirms before the attacker's withdrawal? This would complement the structural verification with probabilistic timing guarantees.

Cryptographic verification. The Alloy models abstract signatures as key-possession checks. A Tamarin or ProVerif model could verify the cryptographic binding properties: that the CAT+CSFS dual verification is sound under the Schnorr signature scheme, that CTV template hashes are collision-resistant under SHA-256, and that Simplicity's jet-based verification is equivalent to the combinator semantics. O'Connor's Coq formalization of Simplicity [32] provides a foundation for this direction.

Cluster mempool impact. The ongoing Bitcoin Core cluster mempool redesign replaces per-transaction ancestor/descendant limits with cluster-based evaluation [48]. This could fundamentally change the fee pinning calculus: the 25-descendant limit exploited in TM1 may be replaced by cluster size limits with different eviction logic. Measuring the fee pinning attack under cluster mempool rules would determine whether CTV's primary vulnerability survives the policy change.

Replacement cycling. Riard [36] demonstrated transaction replacement cycling attacks against Lightning Network HTLCs. The same class of attack may apply to covenant vaults: an attacker could cycle the watchtower's recovery transaction out of the mempool by repeatedly replacing it with conflicting transactions. This interaction between covenant security and mempool replacement policy is unexplored.

Multi-vault composition. Real-world deployments would use multiple vaults simultaneously. The interaction between concurrent vault operations—fee wallet contention in

OP_VAULT, UTXO set management across vaults, and cross-vault batching opportunities—introduces composition effects not captured by single-vault analysis.

Chapter 8

Related Work

This chapter positions our work relative to prior research on Bitcoin vault custody, covenant design, formal verification of cryptocurrency protocols, and layer-two security.

8.1 Vault Custody Protocols

Swambo et al. [43] formalized the vault lifecycle model—deposit, unvault, withdraw, recover—and the watchtower monitoring assumption that underpins all covenant vault designs. Their analysis established the threat vocabulary (key compromise, liveness denial, fund safety) that we adopt and extend with the 11-threat taxonomy (TM1–TM11). A companion paper [42] characterized three approaches to implementing covenants (pre-signed transactions with deleted keys, OP_CHECKOUTPUTSHASHVERIFY, and OP_CHECKTEMPLATEVERIFY), analyzing the security assumptions of each. Their work analyzed deleted-key covenants and pre-signed transaction vaults without comparing opcode-level designs empirically. Swambo's doctoral thesis [41] extended this to the Ajolote autonomous custody system with formal security analysis, but did not include CTV, CCV, OP_VAULT, or CAT+CSFS.

Möser, Eyal, and Gün Sirer [25] introduced covenants as a mechanism for vault-based custody, proposing script-level restrictions on how UTXOs can be spent. Their work predates all four BIP proposals studied here and provides the conceptual foundation for the covenant

approach to custody. O'Connor and Piekarska [33] demonstrated that covenants can be implemented in Bitcoin by adding OP_CAT and OP_CHECKSIGFROMSTACKVERIFY, foreshadowing the CAT+CSFS design studied in this thesis.

O'Beirne's technical report on vaults and covenants [29] provides a practitioner-oriented comparison of vault designs, analyzing the tradeoffs between OP_VAULT's purpose-built approach and CTV's generic template commitment. This informed the design of BIP-345 [31], which was subsequently withdrawn [30] in favor of CCV (BIP-443).

Kirstein et al. [21] proposed Phoenix, a formally verified regenerating vault that recovers from key compromise by regenerating the vault with new keys. Their work uses formal verification (in Dafny) to prove safety properties of a single vault design. Our work differs by comparing five designs under a uniform methodology rather than deeply verifying one.

The Bitcoin PIPEs v2 proposal [1] introduces an alternative approach to covenants using zero-knowledge proofs, enabling programmable vaults without new opcodes. This represents a fundamentally different design philosophy from the opcode-level covenants studied here.

8.2 Covenant Proposals and Analysis

Each covenant proposal has its own design documentation and community analysis, but no prior work compares them empirically.

Rubin [38] specified CTV and discussed its applications (vaults, congestion control, payment pools) but did not provide measured transaction sizes or cross-design comparisons. The bitcoin-dev mailing list discussion identified address reuse as a risk, which we confirm empirically.

Ingala [19] designed CCV (originally MATT) and documented the keyless recovery griefing

concern and the OP_SUCCESS behavior for undefined modes. Our work quantifies the griefing cost asymmetry ($\gamma = 0.27$) and confirms the OP_SUCCESS behavior across five undefined mode values on a production-shaped taptree.

O'Beirne and Sanders [31] specified OP_VAULT with authorized recovery and automatic revault. Harding [18] estimated per-UTXO recovery-cost scaling at approximately 3,000 splits per block. We confirm this for OP_VAULT (3,427 measured) and extend it to CCV (6,172 splits per block, 80% more than Harding's estimate). O'Beirne subsequently withdrew BIP-345 [30]; our lifecycle cost comparison (OP_VAULT 36% more expensive than CCV) provides quantitative context for that decision.

Poelstra [34] demonstrated transaction introspection via OP_CAT and Schnorr signature tricks, constructing a vault with recursive covenants and dynamic destination selection. Our CAT+CSFS vault implements a simplified variant (fixed destination, no recursion) to isolate the dual-verification security properties. The cold key recovery vulnerability (TM11) and the Poelstra reset cost model in Chapter 5 directly build on this work. Rijndael [37] built a working vault prototype using only OP_CAT and the Schnorr G-trick (without CSFS), demonstrating that OP_CAT alone enables covenant enforcement.

8.3 Formal Verification of Bitcoin Contracts

Bartoletti and Zunino [7] introduced BitML, a domain-specific calculus for Bitcoin smart contracts with formal semantics based on process algebra. BitML enables reasoning about contract security through symbolic and computational models. Subsequent work developed a toolchain for compiling BitML to Bitcoin transactions [4] and verified liquidity properties of recursive contracts [9]. Bartoletti, Lande, and Zunino [8] extended this line to Bitcoin covenants, modeling vault contracts in an extended BitML with covenant primitives and

proving safety properties. Our Alloy models operate at a different abstraction level—state-machine reachability rather than process-algebraic semantics—but address the same class of questions: can an attacker extract funds from a correctly constructed vault?

dgpv [15] applied Alloy bounded model checking to Rijndael's OP_CAT vault prototype, verifying fund conservation and covenant enforcement properties within a single design. Our work extends this methodology to all four BIP-specified designs plus Simplicity, using a shared base model (`vault_base.als`) to ensure uniform property definitions. We add the threat model layer (`threat_model.als`) for systematic attacker profiling and the cross-covenant module (`cross_covenant.als`) for multi-design interaction checking—neither of which appears in dgpv's analysis.

O'Connor [32] provided Coq-verified denotational semantics for the Simplicity language, establishing the formal foundation for reasoning about Simplicity programs. Our Alloy model of the Simplicity vault operates at the state-machine level (vault lifecycle transitions) rather than the language semantics level, complementing O'Connor's foundational work with application-level verification. O'Connor and Poelstra have also pursued formal verification of cryptographic primitives used in Bitcoin's libsecp256k1 library using Coq, demonstrating the viability of machine-checked proofs for Bitcoin infrastructure.

Several surveys have cataloged formal verification methods for blockchain smart contracts. Murray and Anisi [24] categorized approaches including model checking, theorem proving, and symbolic execution. Tolmach et al. [44] provided a comprehensive survey of formal specification and verification techniques, covering 50+ tools and frameworks. Almakhour et al. [2] focused on verification methods applicable to production smart contracts, and Singh et al. [40] analyzed how formalization approaches address known vulnerability classes. Our use of Alloy bounded model checking falls within the model checking category identified by these surveys, applied specifically to Bitcoin covenant protocols rather than Ethereum smart

contracts.

8.4 Layer-Two Security and Watchtower Economics

The watchtower monitoring model used in vault security is closely related to the watchtower concept in Lightning Network payment channels. Leinweber et al. [22] proposed TEE-based distributed watchtowers for fraud protection in Lightning, analyzing the economic viability of third-party watchtower services. Our per-UTXO recovery-cost scaling analysis (TM4) addresses a different question—how an attacker can exhaust the watchtower's recovery budget through vault splitting—but shares the assumption of an always-online monitoring service.

Naumenko and Riard [27] analyzed time-dilation attacks on the Lightning Network, where an attacker with network-level capabilities manipulates the victim's view of the blockchain to compress timelocks. This attack class applies to covenant vaults (an attacker could compress the CSV window), but we do not model it because it is orthogonal to covenant-specific properties and requires network-layer simulation beyond our regtest methodology. Riard [36] subsequently demonstrated replacement cycling attacks against Lightning HTLCs, a mempool manipulation technique that may also apply to covenant vault recovery transactions.

Aumayr [5] provided a comprehensive formal treatment of Bitcoin-compatible scalability protocols, including watchtower collateralization and channel closing security. The economic analysis of watchtower reserves in that work parallels our per-UTXO recovery-cost scaling budget calculations, though applied to payment channels rather than vaults.

Table 8.1: Comparison with closely related prior work.

Work	Covenants	Empirical?	Formal?	Fee analysis?	Year
Möser et al. [25]	1 (generic)	No	No	No	2016
Swambo et al. [43]	0[a]	No	Partial	No	2020
Bartoletti et al. [8]	1 (BitML)	No	Yes (process)	No	2020
Kirstein et al. [21]	1 (Phoenix)	No	Yes (Dafny)	No	2021
Harding [18]	1 (OPV)	No[b]	No	Partial	2024
dgpv [15]	1 (CAT)	No	Yes (Alloy)	No	2024
covenants.info [14]	6+	No	No	No	2024
This work	**5**	**Yes**	**Yes (Alloy)**	**Yes**	**2026**

[a]Analyzes vault model generically, not specific BIP opcodes.

[b]Estimates from assumed transaction sizes, not regtest measurements.

8.5 Qualitative Comparisons

The covenants.info website [14] provides a qualitative feature matrix comparing covenant proposals across dimensions such as recursion support, introspection granularity, and application breadth. This resource contains no transaction sizes, no attack cost functions, and no fee-environment analysis. Our work complements it with quantitative measurements.

The B-SSL covenant-free vault model [6] demonstrates vault-like custody using only CSV and CLTV timelocks (no new opcodes). This provides a baseline for what is achievable without covenants—the security properties are weaker (no output constraints on recovery, no dynamic destination selection) but the deployment path requires no consensus changes.

8.6 Summary of Positioning

Table 8.1 compares this work with the most closely related prior efforts across five dimensions.

No prior work combines empirical measurement, formal verification, and fee-environment-dependent security analysis for multiple covenant vault designs. The individual

components—vault lifecycle modeling [43], Bitcoin contract formalization [7, 8], Alloy verification [15], watchtower exhaustion estimation [18]—exist in isolation. Our contribution is the integration of these approaches under a uniform methodology that enables cross-design comparison and the derivation of results (Proposition 5.2 and 5.9) that require data from multiple designs simultaneously.

Chapter 9

Conclusion

This thesis presented the first empirical comparison of four Bitcoin covenant vault designs—CTV (BIP-119), CCV (BIP-443), OP_VAULT (BIP-345), and CAT+CSFS (BIP-347 + BIP-348)—plus a fifth vault built with the Simplicity language on Elements. Through 15 experiments on regtest, Alloy formal verification, and parameterized economic modeling, we established two results that constrain the design space of covenant vaults.

Result 1: Griefing–safety impossibility. No covenant vault simultaneously achieves permissionless recovery and griefing resistance (Proposition 5.2). This is a structural design constraint, not an implementation limitation. Vault designers must choose between maximizing fund safety under key loss (CCV's keyless recovery) and minimizing denial-of-service exposure (OP_VAULT's authorized recovery).

Result 2: Structural design-space decomposition. Covenant vault designs are characterised by four nearly-orthogonal dimensions: fee model (f), amount flexibility (a), recovery gating (g), and recovery binding (b). Each dimension carries a deterministic feature-to-vulnerability implication (Lemmas 5.5–5.8): the anchor-output fee model opens TM1 (fee pinning); partial-withdrawal flexibility opens TM4 (per-UTXO recovery-cost scaling); keyless recovery opens TM2 (permissionless griefing, by Proposition 5.2); and unbound recovery opens TM11 (cold-key theft for the full vault value). Proposition 5.9 estab-

lishes that no proposed Bitcoin Script extension occupies the dominant design-space corner $(f_{\neq\text{anchor}}, a_{\text{atomic}}, g_{\text{key}}, b_{\text{bound}})$ at which $\mathcal{L} = 0$ simultaneously across {TM1, TM2, TM4, TM11}. Every proposal differs from the corner in at least one dimension, and by the corresponding lemma that difference forces a positive defender loss on the threat that the dimension controls. The decomposition also provides a framework for localising future BIPs in the lattice: their vulnerability profile follows from their feature tuple without further measurement.

Empirical corrections. We corrected prior hand-estimates of OP_VAULT transaction sizes by 45–46% (trigger: 200 → 292 vB, recovery: 170 → 246 vB), revealing a 36% lifecycle cost premium over CCV. We measured CCV's watchtower splitting capacity at approximately 6,172 splits per block—80% more than Harding's estimate of ∼3,000, which assumed OP_VAULT-sized transactions. These are the first regtest-validated vsize measurements for all four Bitcoin covenant vault lifecycles.

Batched recovery and ordering flip. BIP-345's "Batching" section (authorised recovery, lines 613–621) and BIP-443's default aggregation mode permit the defender to recover N Unvaulting UTXOs in a single transaction. Measured on regtest, the amortised per-input cost is 68 vB for CCV and 92 vB for OP_VAULT. The dust-threshold fee rate rises by 1.8× and 2.7× respectively, and the relative vulnerability of the two designs under TM4 reverses: CCV is safer single-input, OP_VAULT is safer batched (Section 5.4.1).

Original implementations. We built a CAT+CSFS vault using explicit `OP_CHECKSIGFROMSTACK` (BIP-348) dual verification—distinct from prior OP_CAT-only vaults that use the Schnorr G-trick without CSFS [34, 37]—and a Simplicity vault on Elements using jet-based covenant enforcement. Both are open-source. The CAT+CSFS

implementation revealed the cold key recovery vulnerability (TM11): bare `OP_CHECKSIG` recovery grants immediate, unrestricted fund theft upon cold key compromise. The Simplicity vault serves as a cross-substrate reference point, showing output-constrained recovery on all paths under the federated sidechain trust model.

Formal verification. Alloy bounded model checking verified fund conservation, recovery liveness, and no-extraction properties across the four Bitcoin Script extension designs plus the Simplicity reference. The one expected assertion failure—cold key extraction in CAT+CSFS—structurally confirms TM11. The three-layer evidence methodology (Alloy for structural possibility, regtest for concrete cost, fee model for economic rationality) provides complementary assurance that no single analysis technique could achieve alone.

9.1 Future Work

Several directions extend this research:

- **TRUC/v3 impact.** If adopted, TRUC transactions would eliminate CTV's descendant-chain fee pinning, removing its primary vulnerability and requiring re-evaluation of the TM1 branch of Proposition 5.9's counterexample enumeration.
- **Cluster mempool.** The Bitcoin Core cluster mempool redesign may change descendant limit semantics, affecting the fee pinning attack construction.
- **Temporal verification.** TLA+ or UPPAAL models could verify recovery race timing properties that Alloy's state-machine abstraction cannot capture.
- **CTV+CSFS activation.** As CTV and CSFS move toward activation signaling, the measurements and cost models in this work can inform deployment decisions. The fee-

conditional guidance (Section 5.10) is directly applicable to vault designers choosing between CTV-only and CAT+CSFS vaults.

- **Multi-vault composition.** Production deployments managing multiple concurrent vaults introduce fee wallet contention, cross-vault batching opportunities, and UTXO management complexity not captured by single-vault analysis.

The framework, all measurements, formal models, and both original vault implementations are available at https://github.com/PraneethGunas/covenant-vault-research.

Bibliography

[1] Manuel Abdalla, Brandon Carmer, Mohsen El Gebali, et al. Bitcoin PIPEs v2. IACR ePrint 2026/186, 2026. URL https://eprint.iacr.org/2026/186.

[2] Mouhamad Almakhour, Layth Sliman, Abed Ellatif Samhat, and Abdelhamid Mellouk. Verification of smart contracts: A survey. *Pervasive and Mobile Computing*, 67:101227, 2020. doi: 10.1016/j.pmcj.2020.101227.

[3] Andreas M. Antonopoulos. *Mastering Bitcoin: Programming the Open Blockchain*. O'Reilly Media, 2nd edition, 2017.

[4] Nicola Atzei, Massimo Bartoletti, Stefano Lande, Nobuko Yoshida, and Roberto Zunino. Developing secure Bitcoin contracts with BitML. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1124–1128, 2019. doi: 10.1145/3338906.3341173.

[5] Lukas Aumayr. *Foundations of Bitcoin-Compatible Scalability Protocols*. PhD thesis, TU Wien, 2024.

[6] B-SSL. Covenant-free vault model. Delving Bitcoin, 2025.

[7] Massimo Bartoletti and Roberto Zunino. BitML: A calculus for Bitcoin smart contracts. In *ACM Conference on Computer and Communications Security (CCS)*, pages 83–100, 2018. doi: 10.1145/3243734.3243795.

[8] Massimo Bartoletti, Stefano Lande, and Roberto Zunino. Bitcoin covenants unchained. In *International Conference on Information Security (ISC)*, pages 25–42, 2020. doi: 10.1007/978-3-030-61467-6_3.

[9] Massimo Bartoletti, Stefano Lande, and Maurizio Murgia. Verifying liquidity of recursive Bitcoin contracts. *Logical Methods in Computer Science*, 18(1), 2022.

[10] Bitcoin Core Developers. Bitcoin core relay policy documentation. Bitcoin Core, 2024. URL https://github.com/bitcoin/bitcoin/blob/master/doc/policy/. Defines dust threshold, descendant limits, and RBF rules.

[11] Bitcoin Developer Community. CTV+CSFS: Can we reach consensus on a first step towards covenants? Delving Bitcoin, 2025.

[12] BtcDrak, Mark Friedenbach, and Eric Lombrozo. BIP-112: CHECKSEQUENCEVERIFY. Bitcoin Improvement Proposal, 2015. URL https://bips.dev/112/.

[13] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Michael Peyton Jones, and Philip Wadler. The extended UTXO model. In *Financial Cryptography and Data Security (FC 2020 Workshops)*, pages 525–539, 2020. doi: 10.1007/978-3-030-54455-3_37.

[14] Covenants.info Contributors. covenants.info — community covenant comparison. https://covenants.info, 2024.

[15] dgpv. Analyzing simple vault covenant with Alloy. Delving Bitcoin, 2024. URL https://delvingbitcoin.org/t/analyzing-simple-vault-covenant-with-alloy/819.

[16] Johnny Dilley, Andrew Poelstra, Jonathan Wilkins, Marta Piekarska, Ben Gorlick, and Mark Friedenbach. Strong federations: An interoperable blockchain protocol, 2016.

[17] Mark Friedenbach, BtcDrak, Nicolas Dorier, and kinoshitajona. BIP-68: Relative lock-time using consensus-enforced sequence numbers. Bitcoin Improvement Proposal, 2015. URL https://bips.dev/68/.

[18] David A. Harding. OP_VAULT comments. Delving Bitcoin, 2024. URL https://delvingbitcoin.org/t/op-vault-comments/521.

[19] Salvatore Ingala. BIP-443: OP_CHECKCONTRACTVERIFY. Bitcoin Improvement Proposal, 2024. URL https://bips.dev/443/.

[20] Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, revised edition, 2012.

[21] Uri Kirstein, Shelly Grossman, Michael Mirkin, James Wilcox, and Mooly Sagiv. Phoenix: A formally verified regenerating vault. In *arXiv preprint*, 2021.

[22] Michael Leinweber, Matthias Grundmann, Ludwig Schönborn, and Hannes Hartenstein. TEE-based distributed watchtowers for fraud protection in the Lightning Network. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology (DPM/CBT)*, pages 177–194, 2019. doi: 10.1007/978-3-030-31500-9_11.

[23] Eric Lombrozo, Johnson Lau, and Pieter Wuille. BIP-141: Segregated witness (consensus layer). Bitcoin Improvement Proposal, 2015. URL https://bips.dev/141/.

[24] Yvonne Murray and David A. Anisi. Survey of formal verification methods for smart contracts on blockchain. IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C), 2019.

[25] Malte Möser, Ittay Eyal, and Emin Gün Sirer. Bitcoin covenants. In *Financial Cryptography and Data Security (FC 2016 Workshops)*, pages 126–141, 2016. doi: 10.1007/978-3-662-53357-4_9.

[26] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL https://bitcoin.org/bitcoin.pdf.

[27] Gleb Naumenko and Antoine Riard. Time-dilation attacks on the Lightning Network. *Cryptoeconomic Systems*, 2021. URL https://cryptoeconomicsystems.pubpub.org/pub/riard-lightning-dilation.

[28] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*, volume 12827 of *Lecture Notes in Computer Science*, pages 189–221. Springer International Publishing, 2021. doi: 10.1007/978-3-030-84252-9_7. Contains the BIP-340 Schnorr signature EUF-CMA security reduction under the fixed-public-key model used by the CAT+CSFS dual-verification argument in Ch. 2 and Ch. 5.

[29] James O'Beirne. Vaults and covenants. Technical report, 2023. URL https://jameso.be/vaults.pdf.

[30] James O'Beirne. Withdrawing OP_VAULT (BIP-345). Delving Bitcoin, 2025.

[31] James O'Beirne and Greg Sanders. BIP-345: OP_VAULT. Bitcoin Improvement Proposal (withdrawn), 2023. URL https://bips.dev/345/. Withdrawn in favor of BIP-443.

[32] Russell O'Connor. Simplicity: A new language for blockchains. In *Proceedings of the ACM SIGPLAN Workshop on Programming Languages and Analysis for Security (PLAS)*, 2017. doi: 10.1145/3139337.3139340.

[33] Russell O'Connor and Marta Piekarska. Enhancing Bitcoin transactions with covenants. In *Financial Cryptography and Data Security (FC 2017 Workshops)*, pages 191–198, 2017. doi: 10.1007/978-3-319-70278-0_12.

[34] Andrew Poelstra. CAT and Schnorr tricks I & II. Blockstream Research Blog, 2021. URL https://medium.com/blockstream/cat-and-schnorr-tricks-i-faf1b59bd298.

[35] Antoine Poinsot. Response to CTV+CSFS activation proposal. bitcoin-dev mailing list, 2025. Called for comprehensive exploration and proof before activation.

[36] Antoine Riard. Replacement cycling attacks on Lightning Network. bitcoin-dev mailing list, 2023.

[37] Rijndael. Basic vault prototype using OP_CAT. Delving Bitcoin, 2024. URL https://delvingbitcoin.org/t/basic-vault-prototype-using-op-cat/576.

[38] Jeremy Rubin. BIP-119: OP_CHECKTEMPLATEVERIFY. Bitcoin Improvement Proposal, 2020. URL https://bips.dev/119/.

[39] Tim Ruffing and Andrew Poelstra. BIP-348: OP_CHECKSIGFROMSTACK. Bitcoin Improvement Proposal, 2024. URL https://bips.dev/348/.

[40] Amritraj Singh, Reza M. Parizi, Qi Zhang, Kim-Kwang Raymond Choo, and Ali Dehghantanha. Blockchain smart contracts formalization: Approaches and challenges to address vulnerabilities. *Computers & Security*, 88:101654, 2020. doi: 10.1016/j.cose.2019.101654.

[41] Jacob Swambo. *Evolving Bitcoin Custody*. PhD thesis, University of Wyoming, 2022.

[42] Jacob Swambo, Spencer Hommel, Bob McElrath, and Bryan Bishop. Bitcoin covenants: Three ways to control the future, 2020.

[43] Jacob Swambo, Spencer Hommel, Bob McElrath, and Bryan Bishop. Custody protocols using bitcoin vaults. *arXiv preprint*, 2020.

[44] Palina Tolmach, Yi Li, Shang-Wei Lin, Yang Liu, and Zengxiang Li. A survey of smart contract formal specification and verification. *ACM Computing Surveys*, 54(7):1–38, 2021. doi: 10.1145/3464421.

[45] Anthony Towns. A summary of covenant proposals for Bitcoin. Delving Bitcoin, 2025.

[46] Pieter Wuille. BIP-347: OP_CAT for tapscript. Bitcoin Improvement Proposal, 2023. URL https://bips.dev/347/.

[47] Pieter Wuille and Suhas Daftuar. Cluster mempool: A proposed redesign of bitcoin core's mempool. Delving Bitcoin, 2024. URL https://delvingbitcoin.org/t/an-overview-of-the-cluster-mempool-proposal/393. Qualitatively changes the replacement cost calculus for fee-channel pinning (class Z1); scope-limit in this work.

[48] Pieter Wuille and Suhas Daftuar. Cluster mempool. Bitcoin Core, 2024. Replaces per-transaction ancestor/descendant limits.

[49] Pieter Wuille, Jonas Nick, and Tim Ruffing. BIP-340: Schnorr signatures for secp256k1. Bitcoin Improvement Proposal, 2020. URL https://bips.dev/340/.

[50] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP-341: Taproot: SegWit version 1 spending rules. Bitcoin Improvement Proposal, 2020. URL https://bips.dev/341/.

[51] Pieter Wuille, Jonas Nick, and Anthony Towns. BIP-342: Validation of taproot scripts. Bitcoin Improvement Proposal, 2020. URL https://bips.dev/342/. Defines Tapscript sighash computation used in CAT+CSFS introspection.

Appendices

Appendix A

Alloy Model Details

This appendix provides the model architecture, per-module descriptions, and detailed per-covenant verification results summarized in Chapter 6.

A.1 Model Architecture

The Alloy models follow a three-layer inheritance hierarchy (Figure A.1).

`btc_base.als`. Defines the Bitcoin transaction model: signatures for `UTXO`, `Key`, `Transaction`, and `Address`. UTXOs have an amount, owner key, and spent status. Transactions consume inputs and produce outputs. This module provides the vocabulary shared by all vault models.

`vault_base.als`. Defines the abstract vault state machine: four states (`VAULTED`, `UNVAULTING`, `WITHDRAWN`, `RECOVERED`), transitions between states (trigger, withdraw, recover), and the `VaultFamily` concept that groups UTXOs belonging to the same vault instance. The CSV delay is modeled as an integer field on each vault UTXO. Properties defined at this level (fund conservation, recovery liveness) apply uniformly to all concrete models.

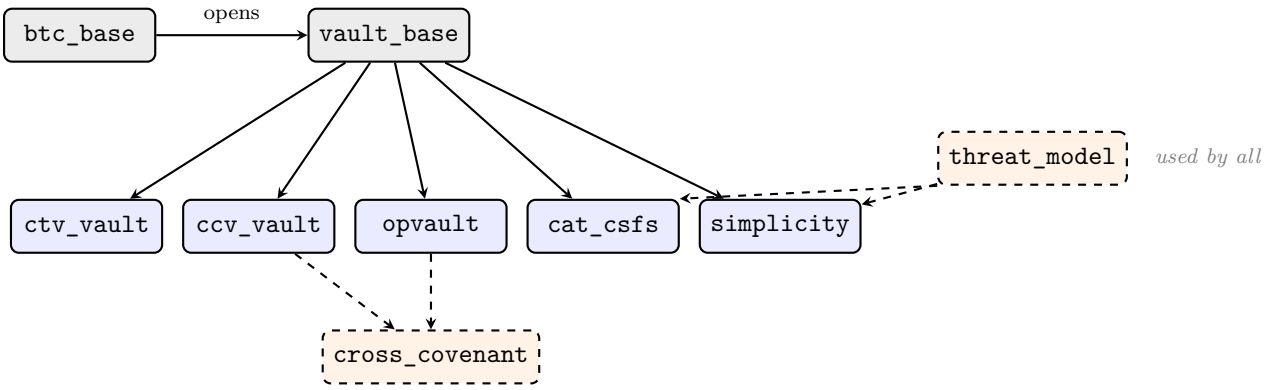

Figure A.1: Alloy model hierarchy. `btc_base` defines UTXOs, keys, and transactions. `vault_base` defines the abstract vault state machine. Five concrete models extend the base with covenant-specific transition guards. `threat_model` and `cross_covenant` are cross-cutting modules.

Concrete vault models. Each of the five models extends `vault_base` with covenant-specific transition guards. `ccv_vault` models the CCV mode enumeration and the OP_SUCCESS behavior for undefined modes. `opvault_vault` models the three-key separation and authorized recovery. `cat_csfs_vault` models the dual-verification binding and unconstrained recovery leaf.

`threat_model.als`. Defines attacker profiles by specifying which keys the attacker controls. Properties are checked under each profile to determine the minimum capability needed for each attack class.

`cross_covenant.als`. Models interactions between vault instances from different covenant types sharing the same UTXO set. Checks whether cross-vault injection is possible and whether fee wallet contention between concurrent OP_VAULT operations can block recovery.

A.2 Per-Covenant Detailed Results

CTV. All six checks pass: no unauthorized extraction (with no keys and with hot key only), CSV enforcement, no state proliferation, eventual withdrawal, and recovery liveness. The address reuse scenario finds an instance where a second deposit creates an unspendable UTXO, confirming the structural vulnerability (TM7).

CCV. All seven checks pass, including the mode confusion containment check (undefined modes trigger OP_SUCCESS but only within the affected leaf—the vault's other leaves remain secure). The griefing scenario demonstrates that any party can invoke recovery without keys (TM2). The splitting scenario shows the revault chain that enables the per-UTXO recovery-cost scaling of class Z3 (TM4).

OP_VAULT. All eight checks pass, including the dual-key-no-theft property (trigger + recoveryauth compromise produces liveness denial but not fund theft, because the recovery address is pre-committed). The key-loss scenario demonstrates permanent inability to recover if the recoveryauth key is destroyed.

CAT+CSFS. Five of six checks pass. The `catcsfsNoExtraction_ColdKeyOnly` check *fails*: the Alloy Analyzer finds a trace where the cold key holder invokes recovery with an attacker-controlled output address, extracting funds. This is the structural confirmation of TM11 and the only expected assertion failure across all models. The destination lock and CSV enforcement checks pass.

Simplicity. All six checks pass: no extraction under any single-key compromise, output binding enforcement, CSV enforcement, and no state proliferation. The recovery-constrained

scenario confirms that cold key compromise can only recover to the pre-committed address (unlike CAT+CSFS).

A.3 Limitations of the Formal Models

The Alloy models operate at the state-machine level, abstracting away cryptographic primitives, serialization details, and temporal dynamics. Signatures are modeled as key-possession checks (if the attacker holds the key, the signature succeeds). Hash functions are assumed collision-resistant. Network propagation, mining centralization, and mempool dynamics are not modeled. These abstractions mean the Alloy verification confirms *structural* properties (state reachability, authorization requirements) but not *cryptographic* properties (signature unforgeability, hash binding) or *temporal* properties (race conditions, confirmation timing). The regtest experiments partially compensate for the temporal gap by demonstrating actual transaction construction and broadcast, while the cryptographic properties are inherited from the underlying BIP-340 Schnorr and SHA-256 specifications.

Appendix B

Simplicity on Elements: The Unoccupied Optimal Corner

B.1 Scope and Motivation

Simplicity [32] is an *alternative script primitive* — a typed combinator calculus with Coq-verified denotational semantics — deployed on Elements [16], a Bitcoin-adjacent sidechain whose blocks are produced by known functionaries rather than by proof-of-work miners. Execution uses native-code *jets* for known combinator subtrees. Because Elements' consensus model differs from Bitcoin's, we exclude Simplicity from the formal results of Chapter 5 and report it here as a cross-substrate reference: Simplicity demonstrates that the dominant corner $D^\star$ of Chapter 5 is occupiable on some substrate, even while no Bitcoin covenant proposal currently occupies it.

This distinction is load-bearing for the thesis's claim. Chapter 5's contribution is *unoccupied on Bitcoin*, not *theoretically unoccupiable*. Simplicity is the constructive existence proof for the latter. A reader who conflates the two would mistake a specification gap for an impossibility theorem.

B.2 Design-Space Position

Our Simplicity vault occupies the strict-best axis values on every axis where such a value exists, in the vocabulary of Section 2.10:

- A1 = `cold-key`: key-gated recovery via a Schnorr signature check.
- A2 = `no`: atomic withdrawal; the program enforces the complete output tuple via `jet::outputs_hash() = param::VAULT_RECOVER_OUTPUTS_HASH` (see `simf/vault.simf`).
- A3 = `dynamic` (Elements fee structure): fees live inside the transaction; no out-of-band anchor.
- A4 = `dedicated-jets-combinator`: the withdrawal destination is bound through a typed output-hash commitment at compile time, stronger than any of the Bitcoin proposals on the same axis.
- A5 = `dedicated-jets-combinator`: same binding for the recovery leaf, eliminating class Z5.
- A6 = `single-mode`: a program's meaning is determined by its Merkle root; no `OP_SUCCESSx`-style fallthrough surface.
- A7 = `typed-combinator-introspection`: the fundamental operators are total, typed, and Coq-verified.

Consequently, Simplicity's vault program is immune by construction to classes Z1, Z2, Z3, Z4, Z5, and Z6 — every attack class identified in Chapter 5. It occupies the strict-best position on A1, A2, A4, A5, and A7 simultaneously; no Bitcoin proposal does.

B.3 Lifecycle Measurements

The Simplicity vault lifecycle totals 865 vB (deposit 269, trigger 298, withdraw 298, recover 286), 49–135% larger than the Bitcoin designs of Chapter 4. The overhead bundles Elements' serialisation (∼30–40 bytes per output for confidential-transaction fields even in non-confidential mode) and Simplicity's bit-oriented DAG witness format. Elements functionaries do not operate a public mempool, so class Z1 (fee-channel pinning) has no substrate analogue under honest-federation assumptions.

Our Simplicity vault is atomic: recovery binds the complete output tuple via the jets commitment cited above, admitting neither partial withdrawal nor batched recovery and placing the program at A2 = `no` in the design-space lattice. Class Z3 (per-UTXO recovery-cost scaling) has no surface. Cold-key compromise (TM11) forces recovery but cannot redirect funds, matching CTV and OP_VAULT. A different Simplicity program could express A2 = `yes` by replacing the whole-tuple commitment with per-output checks; that design would inherit class Z3 under the D2 implication (Section 2.10) but is not what we measured.

B.4 Foundations and Substrate

Simplicity's Coq-verified denotational semantics give stronger guarantees than Alloy bounded model checking of the Bitcoin Script extensions can produce — a Simplicity program's meaning is determined by its Merkle root, under a formal semantics that has been mechanically verified. However, Simplicity on Liquid is live today under *federation trust*, whereas the four Bitcoin Script extensions remain under *Bitcoin consensus review*.

The substrate choice is a security-relevant decision alongside the per-covenant analysis of Chapter 5: a vault on Simplicity inherits the federation's liveness and censorship proper-

ties; a vault on Bitcoin inherits proof-of-work's. The dominant corner being occupiable on Simplicity is therefore not a turn-key prescription to migrate vaults to Liquid; it is evidence that the unoccupied status among current Bitcoin proposals reflects a consensus-substrate constraint rather than a design impossibility.

B.5 Future Work: Porting the Optimal Corner to Bitcoin

A Simplicity-style vault on Bitcoin would require Simplicity's taproot leaf type, which is specified in a draft BIP but not under active Bitcoin consensus review. Closing the Simplicity-to-Bitcoin gap depends on that soft-fork path. An alternative that preserves Bitcoin's consensus substrate is a CAT+CSFS-based bound-recovery construction (Poelstra's sketch [34]) that moves CAT+CSFS to A5 = `signature-dual-bind`; this would eliminate class Z5 while retaining the hot-key-theft immunity of class Z4, placing CAT+CSFS at the dominant corner of the Bitcoin sublattice. Neither path is available as of thesis submission; we leave both as open problems.

Appendix C

Rationality Block Template and Full Catalogue

C.1 Purpose

Chapter 5 derives six attack classes, each with a *Rationality and Scope* block structured per the seven-field schema of Section 3.3. The FC 2027 paper compresses these blocks into a single matrix (`conference/FC-2027/paper/appendix-b-threat-matrix.tex`, §B.2); for the thesis, the block form preserves the full reasoning chain for each class. A reviewer or deployment engineer can audit any field against their own environment; a claim that is valid under the thesis's conditions but not under some downstream reader's condition should be flagged by a mismatch in fields (3)–(5) rather than by reading between the lines.

This appendix restates the six blocks in free-text form. The thesis chapters reference these blocks by subsection label (e.g., §C.2 for class Z1) rather than by embedding the prose inline.

C.2 Class Z1: Fee-Channel Pinning

(1) Attacker capability. The attacker holds the hot key of the target vault and capital sufficient for the anchor-descendant chain (approximately 14,300 sats at the reference dust-

relay rate of 3 sat/vB). Mempool access is required — specifically, the ability to broadcast transactions to a Bitcoin Core node that propagates under default policy. The attacker is assumed to know the vault's design and axis values (Kerckhoffs, Section 2.4.1).

(2) Defender model. An online watchtower that monitors the mempool and blockchain and attempts CPFP replacement on the unvaulting UTXO's anchor output when an unauthorised trigger is observed. The watchtower operates a default Bitcoin Core mempool with the 25-descendant relay limit active. The attacker's descendant chain saturates this limit before the watchtower's CPFP transaction can be accepted; BIP-125 Rule 3 then requires the replacement to pay for evicting every descendant, which is economically infeasible against a dust-rate chain.

(3) Economic preconditions. $\mathcal{L}(\mathrm{TM1})$ and the adversary advantage Adv are both r-invariant: the attack cost $C_{\mathrm{pin}} = 25(\rho + 1) \cdot \mathrm{vsize}_{\mathrm{desc}} + \alpha_{\mathrm{anchor}}$ is bounded in absolute terms (not as a function of vault value or fee rate). For any reference vault value $V \gg C_{\mathrm{pin}}$, the attack is rational at every fee regime. We measured $C_{\mathrm{pin}} \approx 14{,}300$ sats against a 0.5 BTC reference vault; that is 0.03% of the vault.

(4) Protocol / network assumptions. Bitcoin Core default mempool policy: 25-descendant limit from BIP-125; 3 sat/vB dustRelay threshold (policy, not consensus); RBF Rule 3 requiring the replacement to pay both a higher fee rate and a higher absolute fee than the cumulative descendants it evicts. The anchor output sized at 3,300 sats (CTV reference design) is spendable by any party, which lets the attacker pay its descendant chain without any key material beyond the hot key used for the trigger.

(5) Scope of validity. Applies iff the covenant's axis A3 has value `out-of-band-anchor` — i.e., fees attach via a CPFP anchor output rather than being deducted from vault value or supplied via a separate fee-wallet input. Does not apply to covenants with dynamic fee inclusion (CCV, OP_VAULT, CAT+CSFS), whose trigger transactions do not carry a pinnable anchor handle: the attacker cannot pin a transaction whose very existence they are trying to delay.

(6) Rational-attacker check. Rational at any r given hot-key compromise: payoff $\approx V$ (full vault theft once the watchtower's recovery is blocked and the hot key completes withdrawal after CSV expiry), cost $\approx C_{\text{pin}}$, residual value risk to the attacker limited to the descendant-chain sats.

(7) Counter-factual mitigations. Three distinct counter-factuals eliminate or shrink the class:

1. **TRUC/BIP-431 (recommended).** The TRUC (`version=3`) transaction type collapses the descendant limit to 1. A `OP_CTV` unvault that uses TRUC eliminates the pinning surface at the relay-policy level; this is available in Bitcoin Core v28.0+ and requires only that the unvault template commit to the TRUC version.

2. **Cluster mempool redesign.** The cluster-mempool proposal (Wuille and Daftuar [47]) changes replacement-cost accounting in ways that qualitatively alter the pinning-cost calculus. Scope-limit rather than a within-scope mitigation; treated as future work.

3. **P2A-style smaller anchor.** Reducing the anchor output from 3,300 sats (BIP-118-style bare anchor) to the 330 sat P2A companion specified in BIP-431 reduces the absolute attack cost but does not eliminate the structural surface.

A vault-design recommendation derived from this block is: *always deploy CTV unvault transactions as TRUC wherever Core v28.0+ is present on the network path.*

C.3 Class Z2: Permissionless Griefing

(1) Attacker capability. Zero keys for CCV (the class's only susceptible covenant). The attacker needs only mempool observation (to detect unauthorised triggers as they propagate) and the public recovery address. Kerckhoffs: we assume the recovery address is known, either because it was published for auditability, reused across vaults under the same cold key, or revealed by any prior legitimate recovery.

(2) Defender model. Any. The attack is pure liveness denial: the attacker does not extract funds, only forces the owner to re-deposit and re-trigger repeatedly. The owner's choice is to tolerate the cycle, abandon the vault, or rotate to a different covenant design. A watchtower does not help — the attack triggers the watchtower's own recovery path, which is what the attacker wants.

(3) Economic preconditions. $\mathcal{L}$ scales linearly in fee rate r (each round costs $\text{vsize}_{\text{def}} \cdot r$ plus the owner's re-trigger cost). The asymmetry $\gamma = \text{vsize}_{\text{atk}}/\text{vsize}_{\text{def}}$ is r-invariant.

(4) Protocol / network assumptions. Mempool-observable trigger transactions (standard under all Bitcoin Core releases). No RBF dependence: the attacker's recovery transaction does not contend with the trigger for the same UTXO spend, only with subsequent withdraw attempts.

(5) Scope of validity. Applies iff A1 = `keyless`. Reduces to a key-compromise attack (TM6) under any key-gated recovery model; the key-gated variant has a much narrower attacker pool.

(6) Rational-attacker check. Irrational without external incentive: the attacker has no direct financial gain. Rational only when the external incentive (extortion payout, competitor disruption, state-actor operation) exceeds the round-by-round cost.

(7) Counter-factual mitigations. A1 = `recoveryauth-key` (as in OP_VAULT) eliminates the class by requiring a Schnorr signature on the recovery leaf; any key-gated recovery (cold key, hot key, or dedicated authorisation key) has the same effect. The design tradeoff is that key-gated recovery reintroduces key-loss risk — Proposition 1 of Chapter 5 formalises the incompatibility.

C.4 Class Z3: Per-UTXO Recovery-Cost Scaling

(1) Attacker capability. The trigger (hot) key of the target vault and capital for N partial-withdrawal trigger transactions. Kerckhoffs: the attacker knows the vault's axis values, specifically A2 = `yes`.

(2) Defender model. A watchtower with recovery authority (keyless for CCV, `recoveryauth` for OP_VAULT). The defender-side degree of freedom is the batching flag $b \in \{0, 1\}$: single-input recovery ($b = 0$) scales linearly in UTXO count; batched recovery ($b = 1$) has sub-linear amortisation. Defender policy also matters: a minimal-disruption policy (reactive per-UTXO recovery) maximises attacker leverage, while a

full-sweep-on-first-alert policy collapses the class to one round at the cost of operational disruption.

(3) Economic preconditions. Rate-dependent; the single-block-feasibility threshold $r^{\dagger}_{\mathrm{TM4}}(V, D, b)$ from Equation 5.5 is the governing parameter. Below $r^{\dagger}$ the defender absorbs the per-UTXO liability before dust economics kick in. Above $r^{\dagger}$ the attacker can drive every UTXO below the economic abandonment threshold within a single block window.

(4) Protocol / network assumptions. Bitcoin dust thresholds: 546 sats P2WPKH, 330 sats P2TR. RBF Rule 3 does not permit sibling-eviction, so the attacker's partial-withdrawal chain is single-chain (each partial unvault consumes the previous output). Minimum relay fee $\approx 1\,\mathrm{sat/vB}$ bounds the attacker's per-tx cost from below. Mempool min-fee during congestion can float above the attacker's preferred rate; this is a scope modifier but not a sufficient mitigation.

(5) Scope of validity. Applies iff A2 = `yes`. The class is a *structural cost tail of partial withdrawal*, independent of adversary behaviour; an attacker accelerates the distribution but is not required for the class to manifest. Legitimate operational partial withdrawal can produce the same distribution over time. Does not apply to CTV or CAT+CSFS (A2 = `no`).

(6) Rational-attacker check. Rational only above $r^{\dagger}$ and only with external incentive (competitor, extortion, state actor). A pure liveness attacker is irrational at any r — no financial gain. An attacker who holds the hot key *and* drives the vault below $r^{\dagger}$ can extract value from UTXOs the watchtower abandons, assuming CSV expiry on those UTXOs.

(7) Counter-factual mitigations. Three distinct counter-factuals:

1. **Defender batching.** Raises $r^\dagger$ by 1.8–2.7× (Chapter 4). Materially absorbs the tail under moderate congestion. Requires BIP-345 §"Batching" or BIP-443 default aggregation support in the watchtower implementation.

2. **Full-sweep-on-first-alert.** Collapses the multi-round loop to one round at the cost of forcing the owner to re-deposit. Appropriate when the operational cost of a full sweep is less than the expected loss from the cost tail.

3. **Operational limits on per-epoch trigger count.** Not a protocol-level mitigation, but a watchtower configuration that triggers a full sweep after k unauthorised triggers per unit time limits the attacker's burst capacity.

Reframing note. Earlier framings of this class as "watchtower exhaustion" cast it as a multi-round attacker-vs-watchtower game. That framing does not survive a defender with full recovery authority who can full-sweep on first alert, collapsing the multi-round narrative to one round. The reframe above identifies the class as a structural cost tail of partial withdrawal — attacker-accelerable but not attacker-required — with the defender's degree of freedom in recovery batching.

C.5 Class Z4: Hot-Key Theft via Destination Substitution

(1) Attacker capability. The hot (trigger) key; mempool access; optional fee-pinning capital (if class Z1 composes, as it does for CTV).

(2) Defender model. An online watchtower able to race the attacker's unvault-to-withdraw sequence during the CSV delay. The race bias tilts toward the attacker when class Z1 composes (CTV with compromised fee key); otherwise the defender wins the race generically by broadcasting the recovery transaction with higher fees.

(3) Economic preconditions. Race outcome is r-independent except via class Z1 composition, which forces the watchtower's CPFP recovery to fail regardless of fee rate.

(4) Protocol / network assumptions. CSV delay length; mempool propagation dynamics. Riard's replacement cycling attack [36] is adjacent but out of scope here; it would compose with class Z4 in the same way that class Z1 composes, but is a mempool-layer rather than a script-layer mitigation target.

(5) Scope of validity. Applies iff A4 is one of {`template-hash-digest`, `per-output-amount+script`, `dedicated-opcode`} — i.e., any value other than `signature-dual-bind`. CAT+CSFS, uniquely, has A4 = `signature-dual-bind` and is immune by construction.

(6) Rational-attacker check. Rational only paired with fee-pinning budget (composition with class Z1) or against a defender with ineffective watchtower response. Otherwise the defender wins the race generically and the attack yields griefing only (which is irrational without external incentive per class Z2).

(7) Counter-factual mitigations. A4 = `signature-dual-bind` eliminates the class by cryptographic construction. The CAT+CSFS dual-verification pattern forces the assembled sighash preimage to equal the consensus sighash preimage under Schnorr EUF-CMA secu-

rity [28]: a valid signature over two distinct messages under the same key would reduce to discrete-log. The `hashOutputs` field is a component of both preimages, so the output set is forced. A hot-key holder attempting to redirect the withdrawal produces a rejected transaction, not a theft; the outcome degrades to griefing-only.

C.6 Class Z5: Cold-Key Theft via Unconstrained Recovery

(1) Attacker capability. The cold (recovery) key. Kerckhoffs: we assume compromise, not inference — the attack does not derive the cold key from observable data.

(2) Defender model. Any; the attack is instant with no CSV window and no watchtower intervention window. The cold-key-signed recovery transaction is valid the moment it is broadcast and there is no script-level check that would reject an attacker-chosen destination.

(3) Economic preconditions. r-invariant.

(4) Protocol / network assumptions. N/A — pure signature check; no mempool dynamics. The attack completes in one transaction with no race window.

(5) Scope of validity. Applies iff A5 = `plain-signature`. The only covenant with this axis value is CAT+CSFS, whose recovery leaf is a bare `cold_pk OP_CHECKSIG` with `SIGHASH_DEFAULT` and no script-level output constraint.

(6) Rational-attacker check. Rational whenever cold key is compromised; yield V (full vault theft), cost negligible.

(7) Counter-factual mitigations. A5 = `signature-dual-bind` on the recovery leaf eliminates the class. Poelstra's sketch [34] describes a CAT+CSFS-based bound-recovery construction that would move CAT+CSFS to this axis value; it is not in the reference CAT+CSFS vault implementation that this thesis measures. Alternative A5 values (`template-hash-digest` as in CTV, `per-output-amount+script` as in CCV, `dedicated-opcode` as in OP_VAULT) also produce immunity. The class-Z5 surface of CAT+CSFS is therefore the *one structural cost* of its class-Z4 immunity, and it is addressable without moving off CAT+CSFS.

C.7 Class Z6: Mode/Parameter Bypass

(1) Attacker capability. Any observer of a vault whose script contains an undefined mode byte; no keys required.

(2) Defender model. Developer discipline around mode-byte selection; library-provided named constants (e.g., pymatt's `CCV_FLAG_*`); code review.

(3) Economic preconditions. None applicable — full vault bypass at constant 110 vB/spend (measured). The class is insensitive to vault value and fee rate.

(4) Protocol / network assumptions. BIP-341 and BIP-342 taproot `OP_SUCCESSx` semantics: an undefined opcode value in tapscript terminates execution with success regardless

of stack state. This is the forward-compatibility mechanism that permits future soft-forks to assign meaning to currently-undefined bytes without invalidating existing transactions.

(5) Scope of validity. Applies iff A6 = `multi-mode-with-OP_SUCCESS`. The only covenant with this axis value is CCV (BIP-443). CTV, OP_VAULT, and CAT+CSFS all have A6 = `single-mode` and expose a single semantic opcode with no mode selector.

(6) Rational-attacker check. Rational whenever a misconfigured vault exists on-chain. The attacker's payoff is V (full vault theft), the cost is a single spend of $110\,\text{vB} \times r$. The class is a developer-error surface, not a cryptographic vulnerability.

(7) Counter-factual mitigations. Three distinct counter-factuals:

1. A6 = `single-mode`. One opcode per primitive (as in the other three proposals) eliminates the class by construction.
2. Bitmask-gated mode check at script-parse time. Reject undefined modes before execution; eliminates the class without revising soft-fork semantics, but forfeits the forward-compatibility benefit.
3. Library-provided named constants. Makes the undefined-mode surface unreachable from application code; residual risk in independent implementations that hand-write the mode byte. This is the pymatt approach and is the recommended mitigation for CCV deployments.

Framing note. The class is *specified behaviour* under BIP-443's forward-compatibility design (inherited from BIP-341/BIP-342), not a newly-discovered vulnerability. The thesis

contribution here is the systematic mode sweep and the class-level framing that ties the surface to axis A6, not the discovery of `OP_SUCCESSx` semantics.